PRACTITIONER'S GUIDE

Causal Thinking with TP Studio

A practitioner's guide to the Theory of Constraints, illustrated end-to-end with TP Studio.

GENERATED 2026-06-02

TP-STUDIO.STRUKTURERETSUNDFORNUFT.DK

Contents

Foreword

> *Last reviewed against TP Studio v0 (schema v8), Session 133.*

The Theory of Constraints has two literatures and one tool tradition.

The first literature is the novels — *The Goal*, *It's Not Luck*, *Critical Chain*. Goldratt taught the method the way method actually transmits: through characters, through frustration, through the slow turn from "things are just hard" to "things are this way for a *reason*, and the reason has a shape." If you have never read those books, put this one down and read them first. They will teach you why the trees in Part 2 of this book are shaped the way they are. Nothing here substitutes for that grounding.

The second literature is the academic and consulting work — *The Theory of Constraints Handbook*, the AGI white papers, Cox & Schleier's compendium. Rigorous, comprehensive, dense. The trees are explained one notation at a time. By page two hundred you have a vocabulary that works.

Between those two sit the practitioners. They have read Goldratt. They have skimmed the handbook. They have a problem at work. They want to build a Current Reality Tree on Wednesday afternoon, not next quarter. They want to know how to draw the cloud, how to verbalize the entry condition, what to do when the diagram looks wrong, when to stop. The novels teach the why; the handbook teaches the rigor; the practitioner-facing material teaches *how to actually sit down and do it*.

That third corner is what this book is for.

The tool tradition is what makes the third corner viable. Drawing a CRT by hand teaches you a lot, but it doesn't scale past the first few revisions. Drawing one in PowerPoint is a special kind of hell. The TOC software ecosystem developed over the last two decades has given practitioners a canvas; a body of practitioner-facing writing has shown how to use it. TP Studio is an open-source, local-first canvas in that tradition; this book is a practitioner companion written specifically for it.

You will get the most out of this book if you read it with TP Studio open in another window and a real problem on your desk. The exercises do not work in the abstract. Pick a domain you understand — a system that frustrates you at work, a recurring failure mode in a project you are running, a strategic question you cannot get straight in your head — and build the diagrams alongside the chapters. By the end you will have constructed at least one Current Reality Tree and at least one Evaporating Cloud for that problem; if you have done the exercises honestly, you will know something about the problem that you did not know when you started.

The TOC philosophy holds that the people closest to a system are usually capable of analyzing it correctly, given a framework to think in. This book does not aim to teach you anything you do not already know about your own work. It aims to teach you a notation in which what you know becomes legible — to you first, then to others.

Thank you for picking this up. Now open the application and let us begin.

→ Continue to Chapter 1 — The system has a goal

Colophon

Chapter 1 — The system has a goal

> *If you skip this chapter, the rest of the book will still work mechanically. But the diagrams in Part 2 will feel like notation exercises rather than thinking. Read this chapter even if you're TOC-fluent — the vocabulary established here is what the rest of the book leans on.*

The premise

Every Theory of Constraints analysis rests on one assumption, and it is the assumption that distinguishes TOC from the rest of process-improvement.

> ***The premise:*** *Every system has a goal, and every system's performance is limited by one thing — the constraint. Improving anything else does not improve the system. Improving the constraint does.*

That second sentence is the part most people skip past on first read, and the part that matters most. TOC is a *constraint-centric* view of systems. Everything else — the diagrams, the validators, the cloud, the focusing steps — is machinery for finding and dismantling constraints.

This sounds obvious until you actually try to make the obvious version stick in a room of people who are used to thinking about averages, about local efficiency, about doing-more-of-the-good-stuff. The obvious version, when taken seriously, has consequences:

- Improving anything that isn't the constraint produces no system-level gain.
- The constraint is almost never where people think it is.
- The constraint can be physical, but more often it is policy, behavior, or measurement.
- Once you fix one constraint, a new one appears — somewhere else in the system. The work never ends; the leverage just relocates.

The Thinking Processes are the toolset for finding and addressing constraints that aren't physical — and that's almost all of them, in the modern white-collar economy. A constraint of "we don't have enough drilling machines" is something a budget solves. A constraint of "our incentive structure rewards individual heroics and punishes the documentation that would prevent them" needs a Thinking Process. That's most of what we do in real organizations, and that's the gap the trees in Part 2 are built for.

Naming the gap

A constraint is only worth analyzing because it holds some measure short of where it should be. That distance — between where a measure sits today and where you want it — is the *gap*, and every tree in this book is ultimately in service of closing one. It pays to name the gap out loud before you draw anything, so the whole analysis has an anchor to be measured against.

TP Studio gives the gap a home on the document itself. The **"Performance frame"** section of the Document panel holds two optional anchors: a **Low** note — the measure's current, unacceptable level — and a **High** note — its target or desired level. "On-time delivery at 60%" → "Reach 98% within two quarters." It's general to every diagram type, not tied to any particular tree, and it travels with the document, so whoever opens the file later sees the gap the analysis was built to close stated in plain numbers rather than implied by the diagram. Filling it in is a small facilitation discipline that keeps a long investigation honest about what "done" would mean.

The Five Focusing Steps

Goldratt's canonical sequence for working with constraints. Worth keeping in mind throughout the rest of the book, because *every* thinking-process tree is in service of one of these steps.

1. **Identify** the system's constraint.
2. **Exploit** the constraint — make it work to its full current capacity before adding more.
3. **Subordinate** everything else to the constraint — local optimums that fight the constraint are net-negative for the system.
4. **Elevate** the constraint — invest, change, expand. Only when 2 and 3 are exhausted.
5. **Go back to step 1** — the constraint will have moved.

The Thinking Processes map cleanly onto these steps:

Thinking Process	Maps to	What it gives you
Current Reality Tree	Identify	The actual constraint, traced from symptoms backward.
Evaporating Cloud	Identify (the *policy* constraint)	The conflict that holds the current reality in place.
Future Reality Tree	Elevate	The system as it would be once the constraint is broken.
Prerequisite Tree	Subordinate / Elevate	The obstacles to getting from current to future.
Transition Tree	Subordinate / Elevate	The sequenced actions that dismantle the obstacles.
Goal Tree	Step 1 *before* the iteration starts	The objective the constraint is interfering with.
Strategy & Tactics Tree	Elevate (when the elevation is itself a large program)	Deployment-grade decomposition of strategy into tactics.

You will not use all of these every time. Most analyses center on a CRT, dissolve one or two clouds, draft an FRT, and stop there. The PRT and TT come out only when implementation needs sequencing. The Goal Tree is usually the *frame* you draw around the whole investigation, not a separate analysis. S&T is for larger programs — multi-team rollouts, strategy decomposition, the kind of thing you don't sit and draw on a Tuesday afternoon.

Vocabulary you'll need

This book uses the standard TOC vocabulary throughout. Most of these words mean what you'd guess they mean; a few have specific TP Studio analogues worth pinning down once.

Term	Meaning	TP Studio analogue
UDE — Undesirable Effect	A symptom the system produces that nobody wants. The starting point of a CRT.	Entity type `undesirableEffect`, red stripe.
Root Cause	A terminal cause at the bottom of a CRT — the leverage point.	Entity type `rootCause`, amber stripe.
Effect	An intermediate node — caused by something, causing something else.	Entity type `effect`, neutral grey stripe.
Injection	A change you propose to make. The hypothesis you'd test.	Entity type `injection`, emerald stripe.
Desired Effect	What the system would produce instead, after the injection lands.	Entity type `desiredEffect`, indigo stripe.
Assumption	A claim that an arrow in the diagram depends on — and that someone could plausibly challenge.	Entity type `assumption`, violet stripe; surfaces in the AssumptionWell on EC docs.
CLR — Categories of Legitimate Reservation	The six discipline-checks that distinguish a good causal claim from a sloppy one.	The validator system; warnings surfaced in the inspector.
Core Driver	The root cause that ladders up to the most UDEs. The constraint, in CRT form.	The reach-badge value; the `Find core driver(s)` palette command.
EC — Evaporating Cloud	A 5-box conflict diagram showing why a chronic problem is *held in place* by a real tension between two real needs.	Diagram type `ec`.
CSF — Critical Success Factor	A must-be condition for a Goal. The middle layer of a Goal Tree.	Entity type `criticalSuccessFactor`.
NC — Necessary Condition	A sub-condition feeding a CSF. The lower layer of a Goal Tree.	Entity type `necessaryCondition`.
AND group	A set of causes that are individually necessary but only jointly sufficient. Rendered as a junctor circle in TP Studio.	The `andGroupId` field on edges; JunctorOverlay renders it.
Sufficiency vs. necessity	A cause is *sufficient* if it alone produces the effect; *necessary* if the effect can't happen without it.	Edge `kind: 'sufficiency'` vs. `'necessity'`.
CRT — Current Reality Tree	The "Why is it this way?" diagram. Bottom-up causality.	Diagram type `crt`.
FRT — Future Reality Tree	The "What would it look like solved?" diagram. Same causal model, different starting node — an injection instead of a root cause.	Diagram type `frt`.
PRT — Prerequisite Tree	"What's in our way?" The obstacles between current and future, paired with intermediate objectives.	Diagram type `prt`.
TT — Transition Tree	"How do we get there?" Action / precondition / outcome triples.	Diagram type `tt`.
S&T — Strategy & Tactics Tree	A deployment decomposition; each node is a 5-facet card (Necessary Assumption, Strategy, Parallel Assumption, Tactic, Sufficiency Assumption).	Diagram type `st`.

Term	Meaning	TP Studio analogue
Verbalisation	Reading a diagram aloud, edge by edge, in natural language. The discipline that catches what scanning silently misses.	The Verbalisation Strip; the walkthrough overlay; the reasoning narrative export.
Browse Lock	A read-only mode you turn on before sharing or demoing the doc to prevent accidental edits.	The lock icon in the TopBar; the `browseLocked` flag.

Why a tool

You can do a CRT on paper. You can do an EC on a whiteboard. Practitioners have done so for decades. The reason a tool helps is *iteration*.

A CRT done well is not the diagram you draw on Monday. It is the diagram you have on Friday — after you've moved entities around five times, deleted three you thought were causes but turned out to be re-statements of the UDE, found the cloud that the conflict was hiding behind, added the assumption you didn't realize you were making, run the verbaliser and caught the gap, captured a revision, branched to explore an alternative, compared the two side by side, and came back to the original.

A tool — any tool — collapses the marginal cost of those revisions. Whiteboards have erasers, but erasers don't preserve the prior state. PowerPoint preserves state but punishes restructuring. A purpose-built TOC canvas lets you treat the diagram as something *you're allowed to be wrong about*, repeatedly, cheaply.

TP Studio is one such canvas. Where comparable practitioner guides have assumed Flying Logic, this book assumes TP Studio. Most of what follows would transpose to another canvas with light edits. What is specific to TP Studio is mostly *which palette command you press*, not *what you're doing when you press it*.

What's next

The next chapter (Your first canvas) is the 30-minute tool tour. After that, Chapter 3 — Reading a diagram covers the notation: causality conventions, edge polarity, AND / OR / XOR, the CLR. Then Part 2 opens with the Current Reality Tree, which is where the real work begins.

🔁 **Chain to next:** orient yourself to the surface, then learn to read what you'll later draw, then start drawing.

→ Continue to Chapter 2 — Your first canvas

Chapter 2 — Your first canvas

> *30-minute hands-on. You're going to open TP Studio, create an entity, connect two of them, and inspect the result. No method content here — pure orientation to the surface so the rest of the book can reference TopBar buttons and palette commands by name without you stopping to look them up.*

Opening the application

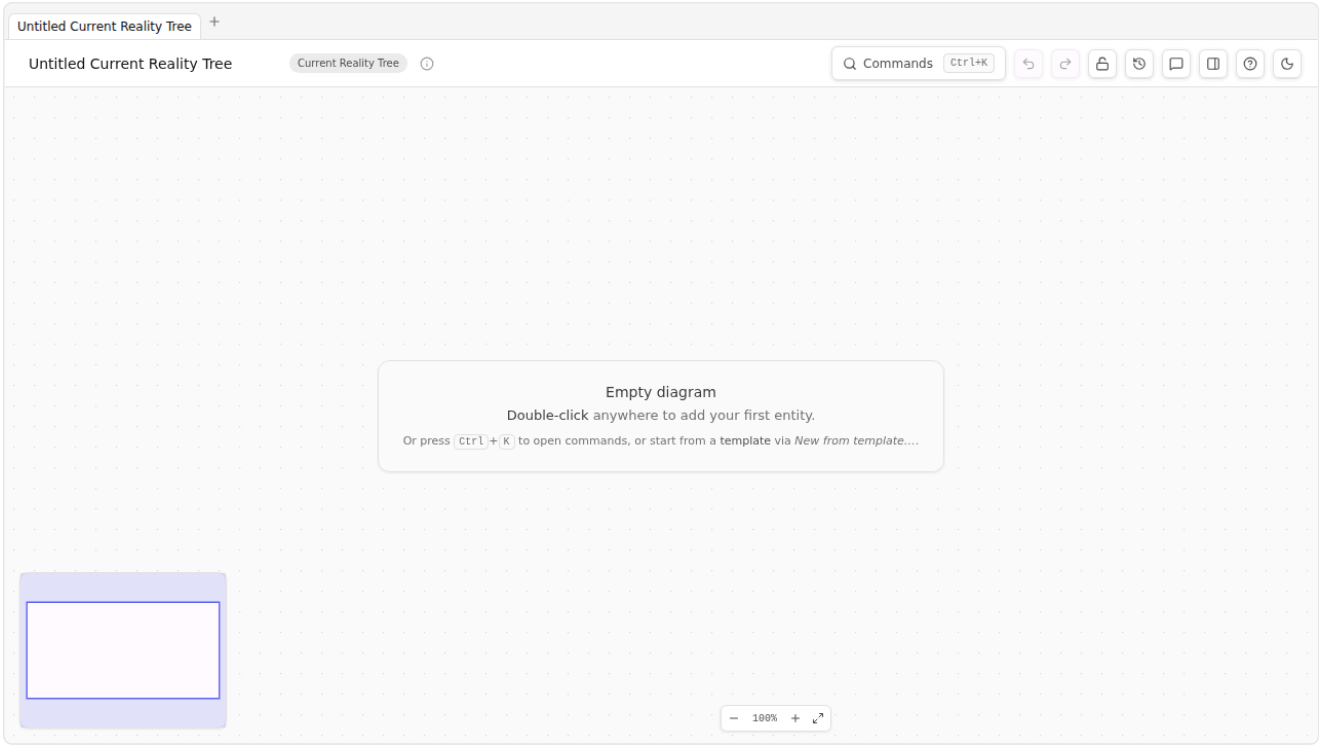
TP Studio runs in a browser tab. Three ways in:

- **The hosted PWA** at https://tp-studio.struktureretsundfornuft.dk/. Works offline after first visit; nothing leaves your machine.
- **A local dev server** if you cloned the repo: `pnpm dev`, then open the URL it prints (usually `http://localhost:5173`).
- **A local preview** of the built bundle: `pnpm preview` after `pnpm build`.

Pick the hosted PWA if you're a reader rather than a contributor. The app is identical either way.

Updates. When a new version of TP Studio ships, a small "New version available — Refresh now" toast appears at the bottom of the canvas; click it to apply. The flow is explicit on purpose: silent reloads while you're mid-edit would be hostile. If you want to check on demand (e.g. after a known release), `Cmd/Ctrl+K → Check for updates` forces the service worker to look — it tells you either *"You're on the latest version of TP Studio"* (green) or surfaces the refresh prompt for a new build (info).

The first time you load TP Studio you'll see an empty canvas with a centered hint:



> ***Empty diagram*** *Double-click anywhere to add your first entity.*

Your work auto-saves to this browser on every change. Closing the tab and reopening it picks up where you left off. No sign-in, no cloud, no upload — your diagrams live in this browser's `localStorage` and nowhere else. (Sharing with others is a separate step covered in Chapter 16.)

What's on screen

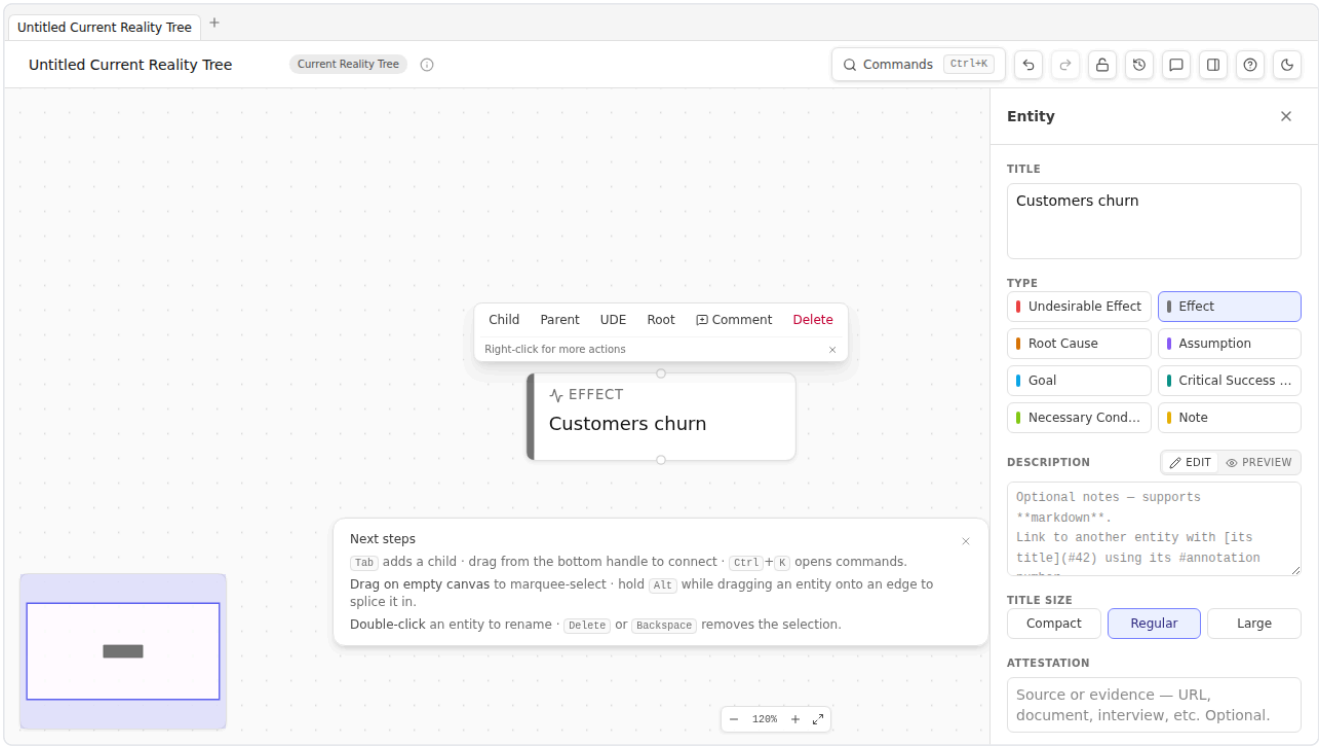
Element	Where	What it does
Title	Top-left	Click to rename the document. A small `CRT` / `FRT` / `EC` / etc. badge next to it shows the diagram type.
Commands button	Top-right	Opens the command palette. Same as `Cmd/Ctrl+K`.
Lock button	Top-right	Toggles Browse Lock — read-only mode. Use it before screen-sharing or demoing.
History button	Top-right (sm+ screens)	Toggles the Revision Panel slide-in.
Help button (?)	Top-right	Opens **Help** — the User Guide and practitioner book, plus keyboard shortcuts & gestures.
Theme toggle	Top-right	Light ↔ dark. (High-contrast variants live in Settings.)
Canvas	Center	The infinite dot-grid where your diagram lives. Pan with middle-click drag or two-finger scroll; zoom with the wheel or `+` / `-`.
Zoom controls	Bottom-left	Zoom in, zoom out, fit-to-view.
Minimap	Bottom-right (when enabled in Settings)	An overview with a viewport indicator.
Inspector	Right panel	Slides in when you select an entity or edge. Holds title, type, description, attestation, span-of-control, attributes, and warnings.
Toaster	Bottom-center	Brief confirmations: "Saved", "Loaded example CRT", "3 open CLR concerns".

Creating your first entity

Double-click the empty canvas anywhere. A blank node appears with the title field already focused. Type a short phrase — a noun-phrase describing something true about your system — and press Enter.

For this walk-through, type: **Customers churn**

Press Enter. The entity commits. You should see this:



That's an entity. By default it's an `effect` type — neutral grey stripe. The type tells you what role this node plays in the causal model; we'll get into types in Chapter 4.

Click the entity to select it. The Inspector slides in from the right with everything about this entity laid out: title, type grid, description, the rest. Try changing the type to `Undesirable Effect` by clicking the red-striped tile in the Inspector's Type grid. Notice the entity's stripe colour change on the canvas.

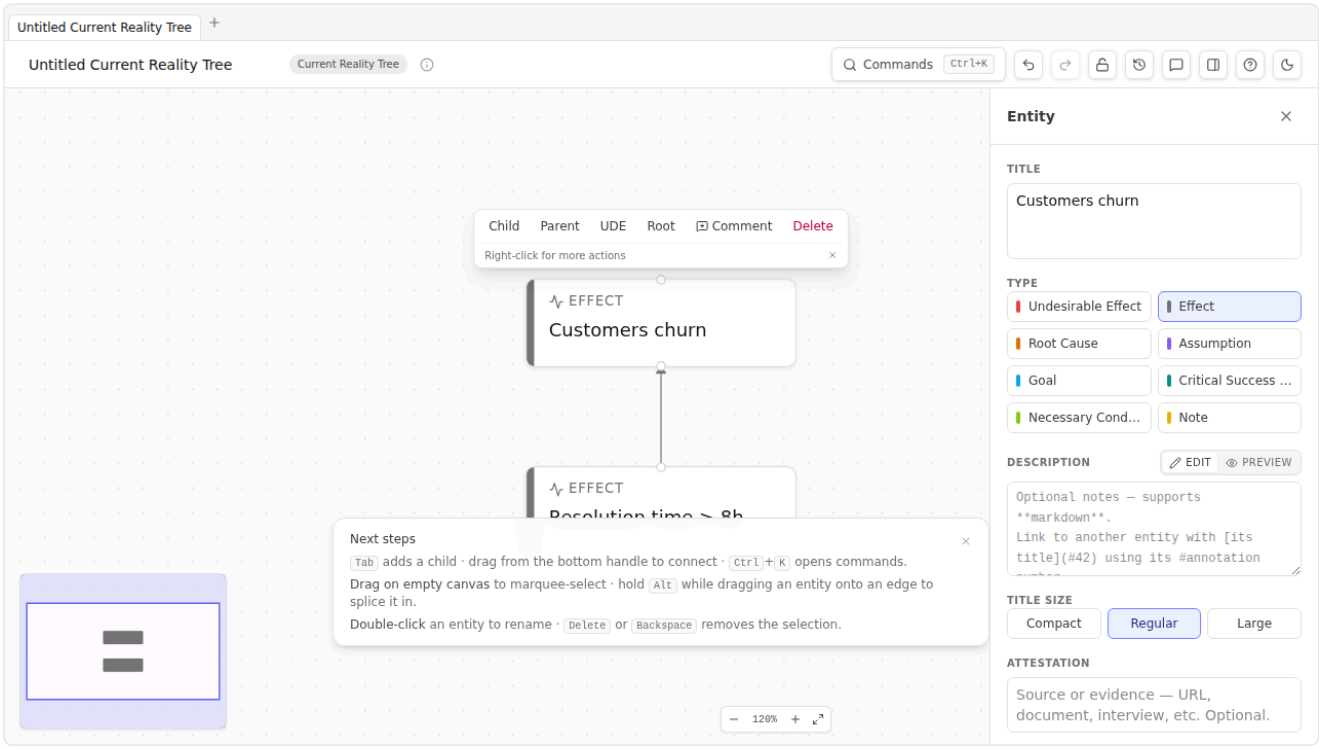
Connecting two entities

Double-click the canvas somewhere below the first entity. Type **Resolution time exceeds 8h** and press Enter. You now have two entities, side by side.

To connect them: hover your mouse near the bottom of an entity — small handle dots appear on the top and bottom edges. Click and drag from the bottom handle of "Resolution time exceeds 8h" up onto "Customers churn". Release. An arrow appears.

Or — faster — select "Resolution time exceeds 8h" by clicking it, then `Alt+click` "Customers churn". TP Studio interprets Alt-click as "connect from current selection to clicked node".

Your canvas now looks like this:



Read the arrow aloud: *"Resolution time exceeds 8h" causes "Customers churn".* That's the CRT reading convention — bottom-up, cause to effect, `because` linking the upper to the lower. (FRT, EC, PRT, and TT each have their own conventions; we cover them in Chapter 3.)

The command palette

`Cmd+K` (Mac) or `Ctrl+K` (Windows / Linux) opens the command palette — the canonical way to do anything in TP Studio that isn't a direct canvas gesture. Try it now. You'll see a search box and a list of commands grouped by category: File, Edit, Diagram, Layout, Export, Settings.

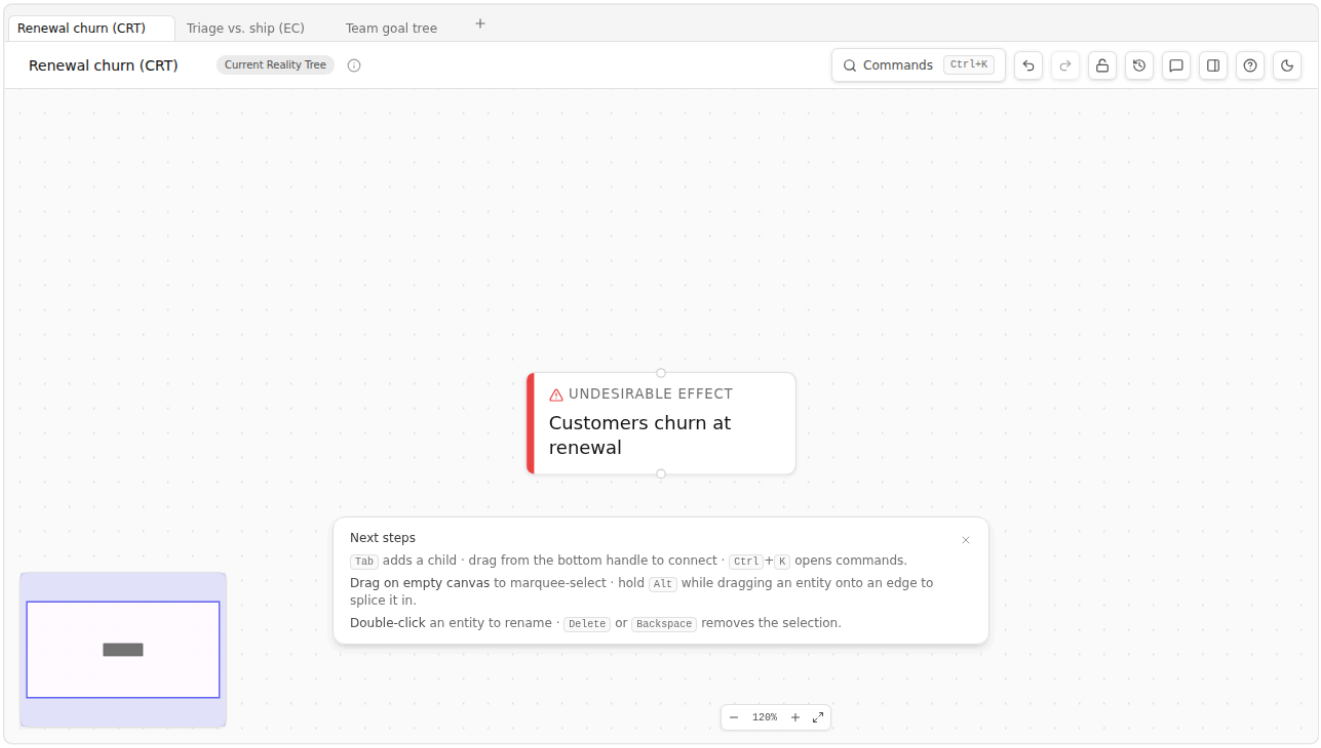
Type **Help** to filter. The top result is "Show keyboard shortcuts". Press Enter to open the Help dialog — that's the reference for every shortcut the application exposes.

A few commands worth memorizing today:

Type into palette	What it does
`New diagram`	Open the picker for fresh CRT / FRT / PRT / TT / EC / Goal Tree / S&T / Freeform docs.
`Load example`	Open the picker for canned example docs in every diagram type.
`New from template`	Open the curated templates library (10 specs covering Goal Trees / ECs / CRTs).
`Export`	Open the unified Export Picker (PNG / SVG / JPEG / PDF / Markdown / OPML / DOT / Mermaid / VGL / Flying Logic XML).
`Capture snapshot`	Save a revision; the History panel will then let you compare or restore.
`Copy read-only share link`	Generate a URL that encodes the entire doc and load it elsewhere in read-only mode.
`Show keyboard shortcuts`	The full key reference.

Working with multiple documents

TP Studio keeps several documents open at once, each in its own **tab** along the top of the canvas. Click a tab to switch; click the **+** to open a fresh CRT; hover a tab and click **✕** to close it; drag a tab to reorder. Closing the last tab leaves a new blank one — there is never zero.



Each tab is independent: its own undo history, autosave, and share link. Reloading the browser restores *every* open tab, not just the active one.

Opening a document — an import, a pattern, a template, an example, a shared link, or an Evaporating Cloud spawned from a conflict — opens it in a *new* tab by default, leaving your current work in place. Prefer the old "replace what's open" behaviour? Turn off **Settings → Behavior → "Open documents in new tabs"** (Appendix D).

The palette (`Cmd/Ctrl+K`) carries **New / Duplicate / Close / Next / Previous tab** and **Forget closed documents** (reclaims storage from closed docs). Installed as an app, the native **`Cmd/Ctrl+T`** / **`+W`** / **`+1`** – **`9`** keys work too (Appendix B).

Saving, exporting, sharing — the one-paragraph version

You don't save. TP Studio saves continuously to localStorage. Close the tab; reopen; your work is there.

You export by opening the Export picker (`Cmd+K → Export`). The picker shows everything in three groups: Images (PNG / SVG / JPEG / PDF / print preview), Markup (Markdown / OPML / DOT / Mermaid / VGL / Flying Logic XML), and Workshop (one-page EC sheet PDF, standalone HTML viewer, JSON, reasoning narrative / outline).

You share by either:

- Exporting the standalone HTML viewer (one file, no network, open by double-clicking on any machine); or
- Generating a read-only share link (`Cmd+K → Copy read-only share link`) — a URL that contains the whole doc encoded into its fragment.

Sharing is fully covered in Chapter 16.

Try it

Five minutes. No reading.

1. Create three entities with double-click. Give them placeholder titles ("A", "B", "C" is fine).
2. Connect A → B → C using Alt+click.
3. Open the Inspector for B and change its type to `Root Cause`. Watch the stripe colour change.
4. Press `Cmd+K → Capture snapshot`. Name the revision "After type change".
5. Delete C. Notice the toast at the bottom.
6. Press `Cmd+Z` to undo. C comes back.
7. Open the History panel (TopBar history button on `sm+` screens, or `Cmd+K → Open history panel`). You'll see your "After type change" snapshot.

You now know more about the surface than you can remember reading. That's the point of the chapter.

Where this lives in the rest of the book

- Diagram-type-specific instructions live in the relevant Part 2 chapter (CRT in 4, EC in 5, etc.).
- Notation conventions live in Chapter 3.
- The CLR validators are Chapter 13.
- Revisions and side-by-side compare are Chapter 14.
- Exports, share links, and prints are Chapter 16.
- Every keyboard shortcut is in Appendix B.
- Every Settings toggle is in Appendix D.

🔁 **Chain to next:** you can drive the surface. Next you need to be able to *read* what you draw on it.

→ Continue to Chapter 3 — Reading a diagram

Chapter 3 — Reading a diagram

> *The notation chapter. Nothing here is original to TOC — these are the conventions Goldratt's tradition has settled on over forty years. The chapter is short because it's reference-shaped: come back to it whenever a later chapter says something like "necessity edge" or "AND junctor" and you need a quick refresher.*

Causality flows up (mostly)

In Current Reality Trees and Future Reality Trees, **causality flows from bottom to top**. Causes sit below; effects sit above. An arrow points *from* the cause *to* the effect. Read aloud: *"Because [cause], [effect]."* Or equivalently: *"[Cause] therefore [effect]."*

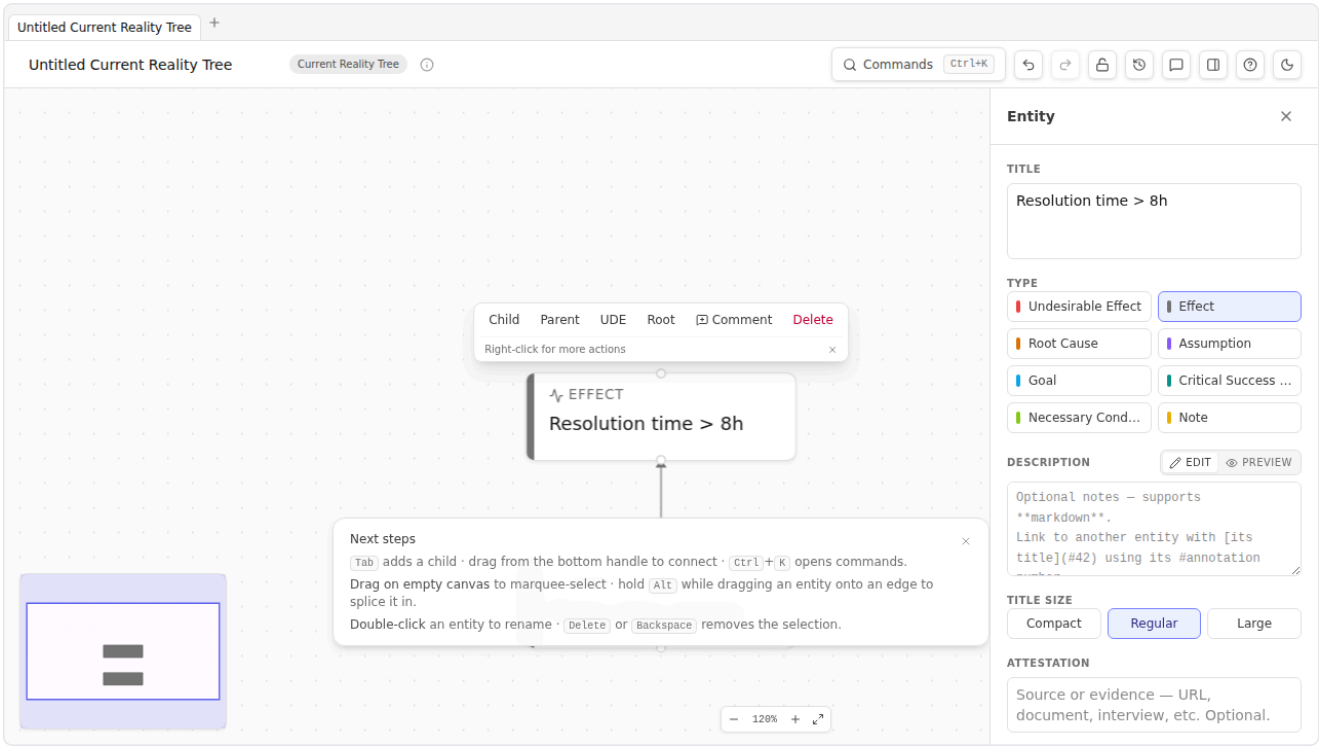
In Prerequisite Trees, Transition Trees, and Evaporating Clouds, the orientation is different — covered chapter-by-chapter. The point worth knowing now: **TP Studio respects per-diagram orientation conventions automatically.** When you load a CRT, dagre lays it out bottom-up. When you load a PRT, dagre lays it top-down. When you load an EC, you get a hand-positioned 5-box layout. Don't fight the convention; the convention is what makes the diagram *readable* to other practitioners.

Causality reading mode

The most common confusion when first encountering a CRT is "do I read up or down?" Goldratt's tradition says read up: *"because A, B; because B, C."* Some practitioners prefer the dual: *"A, therefore B; B, therefore C."* Both are valid.

TP Studio lets you pick a global default in **Settings → Display → Causality reading**:

Mode	What it does
`none`	No fallback label on edges. Best for clean exports.
`because`	Renders a muted italic "because" on each unlabeled edge. Reads bottom-up.
`therefore`	Renders a muted italic "therefore" on each unlabeled edge. Reads top-down.
`auto` *(default)*	Picks per-diagram: `because` for CRT / FRT / TT; `in order to` for PRT / EC; nothing for freeform / S&T.



Per-edge labels always override the global fallback. If you've explicitly labeled an edge — say, "because the SLA target is 8h" — the fallback word disappears and your label renders instead. Aggregated edges (those representing several edges collapsed across a group boundary) skip the fallback too; their `×N` count badge is the more informative thing.

Sufficiency vs. necessity

The most consequential distinction in TOC notation, and the one most easily fudged.

Sufficient cause: the cause, by itself, produces the effect. *"Because A, B"* — A alone is enough. CRT and FRT edges are sufficient causes.

Necessary condition: the effect *requires* the cause; without the cause, the effect can't happen — but the cause alone is not enough. *"In order to obtain B, we must have A"* — A is required, but other things might also be required. EC and PRT edges are necessity edges.

TP Studio's schema makes the distinction explicit. Each edge has `kind: 'sufficiency' | 'necessity'`, set per-diagram-type by default. You can override per-edge in the Edge Inspector when you have a mixed-causality diagram.

The verbaliser uses this field to choose between "because" / "therefore" wording and "in order to" / "we must" wording. The CLR validators use it too: a *Predicted Effect Existence* check fires only on sufficiency claims, because "if A is true, B should also be true *somewhere we can see*" is a sufficient-cause prediction.

Edge polarity

Classical Goldratt CRT notation was binary: an arrow either exists or it doesn't. Practitioner tools, Flying Logic most prominently, extended the notation through the 1990s and 2000s to support **polarity** — does this cause *increase* or *decrease* the effect? TP Studio adopts that extended notation for compatibility with the practitioner tradition. Three values, plus a default:

Polarity	Visual	Meaning
`positive` *(default)*	Plain arrow, no badge	"More of A → more of B." The classic sufficient-cause arrow.
`negative`	Arrow with a rose `−` badge	"More of A → less of B." A is a *reducer*.
`zero`	Arrow with a neutral `∅` badge	"A is flagged as non-influential here." Common in iterated FRTs where a previously-suspected cause has been ruled out.

Polarity is set via the Edge Inspector's Polarity picker or by cycling on the selected edge with the `cycle-edge-polarity` palette command. The cycle order is default → positive → negative → zero → default.

Polarity matters mostly in FRTs and S&T trees, where the question "will this injection actually move the needle?" hinges on whether the chain is all-positive (good), mixed (worth thinking), or contains a negative downstream of your lever (problem).

AND / OR / XOR junctors

A causal arrow on its own claims *sufficiency*: the cause produces the effect. But many real causal patterns aren't single-arrow — they involve *combinations*. Goldratt's original CRT included AND-grouping; the explicit OR and XOR junctors come from the same practitioner-tool tradition that introduced polarity (above), where they were given visual conventions Flying Logic popularized. TP Studio adopts those conventions:

Junctor	Visual	Logical meaning
AND	Violet circle labeled `AND`	All inbound causes must be present for the effect to occur. The set is jointly sufficient; no individual is.
OR	Indigo circle labeled `OR`	Any one of the inbound causes is sufficient. The set is alternative.
XOR	Rose circle labeled `XOR`	Exactly one of the inbound causes occurs; mutually exclusive alternatives.

To group edges into a junctor: select the edges (use shift-click for multi-select), then `Cmd+K → Group as AND` (or `OR` / `XOR`). The selected edges are rewired through a single junctor circle just below the target node, with one short outgoing arrow from junctor up into the target.

Cross-kind exclusivity: an edge belongs to at most one junctor kind. If you group an AND-grouped edge into an OR, the AND group dissolves first.

You'll mostly use AND. It's the conjunctive that makes a CRT honest: if "Customers churn" requires BOTH "Resolution time > 8h" AND "Onboarding is poorly documented", saying "either causes churn" is overstating the strength of each. The AND junctor forces you to be honest about which combinations are sufficient.

Back-edges

Sometimes a diagram has a real cycle. Customers churning *causes* lower retention metrics, which *cause* leadership pressure on the support team, which *causes* deferred refactors, which *causes* the original "resolution time exceeds 8h", which *causes* churn. Round and round.

TP Studio supports flagging an edge as a **back-edge** — a deliberate loop-closer the practitioner has acknowledged as part of the structure, rather than a CLR bug. Back-edges render with a thicker dashed stroke and a small `↻` glyph mid-edge. The cycle CLR validator suppresses its warning for any cycle whose closing edge is back-tagged.

To tag: select the edge, Inspector → Back-edge toggle. Or right-click → "Toggle back-edge".

Most diagrams don't need back-edges. Use them when the structure is genuinely a positive or negative *reinforcing loop* (Senge's term) — a feedback structure you want to study, not a CRT cycle you accidentally drew.

Mutex (EC-only)

In an Evaporating Cloud, the two "Want" entities (D and D′) are *in conflict*. The whole point of the diagram is that you've identified a tension between two real, legitimate wants — they pull you in opposite directions.

TP Studio represents this conflict with a **mutex edge** between D and D′: rendered red with a lightning-bolt (⚡) glyph in the middle. It's the only edge in the diagram that's bidirectional in meaning ("they conflict" rather than "A causes B").

The `ec-missing-conflict` validator fires on any EC document until at least one want↔want edge is flagged as `isMutualExclusion`. Tag it via Edge Inspector → Mutual exclusion, or right-click → Toggle mutual exclusion.

Locus

A second extension Goldratt added: each entity carries a **Locus** flag (in earlier versions of TP Studio this was labelled "Span of Control") indicating who controls it.

Locus	Visual	Meaning
`control`	Emerald `C` pill	Inside your control. You can directly change this.
`influence`	Amber `I` pill	Outside your direct control, but you can plausibly influence. Vendors, neighboring teams, customers via product changes.
`external`	Neutral `E` pill	Outside your control AND your influence. Macro conditions, regulation, market forces.

Why it matters: **the constraint you want to find lives in your control or influence zone**, not in the external zone. The `external-root-cause` CLR validator fires on root causes flagged `external` — the diagram tells you you've found a "cause" that you can't actually do anything about, which usually means you've stopped one step short of the real root cause.

Set via Entity Inspector → Locus, or right-click → Set locus.

Ownership and validation

The Inspector also carries an **Owner** field — a free-form text input naming whoever's accountable for the entity. Decision owner on a UDE, action assignee on a TT action, validation owner on an assumption — whatever role makes sense for the entity's place in the diagram. Below the field is a **Mark validated** button that stamps a timestamp into `entity.lastValidatedAt`; on subsequent visits the timestamp reads back as "Last validated YYYY-MM-DD by <owner>" so an audit trail accumulates across re-validations.

Two consumers downstream:

1. The **Risk Register (CSV)** export (Chapter 16) reads the field directly — the `owner` column populates from `entity.owner`.
2. Future collaboration features will use the same field as the human-readable name for the "who" of accountability without needing a formal user model.

The legacy `attributes.owner` key still works for older docs (the risk-register export checks both, preferring the dedicated field), but new docs should use the typed Owner field.

Evidence — making provenance explicit

Beneath the Owner block is an **Evidence** list — a structured place to record *what the entity is standing on*. Each row carries:

- A **description** — the actual citation, observation, measurement, or claim. Free text.
- A **source** pill — cycles through `Observed → Stakeholder → Metric → Policy → Assumption`. The taxonomy is intentionally narrow:
  - **Observed** — first-hand observation ("I saw the queue grow during demo days")
  - **Stakeholder** — an assertion from someone with stake ("the CFO says renewals will drop 12%")
  - **Metric** — a numeric measurement ("p95 latency = 740ms last week")
  - **Policy** — a documented rule or constraint ("PCI requires segregated networks")
  - **Assumption** — explicitly an unproven belief, kept honest by the label rather than masquerading as fact
- A **strength** pill — `Weak / Moderate / Strong`. Your qualitative judgement; nothing derives it for you.
- An optional **URL** — citation link, dashboard URL, document reference.
- A per-row **Mark validated** button — stamps the current timestamp and credits the entity's Owner as the validator. Distinct from the entity-level validation: the entity audit-trail tracks "is this entity still true overall"; the per-evidence trail tracks "is THIS specific citation still standing."

Use the list to make provenance explicit at workshop time. A UDE backed by `[strong/metric] p95 = 740ms` reads very differently from one backed by `[weak/assumption] customers probably care about latency`, and the difference often surfaces during the trim step of an FRT — the moderate-evidence claims are the ones to challenge first.

Evidence items survive JSON export / share-link reload, and feed the new `evidence` column of the **Risk Register (CSV)** export — rendered as semicolon-joined `[strength/source] description (url)` entries, so a register reader gets the qualitative signal at a glance without losing the original detail.

Entity state and what-if speculation

Reading a diagram is one thing; *interrogating* it is another. TP Studio lets you assign each entity a **state** — the answer to "is this true right now?"

State	Badge	Meaning
`true`	Emerald `T`	The entity holds. The cause is present; the effect has occurred.
`false`	Rose `F`	The entity does not hold.
`unknown` *(default)*	Neutral `?`	Undecided — no claim either way.
`disputed`	Amber `?`	Stakeholders disagree; recorded as contested rather than resolved.

Set a state from Entity Inspector → State, or by cycling the selected entity. A small state badge then rides the node's corner.

What makes states more than annotation is **propagation**. TP Studio folds known states downstream along the causal structure: a `true` cause whose junctor is satisfied lights its effect `true`; a `false` member of an AND junctor blocks the effect; OR/XOR fold by their own rules; negative-polarity edges flip the signal; zero-polarity and back-edges are skipped; cycles short-circuit rather than loop forever. Derived states render in the same badges but without the manual marker, so you can always tell what *you* asserted from what the diagram *concluded*.

What-if speculation is the live version of this. `Cmd+K → Begin speculation` opens a temporary overlay: flip any entity's state and watch propagation re-flow across the canvas in real time, with speculated badges drawn with a dashed ring so they're visibly provisional. A banner offers **Commit** (write the speculated states back as the real ones, in a single undo step) or **Revert** (`Esc`) — discard the overlay and restore reality. Nothing touches the saved document until you commit.

Use it to ask the questions a static diagram can't: "If this assumption turns out false, what downstream effects collapse?" "Which root cause, flipped to true, lights up the most of the tree?" Speculation turns the diagram from a picture into a model you can poke.

The CLR — a one-paragraph preview

Six "Categories of Legitimate Reservation" — the discipline checks Goldratt taught for evaluating someone else's causal claim. TP Studio surfaces them automatically as **warnings** in the Inspector's Warnings list. They are not errors; they are reservations a thoughtful colleague would raise if they were reading your diagram over your shoulder. We get into the CLR in depth in Chapter 13, but for now know that:

- Warnings fire on entities AND edges, depending on what the rule checks.
- Tiers escalate from `clarity` (the most pedagogical) to `predictedEffect` (the most architectural).
- You can dismiss warnings explicitly when you've considered the reservation and decided it doesn't apply.
- The `Start CLR walkthrough` palette command iterates all open warnings one at a time with Resolve / Open-in-inspector actions.

Quick reference card

Print this and tape it next to your monitor:

```
CRT / FRT       — causality flows up.    "Because [below], [above]."
PRT             — IO → obstacle.          "To get [above], must overcome [below]."
TT              — action + precondition.  "Do [action] given [precondition] to obtain
[outcome]."
EC              — 5 boxes in fixed layout. Read aloud per Chapter 5.
Goal Tree       — Goal → CSFs → NCs.      Top-down decomposition.
S&T             — 5-facet cards.          Top-down strategic deployment.

Edge polarity:  default • positive • negative (−) • zero (⌀)
Junctors:       AND (violet) • OR (indigo) • XOR (rose)
Back-edge:      thick dashed + ↻ glyph
Mutex (EC):     red + ⚡ glyph
Span-of-ctrl:   C (emerald) • I (amber) • E (neutral)
Entity state:   T (true) • F (false) • ? (unknown/disputed) • dashed ring = speculated
```

🔁 **Chain to next:** you can read what's on the canvas. Now build one from scratch.

→ Continue to Chapter 4 — Current Reality Tree

Chapter 4 — Current Reality Tree

Why is this happening?

> 🎯 ***What this process is for*** *A Current Reality Tree (CRT) traces a system's painful symptoms — the Undesirable Effects — back to their underlying cause. It answers: "Why is the system producing this?" Done well, it identifies the* ***core driver*** *— the single root cause whose dismantling would eliminate most of the symptoms. That's the constraint.*

The premise

The CRT is the first Thinking Process you reach for, almost every time. It's the *diagnosis*. You don't fix anything by drawing a CRT — fixing happens later, in the Future Reality Tree and Transition Tree. But you can't fix what you can't see; the CRT is what you see *with*.

The premise that makes CRTs work is the one we set up in Chapter 1: every system's underperformance traces to one constraint. Most of what looks like multiple problems is one problem in different costumes. The CRT's job is to reveal that the multiple-symptom appearance is illusory and the underlying cause is singular.

When this premise holds, a CRT for a system with 7-10 visible UDEs typically resolves down to 2-4 root causes, with one of them feeding 70-80% of the UDEs by reach. That last one is the core driver. (If your CRT has 7 UDEs all tracing back to 7 independent root causes, you either drew it wrong or — rarely — you're looking at a genuinely young, small system that hasn't yet accumulated structural pathology. In the latter case, congratulations.)

The method, neutral of tool

1. **List the UDEs.** Talk to the people closest to the system. Ask: "What does this system do that you wish it didn't?" Aim for 5-12 noun-phrases, each describing a single observable effect. ("Customers churn." "Support cost per ticket is up 40%.") Avoid causes ("Because we don't have a triage rubric…") — those go in later.
2. **Pick one UDE and ask why.** For each cause-candidate, ask: "Is this present because of something else?" — and add that something else as a node below. Connect them with a sufficiency arrow upward.
3. **Repeat for every UDE.** Most causes will recur across UDEs — that's the structure you want. A cause that doesn't recur is suspicious; either it's specific to one UDE (which is rare in a real system) or you're stopping too shallow.
4. **Identify converging cause-chains.** When two UDEs trace back through different intermediate effects to the *same* root cause, you've found a structural pattern. Mark it. The convergence is the constraint becoming visible.
5. **Find the core driver.** The root cause with the highest *reach* — the most UDEs it ladders up to — is the candidate. TP Studio's `Find core driver(s)` palette command does this calculation explicitly.
6. **Verbalize.** Read each cause chain aloud, edge by edge. "Because A, B. Because B, C." If a sentence sounds wrong, the diagram is wrong. Fix the diagram.
7. **Stop when the root cause is in your control or influence.** If you bottom out at a cause you flag `external` (the geopolitical situation, the macroeconomy, the customer's mood), keep going one level — there's almost always a `control`-level cause beneath an `external` one. The system you're analyzing belongs to *you*, not the macroeconomy.

The worked example

We'll build a CRT for a fictional but realistic problem: **a B2B SaaS support team chronically firefighting**.

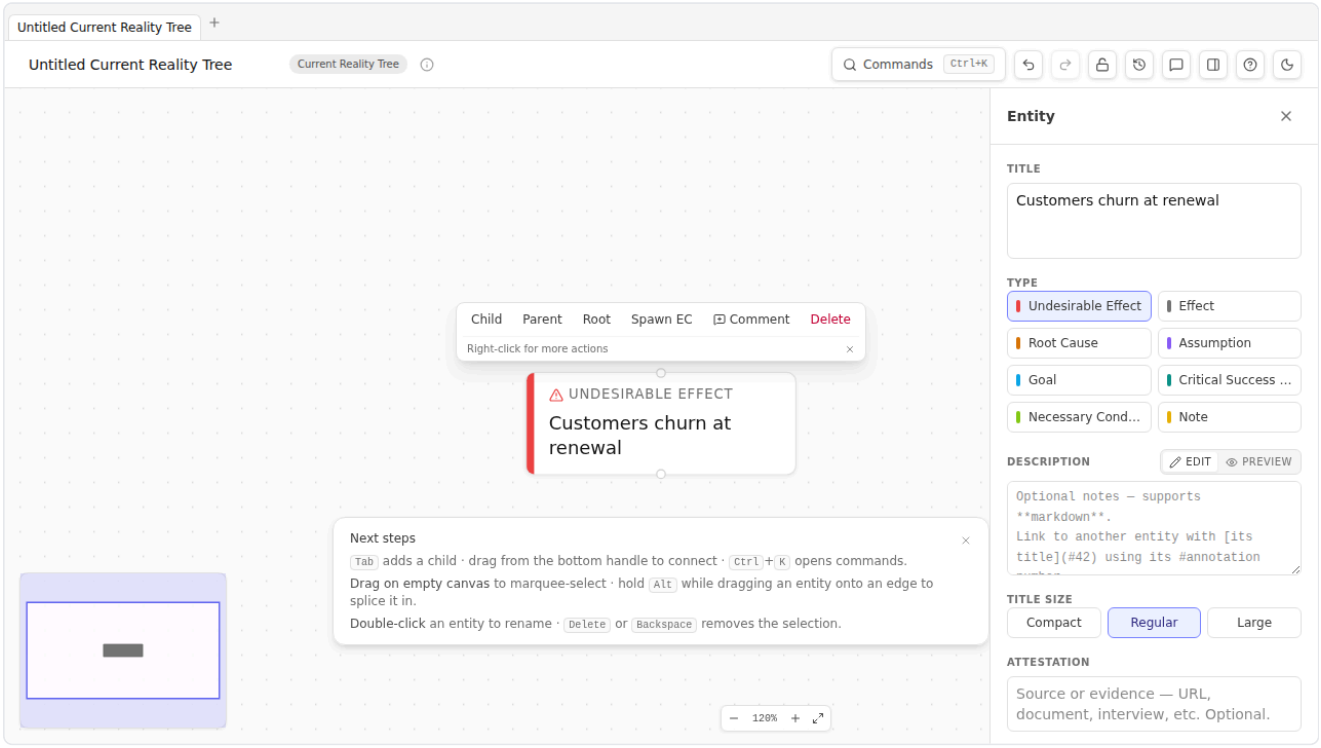
The setup: VP of Customer Success calls you in. "We're losing customers. Support is burnt out. Cost per ticket is up. We've tried hiring more agents and it didn't help. We've tried better tooling and it didn't help. What's actually going on?"

Three UDEs to start. You write them down in TP Studio.

Step 1 — The first UDE

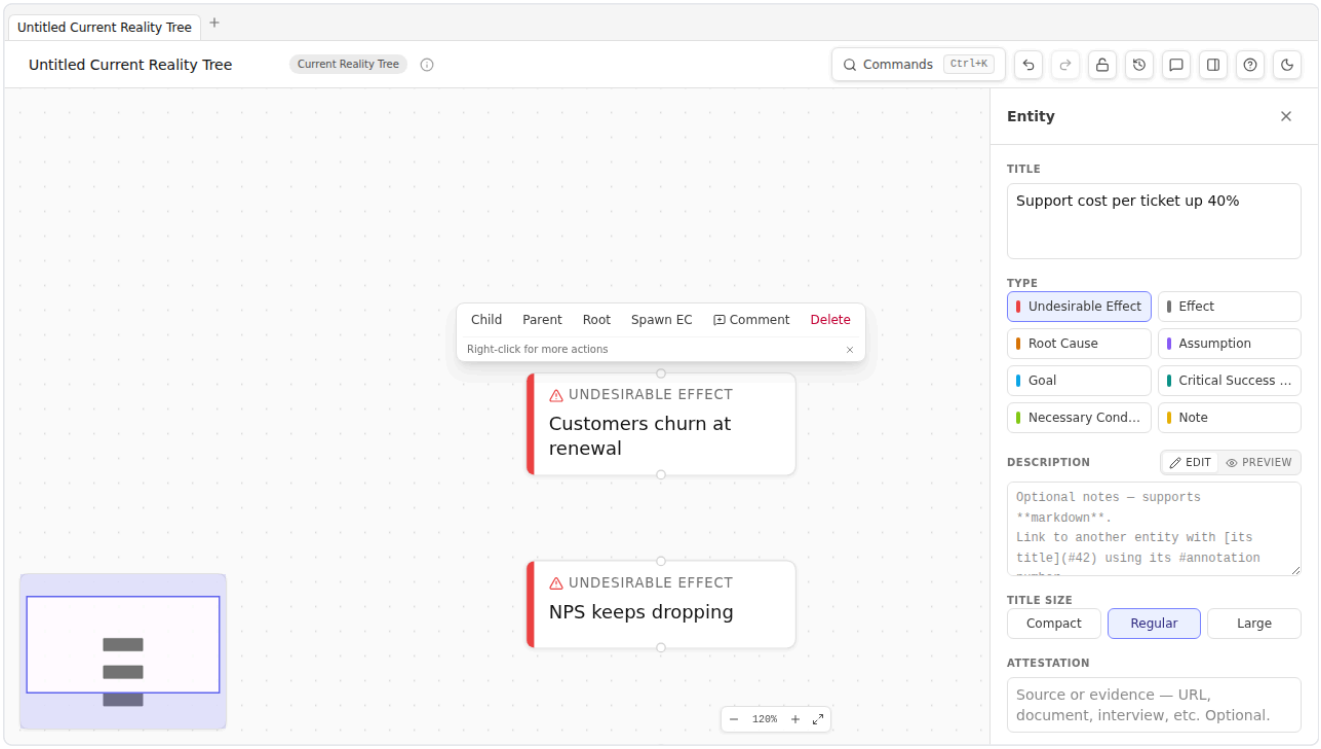
Open TP Studio. `Cmd+K → New diagram → Current Reality Tree`. Empty CRT canvas opens.

Double-click the canvas, type **Customers churn at renewal**, press Enter. Click the entity to select it. In the Inspector's Type grid, click **Undesirable Effect**. The stripe turns red.



Step 2 — The other UDEs

Two more double-clicks. Type **NPS keeps dropping** for one, **Support cost per ticket up 40%** for the other. Mark both as `Undesirable Effect` via the Inspector. Three red-striped entities side by side:



🛠 **How TP Studio helps:** The TopBar's TitleBadge will now say "CRT" — confirming the diagram-type-appropriate validators are active. The CLR walkthrough wizard (`Cmd+K → Start CLR walkthrough`) will already be flagging "UDEs without cause chains" as a clarity-tier warning. We'll get to those.

Step 3 — First cause-chain

Pick the most visceral UDE first. "Customers churn at renewal" — why? Because (you ask the team) deals close on the assumption of a 4-hour SLA but support routinely misses it. So: under "Customers churn at renewal", add a new entity titled **Customer SLA expectations are missed**. Connect upward.

Read it: *"Because the customer SLA expectations are missed, customers churn at renewal."* Sounds reasonable.

Why are SLA expectations missed? Because resolution time runs over 8 hours on most tickets. Add **Resolution time > 8h on most tickets**; connect.

Why? Two reasons, you discover after talking to agents. (1) The team has no shared triage rubric — every ticket gets the same level of investigation regardless of severity. (2) Senior agents context-switch constantly between L1 and L2 tickets, so neither gets done quickly.

Add both as causes. These are *jointly* sufficient — you need them both for the long resolution times. Group them as an AND. (Select both edges → `Cmd+K → Group as AND`.)

Step 4 — The second cause-chain

"NPS keeps dropping" — why? You ask. The biggest detractor theme is "I get a different answer every time I contact you." So: under "NPS keeps dropping", add **Customers get inconsistent answers**.

Why inconsistent? Because — again — no triage rubric. *Same* cause we already added under the other branch. So instead of duplicating it, connect "Customers get inconsistent answers" to the existing **No shared triage rubric** node. The arrow goes up; the node already lives over there.

This is the moment a CRT starts to *feel* like a structural diagnosis. The convergence is the constraint becoming visible.

Step 5 — The third cause-chain

"Support cost per ticket up 40%" — why? Two reasons emerge in conversation: agents redo prior work because there's no canonical answer to common questions; *and* senior agents are bottlenecked because everything escalates to them (which is the same context-switching problem from the first chain).

Add **Agents redo prior work for common questions**. Why? Because there's no consolidated answer base. Why no answer base? Because nobody's been protected from incoming tickets long enough to build one.

Add **Support lead has no protected drafting time**. Connect upward. Also: the senior-agent context-switching pulls in the same direction. Connect.

You should now have something resembling a tree with three UDEs at the top, converging through intermediate effects into a small number of root causes at the bottom. Click each terminal node and mark it `Root Cause` via the Inspector's Type grid.

Step 6 — Find the core driver

`Cmd+K → Find core driver(s)`. TP Studio computes UDE-reach for each root cause and selects the top candidate(s). The toast tells you the score: how many UDEs the highest-reach root cause ladders up to.

In our example: **Support lead has no protected drafting time** is reaching all three UDEs (directly or via the chain). That's the core driver. The constraint isn't "we need more agents" or "we need better tooling" — it's that nobody has been protected from incoming work long enough to build the system the team needs.

🛠 **How TP Studio helps:** Settings → Display → Show UDE-reach badge toggles an amber `→N UDEs` pill on each entity. With it on, the core driver visually announces itself in any CRT.

Step 7 — Verbalize

Click `Cmd+K → Start read-through` to open the walkthrough overlay. TP Studio iterates each structural edge in topological order. Read each sentence aloud.

"Because Support lead has no protected drafting time, agents redo prior work for common questions." Sounds right.

"Because Support lead has no protected drafting time, no shared triage rubric exists." Sounds right.

"Because No shared triage rubric AND Senior agents context-switch constantly, resolution time > 8h on most tickets." Sounds right.

"Because resolution time > 8h, customer SLA expectations are missed." Sounds right.

"Because customer SLA expectations are missed, customers churn at renewal." Sounds right.

Read each chain twice. The second time you'll catch the one that sounds *almost* right but isn't quite — a CLR tier-3 (cause-effect reversal) or tier-4 (predicted effect existence) violation. Fix it. Re-read.

Step 8 — Stop

How do you know the CRT is done?

- Every UDE traces to a root cause that's in your `control` or `influence` zone.

- The core driver is identified; its reach feels structurally honest (not "I made one node connect to everything"; really *connecting*).
- Read-aloud passes the gut check on every chain.
- The CLR walkthrough is empty (no open warnings) — or each open warning has been explicitly dismissed with a "considered and doesn't apply" note in the entity's description.

When all four are true, the CRT is ready for the next step — which is usually Chapter 5, where you ask why the core driver has persisted.

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → New Current Reality Tree` *to start a fresh CRT.*
> - `Cmd+K → Load example Current Reality Tree` *for a 6-entity reference doc to study before drawing your own.*
> - ***Inspector Type grid*** *with one-click switching between Effect / UDE / Root Cause / Assumption.*
> - ***AND grouping****: select multiple edges →* `Cmd+K → Group as AND` *(or right-click → Group as AND).*
> - `Find core driver(s)` *palette command — picks the highest-reach root cause(s).*
> - ***UDE-reach badge*** *(Settings → Display → "Show UDE-reach badge") visualizes reach per entity.*
> - ***Reverse-reach badge*** *shows the symmetric "root causes per UDE" count, useful for verifying that each UDE has a real cause chain rather than dangling.*
> - ***CLR walkthrough****:* `Cmd+K → Start CLR walkthrough` *iterates open warnings.*
> - ***Read-through overlay****:* `Cmd+K → Start read-through` *— the verbalisation discipline made gesture.*

> ***💡 Practitioner tips***
>
> - ***Start with UDEs, not causes.*** *Resist the urge to write your hypothesized causes first. Let the causes emerge from "why does that happen?" questioning. A CRT built top-down from UDEs is more honest than one built bottom-up from preferred conclusions.*
> - ***Re-use existing intermediate nodes.*** *When a second cause chain wants to land at the same intermediate effect you already drew, connect to the existing node — don't duplicate. The convergence IS the diagnosis.*
> - ***Don't be afraid to mark an entity*** `unspecified` ***.*** *The Inspector has an "unspecified — fill in later" toggle. Use it when you know there's a step in a chain but can't yet articulate it; it suppresses the empty-title warning while keeping the structure visible.*
> - ***Verbalize early, not just at the end.*** *Read each new chain aloud as you add it. The error correction is cheaper if you catch the awkward sentence before you've built three more entities downstream of the awkward one.*

> ⚠ ***Common mistakes***
>
> - ***Listing causes as UDEs.*** *"We have no triage rubric" is not a UDE — nobody feels that directly. "Resolution time exceeds 8h" is a UDE. The UDE layer is what customers / employees / the market would say is wrong, not what you the analyst would say is wrong.*
> - ***Bottoming out at an external cause.*** *"Macroeconomic conditions are tough" is not a root cause for your CRT. Almost always there's a* `control` *-level cause underneath ("we haven't refreshed our pricing model since the rate hike"). The* `external-root-cause` *warning flags this.*
> - ***Drawing a CRT without an AND group when one is needed.*** *If A and B are both required for C, drawing them as two single arrows to C is technically wrong — it claims each is sufficient. Group as AND.*
> - ***Treating the CRT as the deliverable.*** *The CRT is a diagnostic intermediate. The deliverable is usually the FRT / TT pair that follows. If you find yourself polishing a CRT for a stakeholder pack, you've stopped one diagram too early.*

> 🛑 ***When to stop***
>
> - *Every UDE has a path to a* `control` *or* `influence` *root cause.*
> - *The core driver is identified and its reach is honest.*
> - *The CLR walkthrough is empty or every open warning is intentionally dismissed.*
> - *You can read each chain aloud without rewording.*
> - *You're ready to ask the next question: why hasn't this been fixed already? (That's the EC in Chapter 5.)*

🔁 **Chain to next:** the CRT shows you *what* is wrong. The Evaporating Cloud shows you *why nobody has fixed it yet*.

→ Continue to Chapter 5 — Evaporating Cloud

Chapter 5 — Evaporating Cloud

Why are we stuck?

> 🎯 ***What this process is for*** *An Evaporating Cloud (EC) reveals the conflict that holds a chronic problem in place. It answers: "Why hasn't this been fixed already?" The diagram is a five-box pattern showing two legitimate but opposing wants, each rooted in a real need, both subordinated to a common goal. When you draw it well, the cloud "evaporates" — you find the false assumption that made the conflict feel necessary, and the resolution emerges with it.*

The premise

If a problem has persisted in a system populated by reasonable people, **it's persisted for a reason**. Most chronic organizational problems aren't "we haven't gotten around to fixing them yet" — they're "every time we try to fix them, we create a different problem instead, and that other problem feels worse, so we revert." The thing that makes you revert is the conflict.

The Evaporating Cloud forces you to write that conflict down. Not as "Engineering vs. Product" (organizational shorthand) or "we need to choose between speed and quality" (false dichotomy in disguise), but in the specific 5-box structure that lets you see *which assumption* is making both sides feel they must hold their position.

Once the false assumption is named, the cloud evaporates. The "conflict" was never between the wants; it was between two ways of getting to the same goal under a constraint that no longer applies or never applied in the first place.

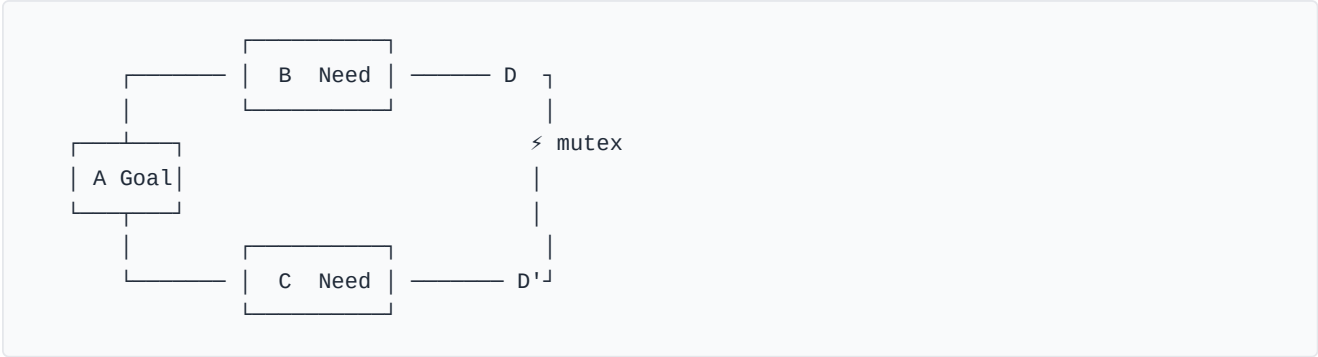
This is Goldratt at his most provocative. He claimed — and TOC practitioners since have largely confirmed — that almost every chronic conflict in an organization is *false*. The wants are real, the needs are real, the goal is shared; what's wrong is the unstated assumption that you must choose. The EC's job is to drag that assumption into the light.

The 5-box structure

Slot	Role	What goes here
A	The common Goal	What both sides agree they're ultimately trying to achieve.
B	The Need behind want D	A legitimate prerequisite for the Goal.
C	The Need behind want D′	A different legitimate prerequisite for the Goal.
D	The Want — one side's chosen action	What B says "to satisfy me, we should…"
D′	The Want — the other side's chosen action	What C says "to satisfy me, we should…"

The conflict is between **D and D′**. Both can't be done simultaneously (that's the mutex). But B and C are both real needs, and A is a shared goal — so the conflict can't be resolved by picking a side. It can only be resolved by finding the assumption that makes D and D′ feel mutually exclusive, and then *not making that assumption anymore*.

Visualized, the EC looks like a flat tree on its side:



Read it in either direction. **Goldratt's preferred read** is A-first (top-down): *"In order to achieve A, we must satisfy B. In order to satisfy B, we must do D. In order to achieve A, we must also satisfy C. In order to satisfy C, we must do D′. And D conflicts with D′."* The whole point of the diagram is the awkwardness of that last sentence.

The method, neutral of tool

1. **Name the conflict in shorthand.** "We need to ship fast vs. we need to ship reliably." "Engineering wants stability vs. Product wants velocity." Whatever the room would say. This isn't the EC; this is the *cover label* for the conversation.
2. **Find D and D′ — the two wants.** Concrete. Action-shaped. "Ship the feature this sprint" / "Defer the feature one sprint." Not "be faster" / "be more careful." The wants are what people actually advocate when they argue.
3. **Find B and C — the two needs.** For each want, ask: *why do you want that?* The answer is the need. The need isn't the next action; it's the *condition the want is in service of*. B = "Maintain market position by shipping ahead of competitor X." C = "Avoid eroding customer trust through buggy releases."
4. **Find A — the shared goal.** Both B and C are conditions for what *bigger* thing? The answer is the goal. Almost always: "long-term commercial success of this product," "customer satisfaction," "team sustainability." If you can't find a shared A, the conflict is at a higher level than you've named it; back out and re-frame.
5. **Name the mutex.** Why do D and D′ conflict? Often "we only have one engineering team's time this sprint." Sometimes "you can't simultaneously have stable and not-yet-tested code in the same release."
6. **List the assumptions on each arrow.** Every necessity arrow ("B requires D") rests on assumptions. Write them down. There will be more than you expect.
7. **Find the breakable assumption.** One of those assumptions, when challenged, is false or no-longer-true. Naming it is the *injection*. The cloud evaporates: it turns out D and D′ aren't actually mutually exclusive, OR they were both wrong actions for B and C, OR the mutex isn't real.

The worked example

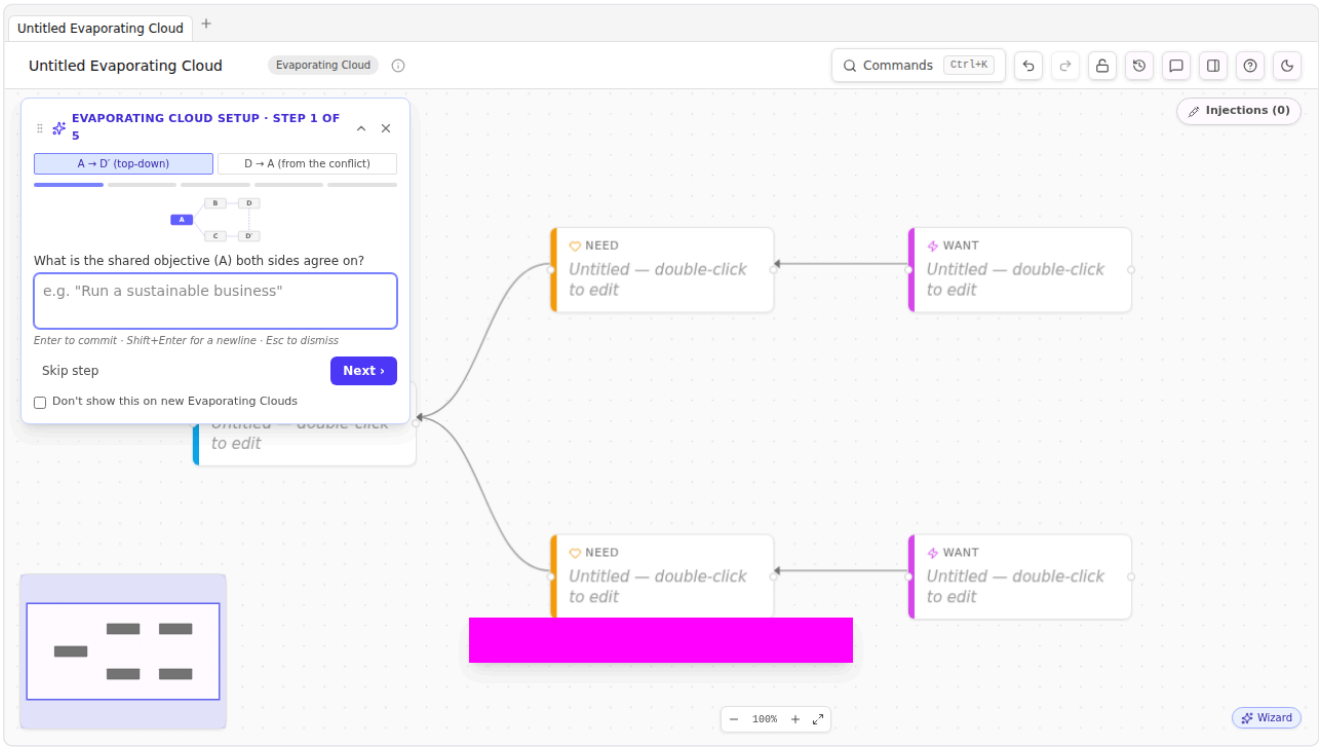
We continue from Chapter 4. The CRT identified "Support lead has no protected drafting time" as the core driver. The natural follow-up question: *why has that persisted?*

Because the support team can't actually spare anybody. Every hour the lead spends drafting is an hour fewer on the queue, the queue grows, customers complain, leadership escalates, lead is back on the queue. The system has been in this loop for months.

Underneath: a conflict. Let's draw it.

Step 1 — Start the EC

`Cmd+K → New diagram → Evaporating Cloud`. The Creation Wizard opens at step 1, prompting for **A — the common goal**. The canvas is pre-seeded with five empty boxes in the canonical 5-box layout.



🛠 **How TP Studio helps:** The EC creation wizard is a 5-step guided dialog that fills the slots in order (A → B → C → D → D′). Each step commits the current entity live, so partial dismissals leave the canvas in a useful state. The Wizard's "from the conflict" toggle reverses the order to D-first (D → D′ → B → C → A) — useful when the cover-label conflict is what surfaced first in conversation and the goal is the thing you haven't yet articulated.

Step 2 — Write A

The common goal. For our example: **A sustainable support function**. Type it, click Next.

The wizard auto-positions A at the canonical left-center position and gives it the sky-blue Goal stripe.

Step 3 — Write B (need behind want D)

The wizard prompts: "What is the first need that the goal requires?" Think about who's been advocating *not* protecting drafting time. That's leadership. Why? Because they need responsiveness to keep customers retained while they're already losing some. So B = **Keep ticket queue responsive to retain at-risk customers**.

Step 4 — Write C (need behind want D′)

"What is the second, conflicting need?" The advocate for protected drafting time would be — also the support lead, also you-the-analyst. Why? Because without the canonical answer base and the triage rubric, the team can't scale; you'll be in the same loop in six months. C = **Build durable structure (rubric + answer base) so the team scales**.

Both needs are legitimate. Both serve A.

Step 5 — Write D

"What is the first action that satisfies need B?" The current behavior is: **Support lead stays on the queue**. That's what's been happening for the last six months.

Step 6 — Write D′

"What is the conflicting action, satisfying need C?" **Support lead takes 2 days/week off the queue to build structure**.

Both wants are concrete actions. They conflict because the lead is one person — there isn't a both-at-once.

Step 7 — Mark the mutex

The wizard closes; the canvas now shows the 5-box EC. Click the edge between D and D′ (if it exists; otherwise, draw it). Inspector → Mutual exclusion toggle → on. The edge turns red with a ⚡ glyph. The `ec-missing-conflict` validator stops firing.

Step 8 — Verbalize

Open the **VerbalisationStrip** above the canvas (it's there by default on EC docs). It renders the cloud as a paragraph:

> *In order to achieve A sustainable support function, we must keep ticket queue responsive to retain at-risk customers. In order to keep ticket queue responsive to retain at-risk customers, support lead stays on the queue. In order to achieve A sustainable support function, we must also build durable structure (rubric + answer base) so the team scales. In order to build durable structure (rubric + answer base) so the team scales, support lead takes 2 days/week off the queue to build structure. And these two wants conflict.*

Read it aloud. It should sound exactly like a problem somebody at the company would describe. If it doesn't, the cloud isn't the right one.

Try the two-party variant. Document Inspector → EC Verbal Style → `twoSided`. Now the strip reads:

> *In order to achieve A sustainable support function, leadership wants to keep ticket queue responsive. To do so, they want support lead stays on the queue. In order to achieve A sustainable support function, the team wants to build durable structure. To do so, they want support lead takes 2 days/week off the queue. And these two wants conflict.*

The two-sided framing makes the political contour of the cloud visible. Useful in workshops where it matters who-said-what.

Step 9 — Surface the assumptions

This is where the cloud evaporates.

For each of the four B→D, C→D′ necessity arrows, list assumptions. TP Studio's **AssumptionWell** in the Inspector lets you add them as first-class entities with status (`open` / `valid` / `invalid`). When an assumption is anchored to an edge, TP Studio draws a faint dashed grey line on the canvas from the assumption entity to the midpoint of the

edge it pertains to — so even with multiple assumptions in play you can see at a glance which arrow each is challenging without opening the inspector.

Pick the **B → D** arrow ("To keep queue responsive, support lead must stay on the queue"). Click the edge. The AssumptionWell shows up in the Inspector. Add:

- *Only the support lead can resolve the hard tickets.*
- *Tickets sufficient to keep the queue responsive must all be resolved by humans.*
- *We can't temporarily reduce inbound ticket volume.*
- *No structural improvement could pay off within the timeframe leadership cares about.*

Pick the **C → D′** arrow. Add:

- *Building the structure requires the lead specifically — no other agent can do it.*
- *The structure must be built in dedicated blocks, not incrementally.*
- *Lead time on the structure is more than the timeline before the next renewal cycle.*

Read each one aloud. Some are clearly false; some are clearly true; one or two are *the* assumption — false but never previously named, and false-ness of which dissolves the conflict.

In our example: **"Only the support lead can resolve the hard tickets"** is the key. If two L2 agents could be trained on the hardest 20% of tickets, the lead's queue time drops, freeing the drafting time, *without* sacrificing queue responsiveness. Both wants get satisfied. The mutex was a habit, not a fact.

That false assumption is the **injection**. In the Inspector's InjectionWorkbench, mark it as `valid: false` (i.e., we've decided this assumption is wrong). Add a new entity titled **Train 2 L2 agents on the hardest 20% of ticket types** and mark it as type `Injection`. Link it to the assumption.

The cloud has evaporated.

Step 10 — Stop

The EC is done when:

- All five slots (A / B / C / D / D′) are concrete and read naturally aloud.
- The mutex is flagged on the D ↔ D′ edge.
- Each B→D, C→D′ arrow has at least two assumptions written down.
- At least one assumption per arrow has been actively classified (valid / invalid / open).
- One assumption has emerged as **the breakable one**, with an injection drafted that resolves the conflict.
- You can describe the resolution in one sentence without rationalizing: *"Once we have two L2 agents who can handle the hard tickets, the lead can take protected drafting time without queue slippage."*

Cloud progression — the same tool, escalating roles

Cohen frames the cloud not as a single artifact but as a *progression*. The same five-box structure does different jobs as the analysis deepens:

- A **UDE cloud** sits behind a single undesirable effect — the dilemma that keeps *this* problem alive.
- A **Consolidated cloud** merges several UDE clouds that share a shape.

- A **Core cloud** is the recurring conflict underneath many UDEs at once — the one at the base of the CRT, and the one worth breaking.

A fourth turns up constantly in practice: the **Firefighting** (Lieutenant) cloud — *patch the symptom now* vs. *stop it coming back* — the trap that keeps an organisation reacting instead of improving.

In TP Studio this is an optional **Cloud type** label (Document panel → *Cloud type*, on EC documents): Dilemma, Conflict, UDE, Consolidated, Core, or Firefighting. It's just a label — nothing about drawing or reading the cloud changes — but tagging one drops a small chip by the title so a folder of clouds reads as a progression rather than a pile. The **Pattern library…** ships a *UDE cloud*, a *Core cloud*, and a *Firefighting cloud* as pre-tagged starting points.

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → New Evaporating Cloud` → *opens the* ***Creation Wizard*** *which guides A / B / C / D / D' in order.*
> - ***Reading-direction toggle*** *in the Wizard for A-first (default) vs. D-first (when the conflict surfaced first).*
> - ***EC Verbal Style*** *toggle in the Document Inspector:* `neutral` *("we must") or* `twoSided` *("they want / I want") for two-party framing.*
> - ***Cloud type*** *label in the Document Inspector — tag an EC as a UDE / Consolidated / Core / Firefighting cloud (the TP Basics progression); a chip by the title records the role. Three pre-tagged clouds ship in the Pattern library.*
> - ***VerbalisationStrip*** *above the canvas — renders the cloud as a paragraph, updates live as you edit.*
> - ***AssumptionWell*** *in the EC inspector — first-class assumption records with* `open` / `valid` / `invalid` *status and links to injection entities.*
> - ***InjectionWorkbench*** *in the EC inspector — list every proposed injection with* `implemented` *toggles for FRT carry-forward.*
> - ***Mutex (⚡) edge flag*** *on the D ↔ D' edge. The* `ec-missing-conflict` *validator fires until one such edge exists.*
> - ***EC Workshop Sheet PDF*** *—* `Cmd+K → Export → EC workshop sheet` *— generates a one-page PPT-style layout with the guiding questions baked in. Good for handouts.*
> - `ECReadingInstructions` ***strip*** *above the canvas — the dismissible 1/2/3 numbered hints reminding you of the reading direction. Default-hidden after Session 89; re-enable via* `Toggle EC reading guide` *.*

> 💡 ***Practitioner tips***
>
> - ***D and D' must be actions, not states.*** *"We want stability" isn't a want; it's a value. The want is "we want to gate this release on a 2-week soak test." Concrete actions are what people argue about in meetings.*
> - ***B and C must be needs, not wants.*** *"Need to be fast" isn't a need; it's a paraphrase of D. The need is what the speed is in service of — usually a market position, a customer commitment, a financial reality.*
> - ***A must be genuinely shared.*** *If you can't find an A that both advocates would agree to, the conflict is at a higher level than you've framed it. Back out, ask "and what is that in service of," try again.*
> - ***Don't skip the assumption listing.*** *The temptation is to draw the 5 boxes, declare the cloud "done", and walk away. The breakable assumption hides in the assumption list, not in the boxes. The boxes are scaffolding for the listing exercise.*
> - ***One EC per conflict.*** *Don't try to fit "Engineering vs. Product vs. Customer Success" into one EC. Draw two — one for the Eng/Product conflict, one for the CS/Eng conflict — and see if they share an A.*

> ⚠ ***Common mistakes***
>
> - ***Picking a false A.*** *"Make money" is the lazy A. It's usually true but it's also true for every corporate conflict, which means it doesn't help you. The good A is specific enough that it could plausibly not hold ("be the trusted long-term partner for mid-market SaaS customers"). If the A is generic, the B and C will be generic too.*
> - ***Equating D with the long-term solution.*** *D is the current behavior (or the chosen-side behavior in a debated decision), not the eventual fix. The eventual fix is the injection you draft after dissolving the cloud. Don't shortcut.*
> - ***Writing wishful assumptions.*** *"We could just hire more people" is not an assumption that dissolves the cloud — it's a wish. Assumptions are claims you could plausibly evaluate as true/false right now.*
> - ***Skipping verbalisation.*** *If you can't read the cloud aloud and have it sound like a problem your colleagues would recognize, the cloud isn't real. Re-frame.*

> 🛑 ***When to stop***
>
> - *The cloud reads aloud as a real, recognizable problem.*
> - *Each necessity arrow has assumptions, classified.*
> - *One breakable assumption is named.*
> - *An injection exists addressing the breakable assumption.*
> - *The resolution sentence — "Once X, the conflict dissolves because Y" — is one sentence and stands up to a "really?" challenge.*

🔁 **Chain to next:** the EC names the conflict and drafts an injection. The Future Reality Tree checks whether the injection actually delivers the desired effects without spawning new UDEs.

→ Continue to Chapter 6 — Future Reality Tree

Chapter 6 — Future Reality Tree

What would it look like solved?

> 🎯 ***What this process is for*** *A Future Reality Tree (FRT) tests whether your proposed injections actually solve the problem. It answers: "If I make these changes, what will the system produce instead?" The same causal model as a CRT, but starting from injections instead of root causes and reaching upward to desired effects instead of undesirable ones. The point is prediction: if the FRT is logically sound, the injections will work; if it's not, they won't.*

The premise

A CRT diagnoses. An EC names the conflict. An FRT designs the solution. Each is a deliberate step; skipping straight from EC to "let's do the injection" loses the discipline of *checking your work*.

The check matters because injections have second-order effects. The thing you do to fix UDE-1 might cause UDE-7. The FRT is the place to catch that before the rollout, not after. Goldratt called the unanticipated downstream UDE the **Negative Branch** — and the FRT is structured to make Negative Branches *visible*.

The method

1. **Start with the injections you drafted in the EC**, marked as `Injection` type. They are the bottom of the FRT — the things you will *do*.
2. **Above each injection, list the desired effects you expect.** "The team has protected drafting time." "The triage rubric exists." "Senior agents stop context-switching." Mark each as `Desired Effect`.
3. **Connect upward — sufficient causality, same as CRT.** "Because of injection X, desired effect Y." Verbalize.
4. **Trace forward to the elimination of each original UDE.** The FRT is correct when every CRT UDE has a path *down* to one or more of your injections.
5. **Hunt for Negative Branches.** For each major desired effect, ask: "What new UDE might this *also* cause?" If you find one, draw it as an `Undesirable Effect`. Then look for an injection that prevents it. Add a new injection if needed. Keep iterating.
6. **Flag Positive Reinforcing Loops.** When a desired effect causes something that causes the original desired effect more strongly, you've found a virtuous cycle. Tag the back-edge.
7. **Verbalize the FRT** the same way you verbalized the CRT. If it sounds like a real future, you're done.

Worked example (continued)

From Chapter 5: one injection drafted — **Train 2 L2 agents on the hardest 20% of ticket types**.

`Cmd+K → New diagram → Future Reality Tree`. Empty FRT canvas opens.

Add the injection: double-click, type **Train 2 L2 agents on the hardest 20% of ticket types**, Inspector → Type → Injection. The stripe turns emerald.

Now ask: *if this injection lands, what will be true?*

- Lead's queue load drops ~30%.
- Lead has 2 days/week of protected drafting time.
- L2 agents have a clearer career-development path. *(Bonus desired effect — worth noting.)*

Add each as `Desired Effect` entities (indigo stripe).

Connect upward. Then continue:

- Because Lead has protected drafting time → triage rubric exists → resolution time drops → SLA met → churn drops.
- Because Lead has protected drafting time → consolidated answer base exists → agents stop redoing prior work → cost-per-ticket drops.
- Because L2 agents have career development → retention improves → reduced hiring-and-training overhead. *(Another bonus.)*

Three of the original CRT's UDEs trace cleanly down to the single injection. So far so good.

Now hunt for Negative Branches. Ask: "What might *also* happen?"

- L2 training takes time. During training, queue load on existing seniors goes UP. *(UDE: "Queue load on existing seniors spikes during training.")*
- If the training is poorly designed, L2 agents handle hard tickets badly, customer experience drops. *(UDE: "L2-resolved tickets are lower quality than L3.")*

Add both as UDEs. Each is a real risk of the injection.

For each negative branch, draft a *second-order injection*:

- Hire one temp contractor for 8 weeks to cover queue during training. *(Inject.)*
- Build a 2-week shadowing program where L2s pair with L3s on hard tickets before going solo. *(Inject.)*

The FRT now has 3 injections (one primary, two for negative branches), 5 desired effects, 2 negative-branch UDEs (each suppressed by an injection), and clean paths from the injections to the elimination of the original 3 CRT UDEs.

🛠 **How TP Studio helps:** Two paths.

1. *Inside this FRT:* `Start Negative Branch from this entity` palette command (also right-click context menu) creates a "Negative Branch" group preset (slate-coloured) rooted at the entity you select. Useful for keeping NB sub-trees visually separated *within* the FRT.
2. *As its own document:* `Cmd+K → New diagram… → NBR` creates a dedicated **Negative Branch Reservation** diagram. The NBR diagram type carries its own palette (`injection` / `effect` / `ude` / `desiredEffect` / `assumption` / `note`), a 7-step method checklist (state the injection → trace forward to desired effects → identify turning point → articulate UDEs → choose reactive vs proactive mitigation → apply CLR → decide adopt/modify/reject), and feeds the **Risk Register (CSV)** export described in Chapter 16. Use it when the negative branch is the *centrepiece* of the analysis (e.g. a contentious injection that needs a deeper consequence-mapping) rather than a sub-branch of an FRT.

Trimming a negative branch

Finding a negative branch is only half the move. Once you've drawn the unintended UDE that your injection would also cause, the corrective answer in Goldratt's grammar is to *trim* it — to add a second injection whose whole job is to break the link so the bad effect won't follow. TP Studio makes that a single gesture. Select the undesirable effect at the tip of the branch and run **"Trim this branch (add a trimming injection)"** from the palette: it mints a **trimming**

injection wired to that effect with a *negative* edge — the formal "inject this, and the bad effect doesn't follow" construction, rendered so the polarity is unmistakable. It's one undoable step, so trimming costs you nothing to try. All that's left is to name the new injection to say *what* breaks the branch — the contractor backfill, the shadowing programme — turning a spotted risk into a concrete countermeasure on the canvas.

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → New Future Reality Tree` *to start fresh;* `Load example Future Reality Tree` *for a reference.*
> - ***Group presets****: Negative Branch (slate), Positive Reinforcing Loop (emerald), Archive (slate, collapsed) —* `Cmd+K → Group inspector → Preset` *. Catalog in* `src/domain/groupPresets.ts` *.*
> - ***Back-edge tagging*** *for reinforcing loops — Edge Inspector → Back-edge toggle.*
> - ***Edge polarity*** *matters in FRTs more than CRTs. If a chain has a negative edge halfway up, the eventual desired effect is suppressed, not produced. The polarity badge catches it.*
> - ***InjectionWorkbench carry-forward*** *— if you started in an EC and drafted injections there, the injections (entities of type* `Injection` *) carry to the FRT through the JSON model. Copy-paste between docs preserves them.*
> - `Trim this branch (add a trimming injection)` *— select the UDE at a negative branch's tip; mints a trimming injection wired to it with a negative edge, in one undoable step.*

> ***💡 Practitioner tips***
>
> - ***Look for Negative Branches actively, not passively.*** *The FRT's value is mostly in catching them. Walk through each desired effect and ask "what else?" Don't accept the first answer.*
> - ***Number your injections.*** *The Inspector's* `annotationNumber` *field lets you label them I1, I2, I3. Useful when writing the rollout plan; readers can reference "injection I2" rather than "the second one from the top."*
> - ***The FRT is a draft.*** *It will be wrong about some second-order effects. The rollout will reveal which ones; capture revisions before the rollout so you can compare predicted vs. actual after.*

> ***⚠ Common mistakes***
>
> - ***Trivial FRT.*** *If your FRT is a single injection → single desired effect → "and the UDE goes away", you haven't done the work. Real systems have second-order effects; if you didn't find any, you didn't look.*
> - ***Confusing prediction with hope.*** *"We hope this will lead to…" isn't a cause-effect claim. The FRT's edges are predictions: if this, then probably that. Mark assumptions where the prediction is uncertain.*
> - ***Skipping the Negative Branch hunt.*** *This is the single highest-value step in FRT drawing. Skipping it produces FRTs that read like project plans and predict like horoscopes.*

> 🛑 ***When to stop***
>
> - *Every CRT UDE has a path down to at least one injection.*
> - *Each injection has at least one desired effect spelled out.*
> - *You've looked for Negative Branches against every major desired effect (looking, not just gesturing).*
> - *Each NB has either an injection that suppresses it, or an explicit "we accept this risk" annotation.*
> - *Verbalisation reads as a plausible future, not a wish-list.*

🔁 **Chain to next:** the FRT tells you *what* should happen. The PRT tells you *what's in the way of making it happen*.

→ Continue to Chapter 7 — Prerequisite Tree

Chapter 7 — Prerequisite Tree

What's in our way?

> 🎯 ***What this process is for*** *A Prerequisite Tree (PRT) surfaces the obstacles between* ***where you are*** *and* ***where the FRT says you want to be****, then pairs each obstacle with an* ***Intermediate Objective*** *(IO) — a state that, if achieved, dissolves the obstacle. It answers: "What has to be true* ***first*** *for the injections to land?"*

The premise

You've diagnosed (CRT), named the conflict (EC), designed the solution (FRT). Now you have to *execute*, and execution runs into obstacles: things you don't have, capabilities you can't yet exercise, dependencies that aren't ready. The PRT makes those obstacles explicit and pairs each with the IO that resolves it.

Goldratt's framing: every injection has a precondition tree underneath. The PRT *is* that precondition tree, sequenced and obstacle-aware.

The method

1. **Start with the injection.** Place it at the top of the canvas. (PRTs read top-down, unlike CRT/FRT which read bottom-up. TP Studio's layout strategy for PRT respects this.)
2. **Below the injection, list obstacles.** "We don't have an L2 agent with capacity to take on training." "We don't have a triage rubric document anyone has agreed to."
3. **For each obstacle, pair an IO.** "Two L2 agents reassigned with 40% capacity for training over 8 weeks." "Draft rubric circulated, reviewed, and signed off by the support lead and customer-success VP."
4. **Necessity edges, not sufficiency.** "In order to achieve [IO], we must overcome [Obstacle]." The TP Studio default for PRT edges is `necessity` — TP Studio handles this automatically.
5. **Sequence the IOs.** Some IOs depend on others. Connect them in dependency order. A chain emerges from the bottom (independent IOs) to the top (the injection).
6. **Stop when the leaves (lowest IOs) are things you can do *now*.** If a leaf IO still has its own obstacle that requires a sub-IO, keep descending. The tree is done when every leaf is actionable.

Worked example (continued)

From Chapter 6: primary injection is **Train 2 L2 agents on the hardest 20% of ticket types**.

`Cmd+K → New diagram → Prerequisite Tree`. Empty PRT canvas opens. Add the injection at the top as an `Injection` entity.

Below it, brainstorm obstacles:

- *Obstacle:* We don't have a list of "the hardest 20% of ticket types" — nobody's audited the queue.
- *Obstacle:* Both candidate L2s have full queues already.
- *Obstacle:* No training curriculum exists.
- *Obstacle:* No budget allocated for the contractor backfill (FRT injection I2).

For each, pair an IO:

- *IO:* Ticket-type audit complete; "hardest 20%" defined by escalation count + resolution time. (`intermediateObjective` type, blue stripe.)
- *IO:* L2 capacity freed by re-routing ~40% of their existing tickets to L1 with escalation rules.
- *IO:* Curriculum drafted; 4 modules × 2 hours each; lead reviews.
- *IO:* Budget request approved by Finance for 8 weeks of contractor backfill.

Connect each IO upward through the obstacle to the injection. The PRT now shows: *to get to the injection at top, we must overcome these obstacles, which means achieving these IOs.*

Some IOs depend on others:

- "L2 capacity freed by re-routing" depends on "Ticket-type audit complete" (you need to know what the hardest 20% IS before you can re-route the other 80%).
- "Curriculum drafted" depends on the audit too.

Add those dependency edges. The PRT now reads as a *sequenced plan* — the lowest IOs first, the injection last.

Exporting an ordered plan

A PRT carries no explicit step numbers. Its order lives entirely in the dependency edges — the IO another IO depends on *is* the earlier step, even though nothing on the canvas says "1, 2, 3." That's correct for thinking, but it's awkward to hand to someone who just wants the to-do list. **Export… → "Prerequisite plan (CSV)"** does the translation: it topologically sorts the tree so that every IO appears *prerequisite-first* (an IO that others depend on comes before them) and writes one row per Intermediate Objective — step / objective / overcomes (the obstacle it targets) / depends on / owner / due / status / notes.

That row shape is deliberately spreadsheet- and tracker-shaped: paste it straight into Jira, Trello, or a sheet and you have a backlog ordered the way the analysis says it must be done, with each item still tied to the obstacle that justifies it. It's the bridge from the PRT-as-analysis to the PRT-as-execution-plan. (The export only appears on a document that actually has Intermediate Objectives — there's nothing to order otherwise.)

Sidebars

> **🛠 How TP Studio helps**
>
> - `Cmd+K → New Prerequisite Tree` to start fresh.
> - **`obstacle`** entity type (rose stripe) and **`intermediateObjective`** type (blue stripe) — PRT-specific.
> - ***Necessity edges by default*** *for PRT diagrams — the verbaliser reads "In order to / we must" wording.*
> - ***Dagre top-down layout*** *— PRT lays itself out with the injection at top, IOs descending.*
> - ***Span-of-control*** *flags help you triage IOs by who owns them. Mark each* `control` / `influence` / `external`.
> - **`Export… → Prerequisite plan (CSV)`** *— topologically sorts the IOs prerequisite-first into a step / objective / overcomes / depends-on / owner / due / status / notes row, ready to paste into Jira, Trello, or a spreadsheet. (Appears only when the doc has IOs.)*

> 💡 ***Practitioner tips***
>
> - ***Pair every obstacle with an IO.*** *An obstacle without an IO is a complaint. The IO turns "we can't" into "we will, once X."*
> - ***Surface the boring IOs.*** *"Get budget approved." "Schedule training time on calendars." These are the things that derail real rollouts. They belong in the PRT.*
> - ***Use Notes for context.*** *When an IO has nuance ("this requires Finance approval and we have a window before the next QBR"), capture it in the IO's description or as a sticky Note nearby.*

> ⚠ ***Common mistakes***
>
> - ***Skipping the obstacle and going straight to IOs.*** *Without the obstacle, the IO floats — it's not clear why it's necessary. The pairing is the point.*
> - ***Bottoming out at "we need executive buy-in."*** *That's a meta-IO. Sub-decompose it: what specifically does the executive need to know / agree to / approve? Each is a sub-IO.*

> 🛑 ***When to stop***
>
> - *Every leaf IO is something you can do* ***next week*** *(or whatever short timescale matters for your context).*
> - *Every IO is paired with a named obstacle it resolves.*
> - *Sequencing dependencies are explicit edges, not implicit.*
> - *Span-of-control is set on each IO so the rollout owner knows what's* `control` *vs.* `influence`*.*

🔁 **Chain to next:** the PRT shows *what's in the way*. The TT shows *the actions that dismantle each obstacle, in order*.

→ Continue to Chapter 8 — Transition Tree

Chapter 8 — Transition Tree

How do we get there?

> 🎯 ***What this process is for*** *A Transition Tree (TT) is the* operational plan*: a sequenced set of (action, precondition, outcome) triples that take you from current state to the IOs identified in the PRT. It answers: "What exactly does Maria do on Tuesday morning?"*

The premise

The PRT names obstacles and pairs IOs. The TT decomposes each IO into the concrete *actions* that achieve it, each action gated on a *precondition*, each producing an *outcome*.

The triple structure (action + precondition → outcome) is the smallest unit of TOC's implementation grammar. It maps cleanly to: "Given X, do Y, expect Z." Project plans and operational runbooks live in this grammar.

The method

1. **Start from the highest-priority IO** in the PRT (the one with the most downstream dependents, or the one with the longest lead time).
2. **For each IO, write the *outcome*** — what state of the world must exist when the IO is achieved? (Often the IO statement itself, slightly reworded as an outcome.)
3. **Write the *action*** — the verb-phrase describing what someone does. "Audit the last 6 months of escalated tickets." "Schedule a 2-hour rubric-drafting block on the lead's calendar each Tuesday."
4. **Write the *precondition*** — what must be true *before* the action is doable? "Audit-template is available." "Lead's calendar has been audited and 2-hour blocks identified." This often becomes the *outcome* of an earlier step, creating the sequence.
5. **Group action + precondition with an AND junctor** — both are needed to produce the outcome.
6. **Chain outcomes to next-step preconditions.** The outcome of step N becomes (part of) the precondition for step N+1. The TT reads as a directed chain of triples.
7. **Stop when each step is something a named person can do in a named day.**

Worked example (continued)

From Chapter 7: IO `Ticket-type audit complete`. Let's decompose.

`Cmd+K → New diagram → Transition Tree`. Empty TT canvas opens.

Triple 1:

- **Action:** Pull last 6 months of escalated tickets via the helpdesk's API.
- **Precondition:** API token + read-access scoped to escalation tags.
- **Outcome:** A CSV of ~2400 escalated tickets with type tags and resolution times.

Add three entities (Action, Precondition, Outcome — TP Studio's TT palette gives you the slots). Group Action + Precondition as AND. Connect to Outcome.

Triple 2:

- **Action:** Bucket the CSV by type and compute escalation rate per bucket.
- **Precondition:** CSV exported AND a one-page categorization rubric agreed in advance.
- **Outcome:** A ranked list of ticket types by frequency-weighted resolution time.

Note Triple 2's precondition includes Triple 1's outcome ("CSV exported"). Connect Outcome-1 into Precondition-2's AND group.

Triple 3:

- **Action:** Mark the top 20% by impact and circulate to L2 candidates for sanity-check.
- **Precondition:** Ranked list available AND L2 candidates identified.
- **Outcome:** Hardest-20% list signed off by the L2 candidates and the lead.

Triple 4:

- **Action:** Publish the hardest-20% list as a workspace doc.
- **Precondition:** List signed off.
- **Outcome: IO achieved.** Hardest-20% list defined.

The TT now reads, top to bottom, as four executable steps, each with a precondition that points back to the prior outcome. A reader looking at it knows exactly what to do on Monday.

🛠 **How TP Studio helps:** The `complete-step` validator (CLR tier `sufficiency`, TT-only) fires on any Action whose outgoing edge to its Outcome lacks a non-action sibling (the precondition role). It nudges you toward the triple structure rather than letting you draw a chain of bare actions.

Need and working assumption

The action / precondition / outcome triple is the *structural* unit on the canvas. The classical Transition Tree carries a second triple that lives inside the action itself — the reasoning that justifies it. Besides its Step # ordering, a TT **Action** exposes two optional fields in the inspector: a **Need** ("why is this step needed?") and a **Working assumption** ("the belief that makes this action sufficient"). Read together with the action they spell out the canonical Transition-Tree triple — Action ← Need ← Working Assumption — which is the discipline that stops a TT from degrading into an unexamined checklist: every step should be able to say *why* it's there and *what it's betting on*.

Both are free text and both are optional; a fresh step shows neither. Fill them when a step's rationale isn't self-evident — especially the working assumption, which is exactly the place a reviewer will push ("are we sure that's enough?") and exactly the kind of belief worth writing down before the rollout proves it right or wrong.

Action eligibility

Once your steps carry entity states (Chapter 3), the TT stops being a static plan and starts telling you *what's runnable right now*. Select an action and the Entity Inspector shows an **eligibility readout** that folds the effective states of that action's preconditions:

Status	Meaning
Eligible (emerald)	Every precondition feeding the action's outcome is `true`. Go.
Blocked (rose)	At least one precondition is `false`. The readout names the offender so you know exactly what's in the way.
Pending (amber)	A precondition is `unknown` or `disputed` — the step isn't blocked, but it isn't confirmed ready either.
n/a	The action has no precondition slot (a bare action with nothing to gate it).

Only the *true* preconditions count: sibling actions and assumptions feeding the same outcome are ignored, and a precondition that derives `true` from propagation (without a manual state) still makes the step eligible. Combined with what-if speculation, this answers the live planning question — "if I flip this upstream outcome to done, which steps unblock?" — without editing the saved plan.

Prefer it on the canvas? **Settings → Display → Show action-eligibility badge** mirrors the readout as an at-a-glance pill on each Action node's right edge — emerald `✓` eligible, rose `✗` blocked, amber `…` pending — so you can scan a whole tree for what's runnable without selecting each step. It's off by default (a state-less tree would read all-pending) and reflects speculation live, just like the inspector readout.

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → New Transition Tree` .
> - ***Action-eligibility readout*** *in the Entity Inspector — eligible / blocked / pending / n/a, folded from precondition states. Optionally mirrored as an at-a-glance ✓ / ✗ / … badge on Action nodes via* ***Settings → Display → Show action-eligibility badge****.*
> - `action` *entity type (cyan stripe) — TT-specific.*
> - `complete-step` ***validator*** *flags actions without paired preconditions.*
> - ***AND-junctor grouping*** *is essential to the triple structure; the gesture is the same as elsewhere (select edges → Group as AND).*
> - ***Reasoning narrative export*** *turns the TT into a numbered list ("1. Given X, do Y to obtain Z. 2. Given Z, do…") that pastes into a project doc.*

> ***💡 Practitioner tips***
>
> - ***Name the actor*** *in the Action title — "Maria pulls last 6 months of tickets" not "Pull last 6 months of tickets." Anonymous actions don't get done.*
> - ***Estimate effort in the description.*** *TT actions vary from "5 minutes" to "2 weeks." A reader needs to know which.*
> - ***Use Notes for caveats.*** *The TT is the canonical plan; caveats and contingencies live as Notes beside the chain so the canonical reading stays clean.*

> ⚠ ***Common mistakes***
>
> - ***Skipping preconditions.*** *A TT without preconditions is just a to-do list. The precondition is what makes the dependency structure visible — and what makes the diagram catch the case where two steps look ready but one is actually blocked.*
> - ***Triples too coarse.*** *"Build the rubric" is not a TT triple; it's an IO. The TT decomposes IOs into steps small enough for one person to do in one sitting. "Maria drafts the severity-1 rubric section" is a triple.*

> 🛑 ***When to stop***
>
> - *Every leaf action is sized to fit a person-day or less.*
> - *Every action has its precondition AND-grouped explicitly.*
> - *The outcome chain is traceable from the first action to the IO it produces.*
> - *The reasoning-narrative export reads as a runnable plan.*

🔁 **Chain to next:** the TT is the operational plan. The Goal Tree (next chapter) is a *strategic* decomposition — the frame around the entire CRT→TT process when the constraint is the goal itself.

→ Continue to Chapter 9 — Goal Tree

Chapter 9 — Goal Tree

What does success look like?

> 🎯 ***What this process is for*** *A Goal Tree (originally Intermediate Objective Map) decomposes a single Goal into Critical Success Factors (CSFs) and Necessary Conditions (NCs). It answers: "If success means X, what would have to be true?" Often drawn before a CRT to set the frame; sometimes drawn instead of one, when the problem is "we don't know where to start" rather than "we know things are bad."*

The premise

The CRT works bottom-up from symptoms. The Goal Tree works top-down from objective. The two are duals; you can sometimes infer one from the other.

The Goal Tree is the right starting move when:

- The organization is *planning* (annual strategy, new product launch) rather than *diagnosing* (chronic UDEs).
- You have a high-level goal and need to know what success would actually entail.
- You're trying to align stakeholders before the work starts — the Goal Tree is a shared map.

The structure: one Goal at top; 3-5 CSFs in the middle (the must-be conditions for the Goal); 5-15 NCs at the bottom (sub-conditions feeding each CSF). Each layer connects to the next with necessity edges.

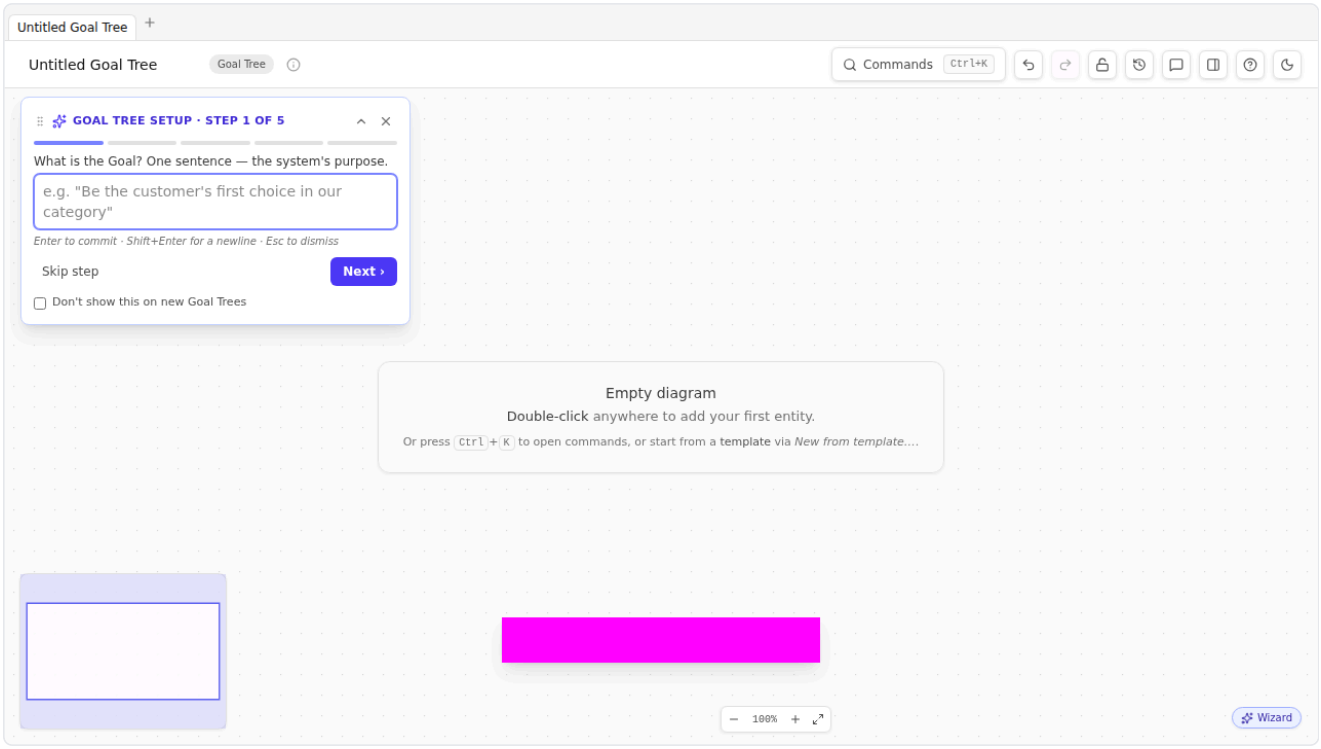
The method

1. **Write the Goal.** One sentence. Specific. Time-bounded if possible. "Hit $10M ARR by EOY 2026." "Ship Customer Portal v2 with 80%+ adoption by Q3."
2. **Brainstorm 3-5 Critical Success Factors.** "What would have to be true for the Goal to be achieved?" Each CSF should be a *condition*, not an action. "Sales pipeline is healthy" is a CSF; "Hire 3 AEs" is not (that's a NC, or a TT action).
3. **Under each CSF, list Necessary Conditions.** Sub-conditions that feed the CSF. Aim for 2-4 per CSF.
4. **Use necessity edges throughout.** "In order to achieve [Goal], we must satisfy [CSF]." "In order to satisfy [CSF], we must have [NC]."
5. **Walk the tree bottom-up.** Starting from a leaf NC, walk up to the Goal. The chain should read aloud as a coherent argument. If it doesn't, the structure is wrong.
6. **Stop when each NC is something you can plan against.** NCs that are themselves wide open (need their own Goal Tree) get bumped to the next iteration; mark them as "needs decomposition" via the Inspector's description.

Worked example

Switch gears. Imagine you're the new GM of a B2B SaaS product line, planning your first year.

`Cmd+K → New diagram → Goal Tree`. The **Creation Wizard** opens at step 1, prompting for the Goal.



🛠 **How TP Studio helps:** The Goal Tree creation wizard mirrors the EC wizard's pattern: 5 steps, each prompting for one slot (Goal → CSF 1 → CSF 2 → CSF 3 → first NC). Each step commits live so partial walks leave the canvas useful.

Wizard step 1: **Goal**. Type **Hit $10M ARR by EOY 2026**. Next.

Step 2: **CSF 1**. *What's one thing that has to be true for that goal?* **Sales pipeline coverage of 3x quota each quarter.** Next.

Step 3: **CSF 2**. **Net retention >= 110%.** Next.

Step 4: **CSF 3**. **Two new vertical-specific use cases shipped and adopted.** Next.

Step 5: **First NC**. Pick a CSF to decompose. Take "Sales pipeline coverage of 3x quota" — what's required? **AE headcount of 8 by end of Q1.** Commit.

Wizard closes; the canvas now has a Goal Tree with one Goal, three CSFs, and one NC. Continue building manually:

- Under "Sales pipeline coverage", add NCs: **Marketing-qualified-lead flow at 200/mo by Q2**, **Pipeline-review cadence weekly with consistent definition of "qualified"**.
- Under "Net retention >= 110%", add NCs: **Customer success function staffed at 1:1.5M ARR**, **Quarterly business review cadence with strategic accounts**, **Expansion playbook documented + trained**.
- Under "Two new vertical use cases", add NCs: **Product-research effort sized at 1 PM + 1 designer for 8 weeks per vertical**, **2 design-partner contracts signed per vertical**.

You now have a Goal Tree of 1 Goal, 3 CSFs, 7 NCs. Read each chain aloud: *"In order to hit $10M ARR by EOY 2026, we must have sales pipeline coverage of 3x quota each quarter. In order to have sales pipeline coverage of 3x quota, we must have AE headcount of 8 by end of Q1."* Coherent.

Multi-goal Goal Trees

Sometimes the strategic frame has two or three top-level goals. Conventional Goal Tree practice says no — the Goal Tree's discipline is its singular goal. But organizations do sometimes have legitimate dual top-level objectives ("Hit $10M ARR AND keep team headcount under 60").

TP Studio supports this with a *soft* warning: the `goalTree-multiple-goals` validator fires (clarity tier) when a Goal Tree has more than one Goal. The warning is dismissible. The warning's one-click action is `convert-extra-goals-to-csfs` — which downgrades every Goal except the oldest to a Critical Success Factor. That's usually the right move: the second "Goal" is actually a constraint on the first.

If you genuinely need two Goals, dismiss the warning and proceed. Just be honest about whether the constraint framing fits better.

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → New Goal Tree` *→ opens the* ***Creation Wizard*** *(Goal → CSF1 → CSF2 → CSF3 → first NC, 5 steps).*
> - `goal` *entity (sky stripe),* `criticalSuccessFactor` *(teal stripe),* `necessaryCondition` *(lime stripe) — Goal-Tree-specific palette.*
> - `add-nc-child` *verb: select a CSF or NC, single-entity toolbar → Add NC. Mirrors the wizard's step 4 logic for adding more NCs after the wizard closes.*
> - `promote-to-goal` *verb: select an entity that isn't yet a Goal, toolbar → Promote to Goal. Useful when an NC turns out to be the real strategic objective.*
> - `goalTree-multiple-goals` *validator with the* `convert-extra-goals-to-csfs` *one-click action.*
> - ***Reasoning narrative export*** *renders the Goal Tree as a top-down necessity argument suitable for stakeholder packs.*

> ***💡 Practitioner tips***
>
> - ***Time-bound the Goal.*** *"Hit $10M ARR" is weaker than "Hit $10M ARR by EOY 2026." The time bound is what makes the tree falsifiable.*
> - ***CSFs are conditions, not projects.*** *"Healthy pipeline" is a condition. "Run the pipeline-improvement project" is not.*
> - ***Goal Tree first, CRT second*** *when you're planning.* ***CRT first, Goal Tree implicit*** *when you're diagnosing existing pain. Both flows are valid.*
> - ***Re-draw the Goal Tree annually.*** *Last year's strategic frame ages out. The Goal Tree is meant to be a living artifact; if it's been static for two years, you've stopped using it.*

> ⚠ ***Common mistakes***
>
> - ***Too many CSFs.*** *Three is a sweet spot; five is okay; seven means you haven't decided what matters. The CSF layer is the strategic shape of the goal; it should be small enough to remember.*
> - ***CSFs and NCs at the wrong level.*** *"Increase MRR 10% MoM" is too granular for a CSF (that's a metric to track, not a condition to have). "Have a pipeline" is too vague for an NC. Calibrate; rewrite.*
> - ***Confusing Goal Tree with the to-do list.*** *The Goal Tree is what would be true, not what would be done. Actions belong in the PRT/TT downstream.*

> 🛑 ***When to stop***
>
> - *One Goal, time-bounded, specific.*
> - *3-5 CSFs, each a condition.*
> - *2-4 NCs per CSF, each something you can plan against.*
> - *Read-aloud passes at every chain.*
> - *You can describe the strategic frame in one sentence using the Goal-Tree vocabulary.*

🔁 **Chain to next:** the Goal Tree is the strategic frame. The S&T tree is the *deployment* of that frame across the organization, one operational level at a time.

→ Continue to Chapter 10 — Strategy & Tactics Tree

Chapter 10 — Strategy & Tactics Tree

Deploying operationally

> 🎯 ***What this process is for*** *A Strategy & Tactics Tree (S&T) is the deployment-grade decomposition of a strategy across an organization, from CEO-level down to individual contributor. Each node is a 5-facet card holding Necessary Assumption (why this matters), Strategy (what), Parallel Assumption (why this specific approach), Tactic (how), and Sufficiency Assumption (why this is enough). Used for cross-functional rollouts, big strategic programs, and as the working document for a TOC-style strategic deployment.*

The premise

The Goal Tree is *what success looks like*. The S&T tree is *what each person does on Monday morning when we deploy the strategy across 30 teams*.

S&T came late to the TOC tradition (Goldratt formalized it in the 2000s) and is the most operationally specific of the thinking processes. Each node carries its own self-contained micro-argument across five facets. The facet names below are Goldratt's; the one-line characterizations are how this book reads them in practice:

Facet	A reading
Necessary Assumption (NA)	The world-state above us has changed enough that *not* acting at this layer has become unacceptable. Captures the trigger handed down from the parent node.
Strategy	A statement of the result we are committing to deliver at this layer — what good looks like, not how we get there. Phrased as an outcome, not an action.
Parallel Assumption (PA)	Of the strategies that *could* respond to NA, why this one. Captures the comparative choice — the alternatives we considered and what made this one the right pick.
Tactic	The concrete things people will do. Verbs, not aspirations. Granular enough that you can tell whether they happened.
Sufficiency Assumption (SA)	If we complete the tactic, the strategy is fulfilled. Captures the bet that Tactic → Strategy is a real causal chain, not a hopeful one.

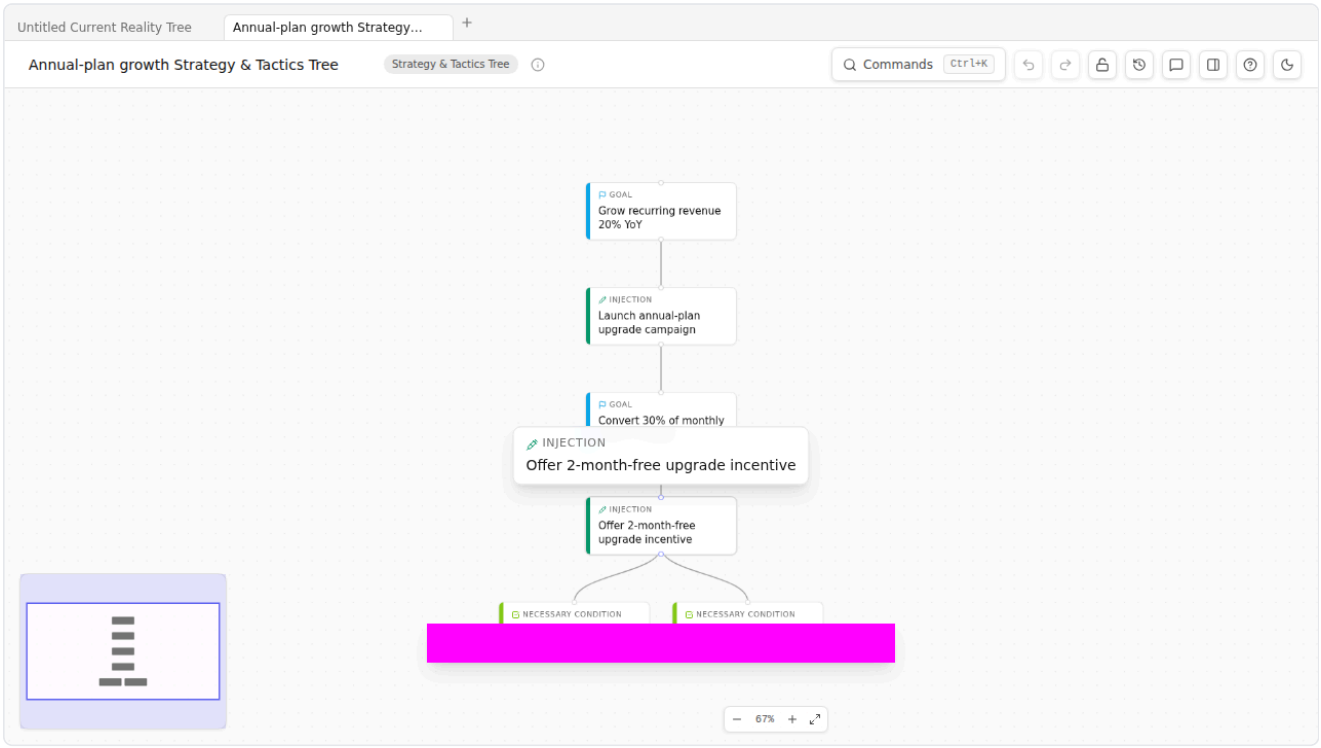
A complete S&T tree at organizational scale might be 40-100 nodes, decomposed across 4-6 levels. Few teams ever build one. The skill matters mostly when you're either *running* a TOC-style deployment or *evaluating one someone else built*.

The method

1. **Top-level S&T node:** the program goal. Fill all five facets.
2. **Decompose into 3-5 sub-strategies.** Each child S&T node represents a strategy that *contributes to fulfilling* the parent's Tactic.
3. **At each level, every facet is filled.** Empty facets are CLR violations (the `st-tactic-assumptions` validator flags missing facets per node).
4. **Continue down until the leaf tactic is operational** — a thing a named team can plan against.

Worked example (sketch)

Open the example: `Cmd+K → Load example → Strategy & Tactics Tree`. TP Studio ships a 2-level S&T example that demonstrates the 5-facet rendering.



A short example tree might look like:

- **Top node:** Strategy = "Achieve 30% growth in customer base by EOY 2026" with the 4 surrounding facets.
- **Child 1:** Strategy = "Expand into healthcare vertical." NA = "Healthcare is underserved relative to TAM." PA = "Healthcare vertical chosen over EdTech because compliance moat is more durable." Tactic = "Build 2 healthcare-specific compliance features, hire 1 vertical specialist." SA = "Healthcare-specific feature set + named specialist convert 3 healthcare design partners → 12 paid → 60 paid within 18 months."
- **Child 2:** "Strengthen referral motion." (Each facet filled similarly.)
- **Child 3:** "Optimize self-serve onboarding."

Each child is a fully-formed micro-argument. Disagreements about deployment surface at the facet level: "I disagree with our PA — I think the right vertical is FinTech." That's a productive disagreement; it points at one specific facet on one specific node, not vague "I don't like the strategy."

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → New Strategy & Tactics Tree`.
> - ***5-facet rendering***: *an `injection` entity in an `'st'` diagram with the reserved `stStrategy` / `stNecessaryAssumption` / `stParallelAssumption` / `stSufficiencyAssumption` attributes renders as a tall 5-row card with the four facets above the title. Click any row to inline-edit.*
> - `st-tactic-assumptions` ***validator*** *(CLR sufficiency tier) flags any tactic with fewer than three necessary-condition feeders. Useful in real S&T trees where each tactic should rest on several supporting NCs.*
> - ***6-step method checklist*** *in the Document Inspector — the canonical S&T sequence (write top-level strategy → fill all 4 facets → decompose into children → fill each child's facets → audit completeness → present).*

> ***💡 Practitioner tips***
>
> - ***Write all 5 facets before moving on.*** *A partially-facetted node is worse than no node — it implies completeness without delivering it.*
> - ***NA and SA are about argumentation, not summary.*** *"NA: It's important to grow" is useless. "NA: Without 30% growth, we don't make Series C metrics, and Series C is needed by Q3 2027" is useful.*
> - ***The PA is where alternatives go to die.*** *A good PA reads: "Strategy X chosen over Strategy Y because [reason], and over Strategy Z because [reason]." If you've never written that explicitly, you haven't seriously considered alternatives.*

> ***⚠ Common mistakes***
>
> - ***Skipping facets to "get to the point."*** *The facets are the point. Without them the S&T tree is just a project plan.*
> - ***Treating it as a Gantt chart with extra steps.*** *S&T isn't sequencing; it's argument structure. Sequencing lives in PRT/TT.*

> ***🛑 When to stop***
>
> - *Every node has all five facets filled.*
> - *Leaf tactics are operational (team-can-plan-against).*
> - *Each parent's tactic is supported by ≥3 child strategies (the `st-tactic-assumptions` validator stops firing).*
> - *You can hand the tree to a deployment lead and they can build a rollout calendar from it without coming back with structural questions.*

🔁 **Chain to next:** the seven structured TPs (CRT, EC, FRT, PRT, TT, Goal Tree, S&T) are the canonical kit. The freeform diagram is for *when the structure doesn't fit*.

→ Continue to Chapter 11 — Freeform diagrams

Chapter 11 — Freeform diagrams

When none of the above fits

> 🎯 ***What this process is for*** *A Freeform diagram is TP Studio's escape hatch. No built-in TOC entity types; no per-diagram CLR validators; just the canvas plus whatever custom types and attributes you define. Useful when the situation is causal-but-not-canonical: stakeholder mapping, system architecture, knowledge graphs, an early-stage think where you haven't yet decided which TP it'll become.*

When freeform is honest

- **The structure is causal, but the TOC types don't fit.** "Adoption curve diagram with cohorts" — UDE/effect/root-cause doesn't apply. Freeform with custom entity classes does.
- **You're at the "I don't know what I'm doing yet" stage.** Sometimes you sit down with a messy problem and need to draw things while you figure out *what kind of analysis it'll become*. Freeform lets you start without committing.
- **The diagram is a deliverable, not an analysis.** Documentation. A reference model. A team's shared mental model. Freeform gives you the same canvas affordances without prescribing methodology.

When freeform is dishonest

- **You're avoiding the discipline of a real TP.** "I'll just use freeform for now" is sometimes a flag that you haven't decided what you're really doing, and the cost of that ambiguity is high. A CRT *forces* you to find root causes; freeform lets you avoid them. If you find yourself drawing freeform when one of the structured types would apply, that's a signal.
- **You want to skip the CLR.** The CLR validators don't fire on freeform diagrams (other than the universal `empty-title` clarity check). That's a feature for genuine non-causal diagrams; a bug for "I want my CRT but without the warnings."

Custom entity classes

The real power of freeform is **custom entity classes** — per-document user-defined types. Open the Document Inspector → Custom entity classes section. Click "Add class".

For each class, set:

- **Label.** What it's called. "Stakeholder", "System Boundary", "Risk", whatever.
- **Color.** From a curated palette.
- **Icon.** One of 57 Lucide icons.
- **`supersetOf`.** Optional. Pick a built-in type the custom class *extends* — `effect`, `rootCause`, etc. Validators that fire on the built-in will also fire on your custom class. Useful when you want a renamed type ("Friction Point" superseding `effect`) but want the validators to still apply.

Custom classes live on the document, so they travel with JSON exports + share links. The class palette appears in the Entity Inspector's Type grid alongside the built-ins.

Custom attributes

Each entity carries a `Entity.attributes` record — typed key/value pairs (`string` / `int` / `real` / `bool`). The Entity Inspector exposes a key/value editor below the warnings list. Use it when an entity needs structured metadata beyond the standard fields:

- A stakeholder entity with `decisionAuthority: int`, `name: string`, `lastEngaged: string`.
- A risk entity with `likelihood: real`, `impact: real`, `mitigation: string`.
- A system component entity with `slaTarget: int`, `currentMTBF: int`.

Attributes round-trip through JSON, CSV exports, and OPML exports.

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → New Freeform diagram`.
> - ***Custom entity classes*** + ***icon picker*** *(57 Lucide icons) in the Document Inspector.*
> - ***Per-entity attributes*** *key/value editor in the Entity Inspector.*
> - ***No diagram-type CLR firing*** *— only the universal* `empty-title` *clarity check applies.*
> - ***Group presets*** *still work — you can structure regions of a freeform diagram with Negative Branch / Reinforcing Loop / Archive presets.*

> ***💡 Practitioner tips***
>
> - ***Use freeform sparingly.*** *If 30% of your TP Studio docs are freeform, look at whether you'd benefit from being more disciplined about TP selection.*
> - ***Promote when you're ready.*** *If a freeform doc starts to look like a CRT-in-progress, switch the diagram type (Document Inspector → Diagram type). Built-in types and validators activate. The structure you already drew survives.*

> ***⚠ Common mistakes***
>
> - ***Using freeform as "default" because you didn't read this book.*** *The structured TPs exist because they catch errors freeform won't. Reach for them.*
> - ***Inventing 12 custom entity classes for a 20-entity doc.*** *Custom classes are powerful; they're also confusing for readers. Three classes is a lot.*

> ***🛑 When to stop***
>
> - *The diagram answers the question it was drawn to answer.*
> - *You haven't accumulated CLR-style mistakes (freeform won't catch them; you have to catch them yourself).*
> - *The set of custom entity classes is small and understandable to a future reader.*

🔁 **Chain to next:** Part 2 is done — you know every TP and when to use each. Part 3 covers the cross-cutting skills: groups, the CLR in depth, iteration via revisions and side-by-side compare.

→ Continue to Chapter 12 — Groups, assumptions, injections

Chapter 12 — Groups, assumptions, injections

> *Three cross-cutting features. Each is a small addition to your repertoire, but each one shows up in nearly every Part 2 chapter, so this chapter is the canonical reference.*

Groups

A **group** in TP Studio is a labeled rectangle around a set of entities. It's structural metadata (which entities belong together) plus a visual chrome (the boundary box on the canvas). Use groups when a region of your diagram has a coherent meaning that the entities alone don't convey.

Three operations:

- **Create a group:** Multi-select entities (shift-click or marquee), then `Cmd+K → Group selected as new group`. The dialog asks for a title.
- **Nest groups:** A group's Group Inspector exposes a "Nest into parent group" picker. Groups can hold sub-groups indefinitely.
- **Collapse a group:** Click the chevron on the group title bar OR Group Inspector → Collapse. The group's contents disappear; the group renders as a single "collapsed-root card" the user can re-expand.

Group presets are the most useful feature here. TP Studio ships five preset (title, color) pairs derived from canonical TOC group naming:

Preset	Color	Use when…
Negative Branch	Slate	The grouped entities are a NB sub-tree (FRT-specific).
Positive Reinforcing Loop	Emerald	The grouped entities form a reinforcing loop.
Archive	Slate (collapsed by default)	The grouped entities have been trimmed out of the active analysis but you want to preserve them.
Step	Indigo	Marking a step of a multi-stage method (often used in S&T trees).
NSP Block	Amber	Marking "Negative Side Product" — a class of negative branch.

Use them. They keep your diagrams legible to other TOC practitioners who read your work.

🛠 `Cmd+K → Move selection to Archive group` is a one-shot that finds the existing Archive group or creates one (with the Archive preset) and moves the selected entities into it. Useful for cleaning a working diagram before presenting.

Assumptions

Assumptions are first-class entities (since schema v7). They sit beside the diagram — not on the causal path — and attach to a specific edge they pertain to.

On a CRT/FRT: Assumptions appear as a small violet pill on the edge they pertain to. The pill is clickable; clicking it selects the edge and opens the Inspector to the assumption.

On an EC: Assumptions live in the **AssumptionWell**, an Inspector tab specifically for the EC. Each assumption record carries:

- A title (the claim).
- A `status`: `open`, `valid`, `invalid`.
- A free-text rationale (description).
- A link to a related injection (when valid: false → "and here's the fix").
- An `implemented` flag (used in the InjectionWorkbench for FRT carry-forward).
- A **kind** sub-type — `Necessary`, `Parallel`, `Sufficient`, or untyped. A compact chip cycles through the three roles plus untyped, mirroring the S&T facet vocabulary (Chapter 10): a *necessary* assumption is the trigger that makes acting unavoidable, a *parallel* assumption is why *this* approach over the alternatives, a *sufficient* assumption is the bet that the chosen tactic actually delivers. Tagging assumptions by kind lets an S&T reviewer see at a glance which part of each micro-argument an assumption is propping up.

The status field is what turns the assumption list from a brainstorm into a decision record. *Open* = "we haven't decided yet"; *valid* = "we've concluded this assumption holds"; *invalid* = "we've concluded this assumption is wrong" → this is where the cloud evaporates.

🛠 **Press** `A` with an edge selected to add an assumption to that edge.

Injections

An injection is an entity of type `Injection`. It represents a proposed change — a thing you would *do* to the system. Injections show up:

- In FRTs as the bottom-of-tree entities driving the desired-effect chain.
- In ECs as the resolution to a broken assumption (linked from the AssumptionWell).
- In TT and PRT as the top-of-tree thing being decomposed.

Two Inspector flags worth knowing:

- `implemented`: a per-injection toggle. The **InjectionWorkbench** lists all injections in a doc; toggling `implemented` marks one as "done", visually distinct. Useful for tracking rollout progress against an FRT.
- **Linked assumption**: when an injection was drafted as a response to a `valid: false` assumption, the link is stored explicitly so the AssumptionWell and the InjectionWorkbench cross-reference.

The injection flower

In Cohen's *TP Basics* an injection is never vetted from one angle. You probe it from three sides: the **Desired effects** it should produce (a Future Reality Tree), the **Negative branch** it might trigger (an NBR), and the **Plan** to implement it (a Prerequisite Tree). In TP Studio those three live as separate documents, stitched together with the "Link to entity in another tab…" cross-document links you met above — which is faithful to the method but scatters one injection's vetting across tabs, where it's easy to lose track of which side you haven't done yet.

"View the injection flower" gathers it back up. From an injection's inspector (or the palette command "View the injection flower…") it pulls that injection's cross-document links into the three petals — Desired effects, Negative branch, Plan — plus an "Other links" catch-all, and reads its development at a glance: the header says "N of 3 sides developed," and any side you've left empty shows a prompt rather than a blank ("No negative branch linked yet — ask 'what could go wrong?'"). Each row jumps to the linked entity, so the flower doubles as a launchpad back into

whichever tab needs more work. It's the one view that answers "is this injection actually finished?" without your having to remember where you put the pieces.

Sidebars

> 🛠 ***How TP Studio helps***
>
> - ***Group presets*** *(* `Cmd+K → Group inspector → Preset` *).*
> - ***AssumptionWell*** *in the EC Inspector — first-class assumption records with status + injection links.*
> - ***InjectionWorkbench*** *in the EC Inspector — listing + status of all injections in the doc.*
> - `View the injection flower` *— gathers one injection's cross-document links into Desired effects / Negative branch / Plan petals (+ Other links) and shows "N of 3 sides developed."*
> - `A` ***shortcut*** *with an edge selected — adds an assumption.*

> 💡 ***Practitioner tips***
>
> - ***Use the Archive group preset early.*** *When you're refining a CRT, you'll discover entities that are wrong or redundant. Don't delete them — Archive them. You'll often want to revisit "why did I think this?" later.*
> - ***Surface assumptions before they're broken.*** *The most useful assumptions are the ones you* ***don't*** *yet know are false. List liberally; classify later.*

> ⚠ ***Common mistakes***
>
> - ***Treating groups as visual chrome only.*** *Groups are structural — they affect exports, copy-paste, and the hoist-into-group feature. Use them for* ***meaningful*** *clustering.*
> - ***Letting assumptions accumulate without classification.*** *A list of 30 open assumptions is no better than no list. Classify them. Or remove them.*

🔁 **Chain to next:** the CLR is the discipline that makes assumptions into real reservations. Next chapter goes deep.

→ Continue to Chapter 13 — The CLR

Chapter 13 — The CLR — your validation conscience

> *The Categories of Legitimate Reservation are the discipline checks Goldratt taught for evaluating a causal claim. TP Studio surfaces them automatically as warnings. They are reservations a thoughtful colleague would raise. They are not errors.*

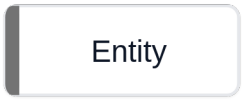
The classical map

Before diving into how TP Studio implements them, here is the classical reference — the eight-box layout Dettmer's *The Logical Thinking Process* uses to teach the CLR, each box pairing the category's question with a tiny example diagram. Cards mirror TP Studio's canvas: amber stripe for causal nodes, neutral grey for effects, red-dashed for the missing element each category catches.

CLARITY

Seeking to understand

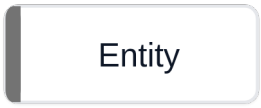
- Would I add any verbal explanation if reading the tree to someone else?
- Is the meaning of words unambiguous?
- Is the connection between cause and effect convincing at face value?
- Are intermediate steps missing (long arrow)?



ENTITY EXISTENCE

Complete, properly structured, valid statements

- Is it a complete sentence?
- Does it make sense?
- Free of embedded "if-then" statements?
- Does it convey only one idea?
- Does it exist in someone's reality?
- Can it be documented with evidence?



CAUSALITY EXISTENCE

Logical connection between cause and effect

- Does an "if-then" connection really exist?
- Does the proposed cause, in fact, result in the stated effect?
- Does it make sense when read aloud exactly as written?
- Is the cause intangible? (if so, look for a confirming predicted effect)

```
marker-end="url(#arrow-indigo)"/> <g filter="url(#card-shadow)"> <rect x="140" y="9" width="100" height="38" rx="5" ry="5" fill="#ffffff" stroke="#e5e7eb" stroke-width="1.25" /> <rect x="140" y="9" width="6" height="38" rx="5" ry="5" fill="#737373"/> <rect x="143" y="9" width="3" height="38" fill="#737373"/> <text x="193" y="33" text-anchor="middle" font-size="13" font-weight="500" fill="#111827" font-family="-apple-system, Segoe UI, Roboto, sans-serif">Effect</text>
```

CAUSE INSUFFICIENCY

A non-trivial dependent element missing

- Can the cause, as stated, result in the effect on its own?
- Are any significant causal factors missing?
- Is an ellipse (AND) required?
- Are any non-dependent causes included by mistake?

```
<text x="60" y="84" text-anchor font-size=" font-weight fill=" font-family apple-system Segoe UI, Roboto sans-serif"
marker-end="url(#arrow-indigo)"/> <line x1="114" y1="76" x2="158" y2="58" stroke="#dc2626" stroke-width="2" stroke-dasharray="5 3" marker-end="url(#arrow-red)"/> <g filter="url(#card-shadow)"> <rect x="160" y="33" width="100" height="38" rx="5" ry="5" fill="#ffffff" stroke="#e5e7eb" stroke-width="1.25" /> <rect x="160" y="33" width="6" height="38" rx="5" ry="5" fill="#737373"/> <rect x="163" y="33" width="3" height="38" fill="#737373"/> <text x="213" y="57" text-anchor="middle" font-size="13" font-weight="500" fill="#111827" font-family="-apple-system, Segoe UI, Roboto, sans-serif">Effect</text>
```

ADDITIONAL CAUSE

A separate, independent cause producing the same effect

- Is there anything else that might cause the same effect on its own?
- If the stated cause is eliminated, will the effect be (almost completely) eliminated?

CAUSE-EFFECT REVERSAL

Effect misstated as the cause

- Is the stated effect really the cause, and the stated cause really the effect?
- Is the stated cause really a reason why, or just how we know the effect exists?

```
marker-end="url(#arrow-indigo)"/>
<line x1="114" y1="76" x2="158"
y2="58" stroke="#6366f1" stroke-
width="2" marker-end="url(#arrow-
indigo)"/> <g filter="url(#card-
shadow)"> <rect x="160" y="33"
width="100" height="38" rx="5" ry="5"
fill="#ffffff" stroke="#e5e7eb"
stroke-width="1.25" /> <rect x="160"
y="33" width="6" height="38" rx="5"
ry="5" fill="#737373"/> <rect x="163"
y="33" width="3" height="38"
fill="#737373"/> <text x="213" y="57"
text-anchor="middle" font-size="13"
font-weight="500" fill="#111827" font-
family="-apple-system, Segoe UI,
Roboto, sans-serif">Effect</text>
```



```
marker-end="url(#arrow-indigo)"/> <g
filter="url(#card-shadow)"> <rect
x="140" y="9" width="100" height="38"
rx="5" ry="5" fill="#ffffff"
stroke="#e5e7eb" stroke-width="1.25" />
<rect x="140" y="9" width="6"
height="38" rx="5" ry="5"
fill="#737373"/> <rect x="143" y="9"
width="3" height="38" fill="#737373"/>
<text x="193" y="33" text-
anchor="middle" font-size="13" font-
weight="500" fill="#111827" font-
family="-apple-system, Segoe UI, Roboto,
sans-serif">Effect</text>
```

PREDICTED EFFECT

Additional corroborating effect resulting from the cause

- Is the cause intangible?
- Do other unavoidable outcomes of the proposed cause exist besides the stated effect?

```
marker-
end="url(#arrow-
indigo)"/> <line
x1="114" y1="58"
x2="158" y2="74"
stroke="#6366f1"
stroke-width="2"
stroke-dasharray="5
3" marker-
end="url(#arrow-
indigo)"/> <g
filter="url(#card-
shadow)"> <rect
x="160" y="6"
width="100"
height="38" rx="5"
ry="5"
fill="#ffffff"
stroke="#e5e7eb"
stroke-width="1.25"
/> <rect x="160"
y="6" width="6"
height="38" rx="5"
ry="5"
fill="#737373"/>
<rect x="163" y="6"
width="3"
height="38"
fill="#737373"/>
<text x="213" y="30"
text-anchor="middle"
font-size="13" font-
```

```
<text x="210"
      y="84"
text-
anchor="middle"
      font-
size="13" font-
weight="500"
fill="#111827"
      font-
family="-apple-
system, Segoe UI,
Roboto, sans-
serif">New</text>
```

TAUTOLOGY

Circular logic

- Is the cause intangible?
- Is the effect offered as the rationale for the existence of the cause?
- Are other unavoidable outcomes identifiable besides the proposed effect?


```
marker-end="url(#arrow-indigo)"/> <line
x1="134" y1="32" x2="114" y2="32"
stroke="#6366f1" stroke-width="2"
marker-end="url(#arrow-indigo)"/> <g
filter="url(#card-shadow)"> <rect
x="140" y="9" width="100" height="38"
rx="5" ry="5" fill="#ffffff"
stroke="#e5e7eb" stroke-width="1.25" />
<rect x="140" y="9" width="6"
height="38" rx="5" ry="5"
fill="#737373"/> <rect x="143" y="9"
```

```
weight="500"
fill="#111827" font-
family="-apple-
system, Segoe UI,
Roboto, sans-
serif">Effect</text>
```

```
width="3" height="38" fill="#737373"/>
<text x="193" y="33" text-
anchor="middle" font-size="13" font-
weight="500" fill="#111827" font-
family="-apple-system, Segoe UI, Roboto,
sans-serif">Effect</text>
```

This map is the territory; TP Studio's warnings are the tools that walk it. Goldratt's original six are the first six boxes; *Predicted Effect* and *Tautology* are Dettmer's additions, used mainly when a stated cause is intangible (you cannot observe it directly, only its consequences).

The six categories

Goldratt named six. The operationalization into validator-style rules that practitioner tools surface — what TP Studio does — owes a debt to William Dettmer's *The Logical Thinking Process* (2007), which formalized the categories for everyday use. Listed here in order of where they typically bite first as you draft a diagram:

1. **Clarity.** The reader can't tell what you mean. An empty title, a verb-less noun phrase, an abstraction so generic it could be glued to any tree.
2. **Entity existence.** You're asserting a state of the world. Is that state actually observable? "Engineering velocity is dropping" lives or dies on whether you can show it.
3. **Cause-effect existence.** The two entities exist; you've drawn an arrow between them; does the arrow correspond to anything in reality? "Engineering velocity dropped because Mercury went retrograde" fails here, however clean the diagram looks.
4. **Cause sufficiency.** This cause alone — without anything else lined up alongside it — produces this effect? Or does the effect require a co-cause that you haven't drawn? The category where AND-groups get born.
5. **Additional cause.** Is there a *different* cause that would also produce this effect? Two roads to the same destination — modelled with OR-junctors, often missed by single-path thinking.
6. **Cause-effect reversal.** Run the same arrow the other way around; does it still make sense? If the answer is "actually, maybe even more sense," you've inverted the causal direction. The classic trap inside a B2B sales org: "deals are slow because morale is low" — until someone points out the direction is "morale is low because deals are slow."

TP Studio's validator system implements rules in each of these tiers. The `Warnings` list in the Inspector surfaces them per-entity / per-edge. The `Start CLR walkthrough` palette command iterates them one at a time.

How TP Studio surfaces them

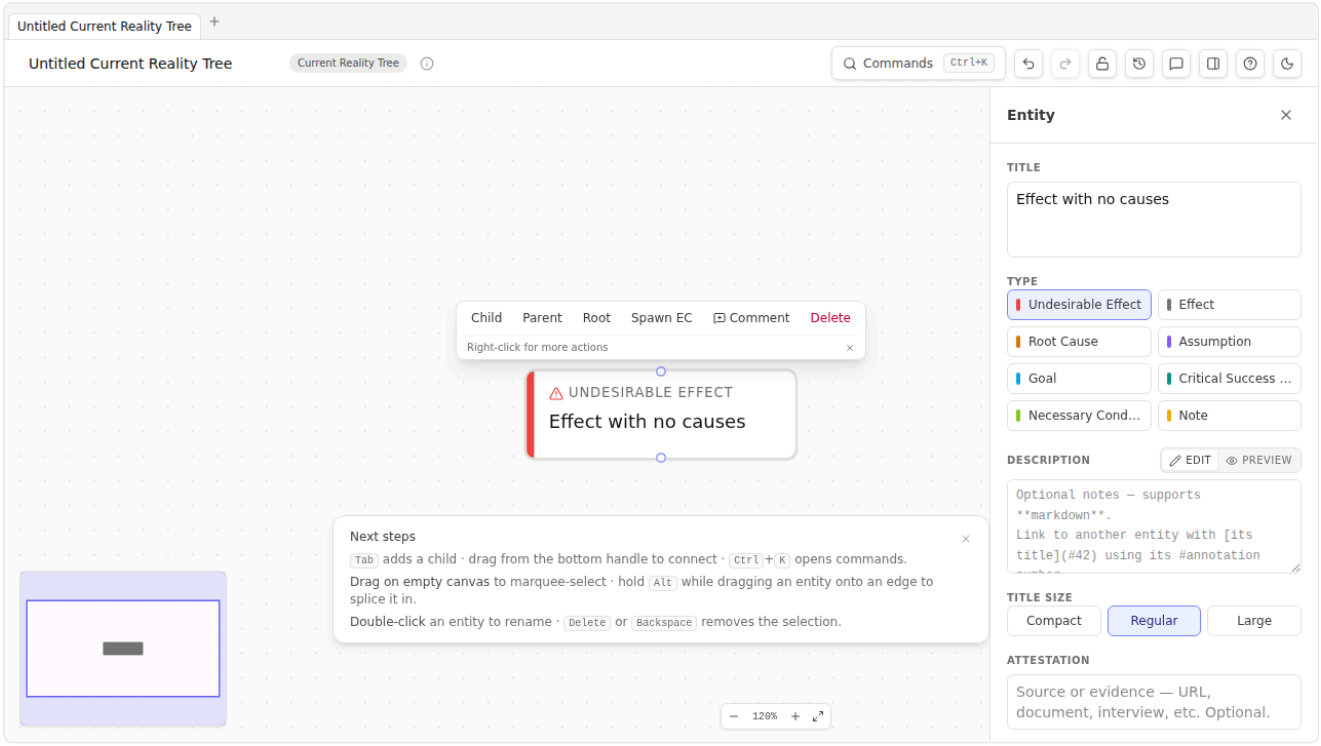
Each validator carries:

- A **tier**: `clarity`, `sufficiency`, `causality`, or `predictedEffect`. The tier governs the warning's color and the order it appears in the walkthrough wizard.
- A **diagram-type scope**: most rules fire only on specific diagram types (e.g., `ec-missing-conflict` is EC-only; `complete-step` is TT-only).
- A **trigger predicate**: a pure function over the doc that returns the set of entities/edges to fire on.
- Optionally, a **one-click action**: a `WARNING_ACTIONS` registry entry that resolves the warning. Example: the `convert-extra-goals-to-csfs` action on the `goalTree-multiple-goals` warning.

The full list is in Appendix C.

Reading warnings

Click any entity with an open warning. The Inspector's Warnings section lists them as bullet items with the tier color (yellow for `clarity`, amber for `sufficiency`, orange for `causality`, red for `predictedEffect`). Each warning has a one-line explanation; some carry a "Fix" button when a one-click action is available.



The walkthrough

`Cmd+K → Start CLR walkthrough` opens a modal that iterates every open warning, one at a time, with **Resolve** / **Open in inspector** / **Dismiss** actions. Useful for ratcheting through a complex diagram's warnings without manually clicking each entity.

The walkthrough is scope-limited to *open* warnings — once you dismiss a warning, it doesn't reappear unless the underlying condition changes. That makes the walkthrough a *clearing* gesture: run it before declaring a diagram done; if it's empty, you've considered every reservation.

Scrutinizing a single edge

The walkthrough clears what the validators *fired*. But absence of a warning is not absence of a reservation — a rule fires only when it can detect its trigger condition, and most of the CLR is too contextual for a predicate to catch. An edge can be warning-free and still be sloppy. The complementary move is to take one claim and interrogate it against the whole CLR by hand.

Select an edge and run **"Scrutinize this edge (walk the CLR questions)"** from the palette, or press the **"Scrutinize against the CLR"** button in the edge inspector. Either opens a walk through all eight canonical categories — Clarity, Entity existence, Causality existence, Cause sufficiency, Additional cause, Cause–effect reversal, Predicted-effect existence, Tautology — one at a time, *including the categories nothing flagged*. Each step states the

category as a question, shows any warning the validators did raise on that edge, and gives you a checkbox to tick as you genuinely consider it. The button is read-only, so it works under Browse Lock — you can scrutinize a colleague's diagram in a review without touching it. Where the walkthrough is a clearing gesture across the diagram, scrutiny is a *deepening* gesture on one edge: it forces you to ask every reservation of a single claim, not just the ones a rule happened to notice.

Dismissing warnings

Two ways to dismiss:

- **Resolve** — fix the underlying condition. The warning stops firing on its own.
- **Dismiss with explanation** — keep the condition, but record in the entity's `description` *why* you're accepting the reservation. The walkthrough stops surfacing this one. The audit trail lives in the description.

Don't dismiss without writing the explanation. A dismissed warning with no rationale is a debt you'll pay later when someone reads the diagram and asks "why does this UDE have no causes?"

Sidebars

> ***🛠 How TP Studio helps***
>
> - ***Per-entity / per-edge Warnings list*** *in the Inspector.*
> - ***`Cmd+K → Start CLR walkthrough`*** *— modal that iterates open warnings.*
> - ***One-click actions*** *on a subset of warnings (e.g., `convert-extra-goals-to-csfs`).*
> - ***Tier-color coding*** *in the warnings list: clarity (yellow) → sufficiency (amber) → causality (orange) → predicted-effect (red).*
> - ***Dismissibility*** *— every warning can be dismissed; dismissals don't recur until the underlying state changes.*
> - ***`Scrutinize this edge (walk the CLR questions)`*** *— walks one edge through all eight CLR categories as questions, including the ones nothing flagged; read-only, so it works under Browse Lock.*

> ***💡 Practitioner tips***
>
> - ***Walk-through before you present.*** *A reader will hit the warnings if you don't.*
> - ***Treat warnings as suggestions, not commands.*** *The CLR is a discipline framework; it doesn't always know your context. Apply judgment.*
> - ***The clarity tier is the most forgiving and most pedagogical.*** *If you're learning, work the clarity warnings until they're empty before tackling the higher tiers.*

> ***⚠ Common mistakes***
>
> - ***Dismissing without explanation.*** *Each dismissal is a future-self bug if the rationale isn't recorded.*
> - ***Treating all warnings as equal.*** *The four tiers are deliberately ordered. A `predictedEffect` warning is structural; a `clarity` warning might just be a typo.*

🔁 **Chain to next:** the CLR is the validation conscience. Iteration is the *building* conscience — revisions, branches, side-by-side compare are how a diagram improves over time.

→ Continue to Chapter 14 — Iteration

Chapter 14 — Iteration — revisions, branches, compare

> *A CRT done well is the diagram you have on Friday, not the diagram you drew on Monday. The H1–H4 features make iteration cheap, which is why the diagram you have on Friday is better.*

Capturing snapshots

A **revision** is a point-in-time snapshot of the document. TP Studio stores up to 50 revisions per document in `localStorage`, indexed by id with optional labels.

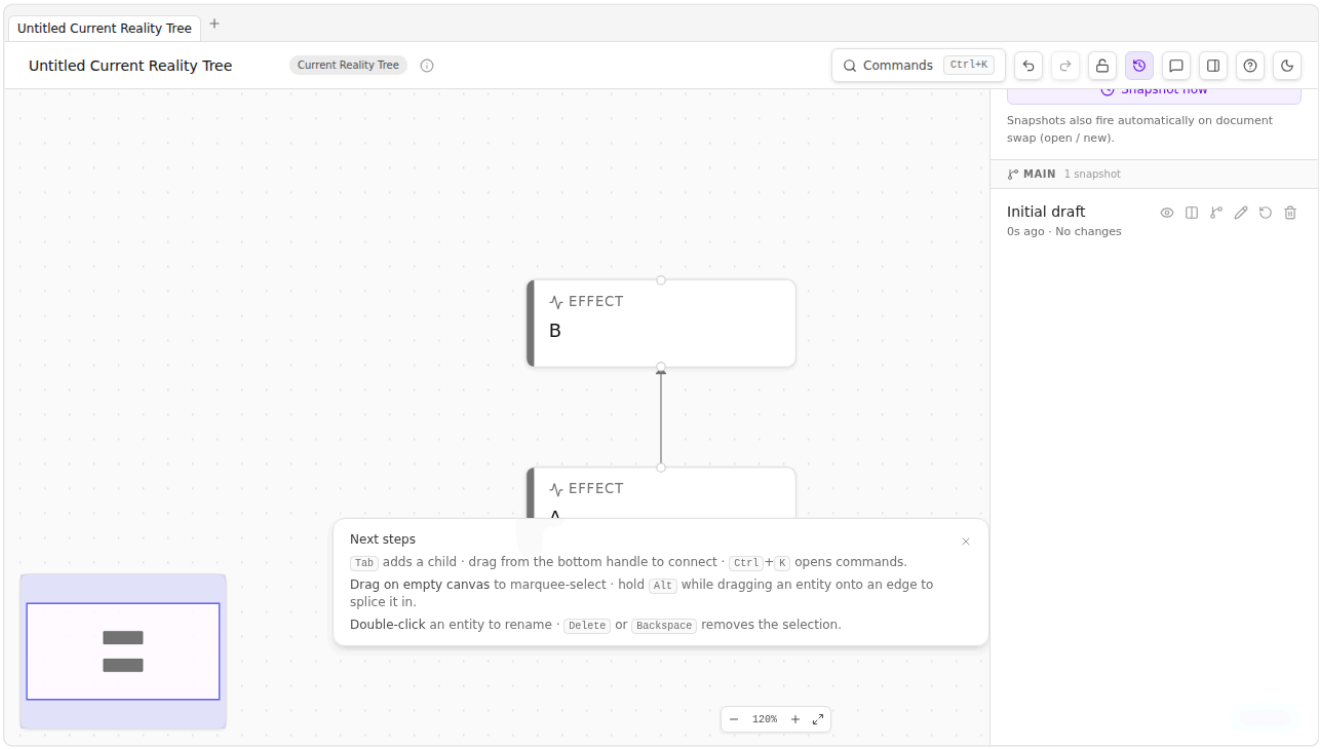
Capture one any time with `Cmd+K → Capture snapshot`. The palette command prompts for a label — give it one. "Before injecting the resolution." "After CRT review with Maria." "Pre-Negative-Branch hunt." The labels are how Future You will find this revision when the History panel has 30 of them.

Automatic snapshots also fire when:

- A document is loaded via Import / Load example / New from template (the prior doc is auto-snapped).
- A "safety snapshot" is captured before a Restore operation (so you can undo the undo).

The History Panel

TopBar → History button (or `Cmd+K → Open history panel`) opens a slide-in panel listing all revisions:



Each row shows the label, the timestamp, and three actions: **Restore** (replace the live doc with this revision), **Compare** (open the side-by-side dialog), and **Branch from here** (give this revision a `branchName`).

The panel groups revisions by branch. The default branch is `Main`. Branching is the way to explore alternatives — "what if we drop the L2 training and just hire one senior?" — without committing your speculation to the canonical history.

Branching

`Cmd+K → Branch from current revision` (or from a revision row in the History panel) prompts for a branch name. The branch becomes a labeled lineage; subsequent snapshots after a restore-from-this-branch belong to the new branch.

TP Studio's branch model is intentionally lightweight. It's not git. There's no merge; there's no checkout. A branch is just a label attached to a revision (and its descendants) so the History panel can group them visually. The point is exploration — "I want to keep my current line of analysis but also try this other thing."

🛠 **When to branch:** before a structurally risky edit ("I'm about to delete five entities and re-draw the cause chain"), before exploring a Negative Branch hunt that you might roll back, before presenting to stakeholders (so the live version is "what we showed them" and your subsequent edits don't change it retroactively).

Side-by-side compare

The H4 feature. Select a revision in the History panel → Compare. A fullscreen modal opens with two panels: snapshot on the left, live on the right.

Entities render as absolute-positioned cards; edges as SVG lines between them. Added entities are emerald-tinted; changed entities are amber-tinted; removed entities appear in the snapshot panel only (not in the live one).

Useful when you've done significant edits and want to see *exactly* what changed structurally. Common workflow: capture a snapshot, do 30 minutes of edits, side-by-side compare to verify the diff matches your intent.

Visual diff overlay (compare mode)

Lighter than the side-by-side dialog. `Cmd+K → Compare with revision…` picks a revision and overlays a per-entity ring tint on the live canvas:

- Emerald ring = added since this revision.
- Amber ring = changed since this revision.

A banner at the top tells you what revision you're comparing against. Esc exits compare mode.

Useful for "I want to know what's new in the live doc without leaving the canvas." The side-by-side dialog gives you more detail; this overlay is faster.

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → Capture snapshot` *(with optional label).*
> - ***History panel*** *slide-in (TopBar button or* `Cmd+K → Open history panel` *).*
> - ***Branch from here*** *action on any revision row.*
> - ***Side-by-side compare*** *dialog.*
> - ***Compare mode overlay*** *on the live canvas with per-entity diff tinting.*
> - ***Safety snapshot on restore*** *— you can undo a restore.*

> ***💡 Practitioner tips***
>
> - ***Label snapshots when you take them.*** *A timestamp alone is meaningless after a week.*
> - ***Snapshot before risky edits.*** *Cheaper than reconstructing.*
> - ***Branch before exploring.*** *"Try this thing" is much cheaper psychologically when you know you can return to the prior branch.*
> - ***Use side-by-side compare before presenting.*** *Comparing the "what stakeholders saw" snapshot to the live doc surfaces the changes you want to talk about.*
> - ***Capture review feedback as comments, not memory.*** *"After CRT review with Maria" is a set of comments pinned to the edges she questioned — they travel with the file, so the feedback and the diagram never drift apart (see Chapter 16).*

> ***⚠ Common mistakes***
>
> - ***Snapshot drift.*** *Capturing 50 unlabeled snapshots; later not knowing which is which. Label.*
> - ***Treating the history as version control.*** *It's not. There's no commit message, no diff in textual form, no merge. It's a time machine for your canvas, not git.*

🔁 **Chain to next:** Part 3 is done — you have groups, the CLR, and iteration. Part 4 takes the diagram out of your private workspace and into the world: verbalisation, sharing, workshops.

→ Continue to Chapter 15 — Verbalisation and walkthroughs

Chapter 15 — Verbalisation and walkthroughs

> *Verbalisation is the TOC tradition's name for "read it aloud, edge by edge, slowly." It is the single most under-rated discipline in TOC and the source of most of the "wait — actually I had this wrong" moments. TP Studio supports it four ways.*

Why aloud

When you read a diagram silently, your brain auto-completes. The arrow you drew Monday says what you *thought* it said when you drew it. Three days later you skim it and your skimming-brain confirms the original intent — even when the entity titles you wrote on Tuesday subtly broke the logic.

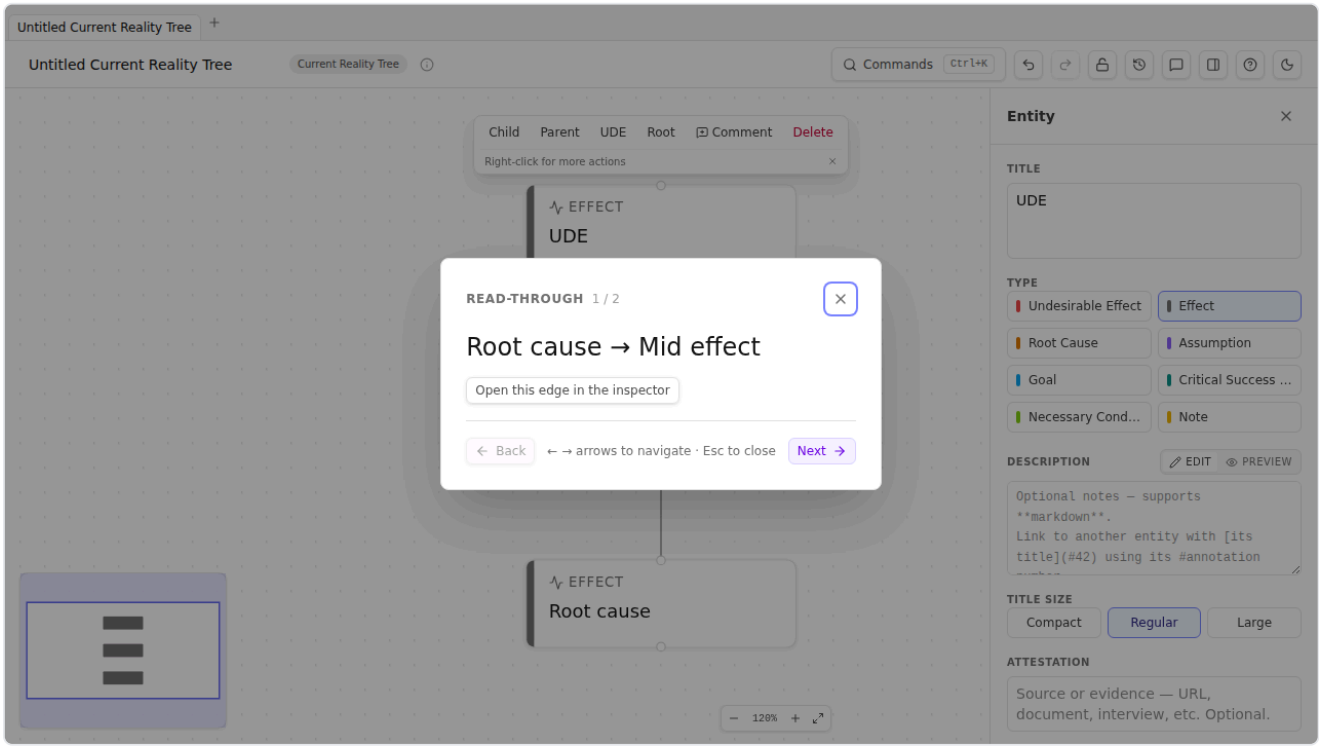
Reading aloud bypasses the autocompletion. The mouth and ear catch what the eye misses. The sentence "Because we ship slowly, customers churn" is structurally fine; the sentence "Because we ship slowly, our quarterly earnings" sounds incomplete in a way you'd notice immediately spoken but not noticed reading.

Goldratt and the practitioners who followed him (Dettmer especially) treat verbalisation as discipline, not affectation. You don't whisper through it; you actually say the sentences. In a workshop, the lead asks one participant to read; everyone else listens.

TP Studio's three supports

1. The Read-through overlay

`Cmd+K → Start read-through` opens a fullscreen overlay that walks every structural edge in topological order. Each step presents one sentence — *"Because [source], [target]"* — with arrow / space navigation between sentences.



Use it solo before you present. Use it with a participant in a workshop. The discipline imposes itself — you can't skim; the overlay shows one sentence at a time.

2. The all-at-once dialog

`Cmd+K → Read entire diagram at once` opens a scrollable panel rendering every edge's sentence in topological order in a single view, with a **Copy all** button that drops the full transcript into the clipboard. Use this when the step-through becomes tedious — a 50+ edge CRT has 50+ clicks in the read-through overlay, but the all-at-once dialog presents the same content in one scroll. Particularly useful for capturing the whole diagram's reasoning into a brief, postmortem, or Slack message without re-typing it.

The two modes are complementary: step-through forces *discipline* (you can't skim, you have to advance), all-at-once gives you *artifact* (the verbal form lives outside the canvas now).

3. The VerbalisationStrip (EC-only)

For Evaporating Clouds, the canonical reading is a single paragraph. The VerbalisationStrip renders it above the canvas, updating live as you edit. Settings → EC Verbal Style toggles between `neutral` ("we must") and `twoSided` ("they want / I want") framings.

Read the strip aloud after every structural edit on an EC. The "this sounds wrong" reaction is the most reliable validator the EC has.

4. The reasoning narrative / outline exports

Two Markdown exports under `Cmd+K → Export → Markup → Reasoning`:

Export	Shape	Use case
Reasoning narrative	Sentence-per-edge, prose paragraphs by chain	Pastes into a doc or a strategy brief; reads as argument prose.
Reasoning outline	Headings = terminal effects; nested bullets = cause chains	Outline form (like OPML); good for review meetings.

Both carry preambles: title, System Scope (if filled), EC conflict statement (on EC docs), and a per-diagram-type appendix (Core Driver for CRTs, triple-form for TTs).

The exports are how verbalisation leaves your screen. The audience that didn't sit through the workshop reads the narrative; they hear what you would have said.

Workshop technique — "read it back to me"

Three-person variant useful for workshop facilitation:

1. **Builder** drives the canvas, says nothing aloud.
2. **Verbaliser** reads each new edge aloud as it's drawn: *"You're claiming because A, B?"*
3. **Listener** says "okay" or "wait — really?"

The verbaliser is the discipline; the listener is the CLR. The builder is the analyst. Switching seats every 15 minutes spreads the discomfort of being verbaliser around the room.

Sidebars

> ***🛠 How TP Studio helps***
>
> - `Cmd+K → Start read-through` *— step-through fullscreen verbalisation overlay (one edge per page).*
> - `Cmd+K → Read entire diagram at once` *— single-view scrollable panel with Copy-all (full transcript).*
> - ***VerbalisationStrip*** *on EC canvas — live paragraph rendering.*
> - `Cmd+K → Start CLR walkthrough` *— partner discipline; iterates open warnings.*
> - ***Reasoning narrative export*** *— Markdown paragraph form per chain.*
> - ***Reasoning outline export*** *— Markdown nested-list form.*
> - ***EC Verbal Style toggle*** *— neutral vs. two-sided framing.*

> ***💡 Practitioner tips***
>
> - ***Aloud means aloud.*** *"I read it in my head" isn't verbalising.*
> - ***Read in the morning.*** *Tired brain skims worse than fresh brain. Don't verbalise after lunch.*
> - ***Capture a snapshot before you re-read.*** *When verbalising surfaces a structural problem, you'll be editing in five minutes. A pre-read snapshot is the baseline you'll compare against later.*

> ***⚠ Common mistakes***
>
> - ***Skipping verbalisation because the diagram "looks right."*** *Looking right and reading right are different. The diagram that looks right but reads wrong is the most common bug.*
> - ***Speed-reading the strip.*** *Slow down. The point of aloud is the time spent on each sentence.*

🔁 **Chain to next:** verbalised diagrams want to be shared. Next chapter covers exports, share links, prints.

→ Continue to Chapter 16 — Sharing your work

Chapter 16 — Sharing your work

> *TP Studio is local-first; nothing leaves your browser unless you choose to send it. The Export picker is the single doorway out. This chapter is the doorway's map.*

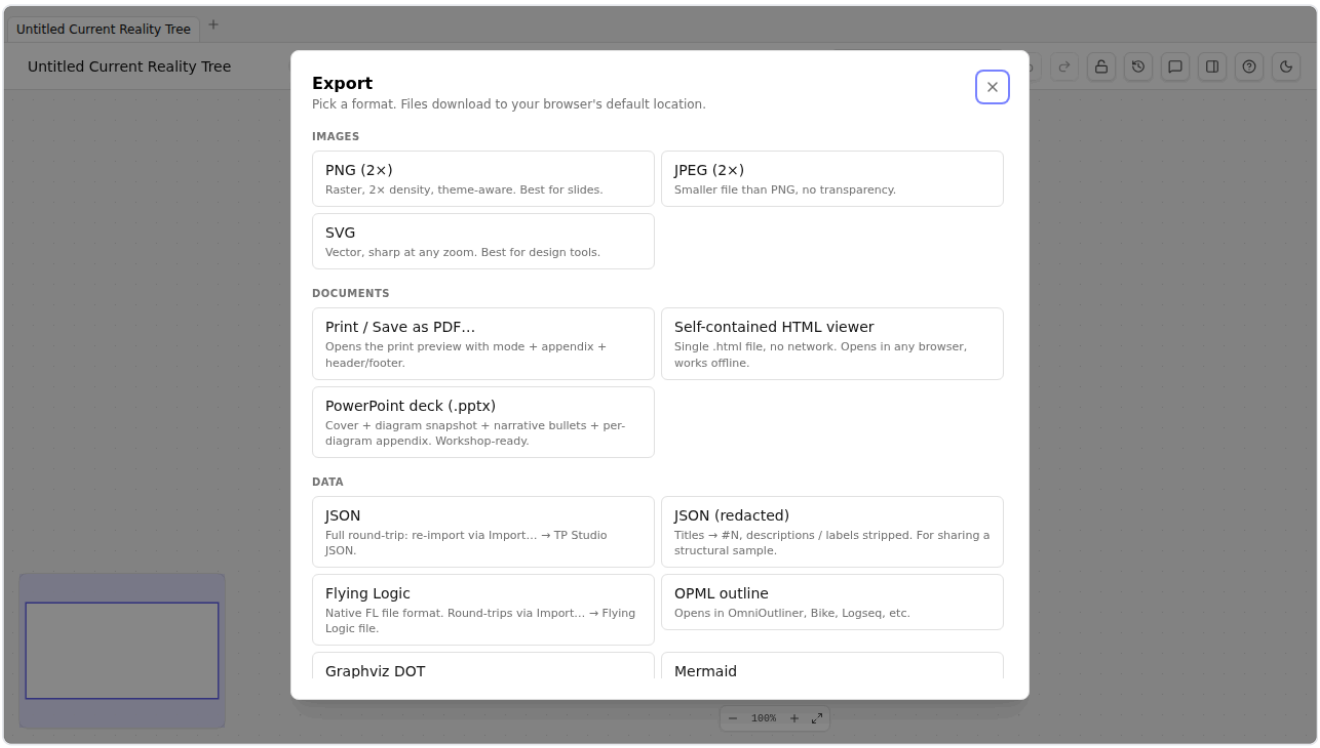
Three modes of sharing

Mode	Best for	Tradeoff
Standalone HTML viewer	Sharing the full diagram with someone who doesn't have TP Studio. Double-click to open.	Single self-contained file. Read-only. Heaviest in size.
Read-only share link	Quick share via Slack / email / chat. URL fragment encodes the full doc.	URL gets long for big diagrams. Receiver opens in their own TP Studio session (Browse Lock auto-engages).
Image / vector export	Pasting into a deck, doc, or wiki.	Static. Lossy if PNG; vector if SVG/PDF. No interactivity.

Pick by the audience. Engineers and analysts: share link. Slide deck and stakeholder pack: vector PDF. The non-technical "open this with the file" audience: standalone HTML viewer.

The Export Picker

`Cmd+K → Export` opens the unified picker:



Three groups:

Images

- **PNG** — high-DPI raster, theme-aware. 2× density by default. Good for ad-hoc paste.
- **JPEG** — lossy raster. Smaller files. Web-friendly.
- **SVG** — vector. Sharp at any zoom. Importable into Figma, Illustrator, etc.
- **PDF** — true vector PDF via jspdf + svg2pdf. Paginated if the diagram exceeds page-height. Optional annotation appendix.
- **Print / Save as PDF…** — opens the **Print Preview Dialog** with mode picker (Standard / Workshop / Ink-saving), annotation appendix toggle, and header/footer merge fields. The browser print dialog opens after.
- **PowerPoint deck (.pptx)** — workshop-ready `.pptx` with one slide per major section: cover (doc title + diagram type), System Scope (if filled), an embedded screenshot of the canvas, EC conflict statement (EC-only), paginated reasoning bullets (≤7 sentences/slide in topological order, assumption-edges filtered), Likely Core Driver(s) (CRT-only), and Method-checklist progress. Indigo brand band; vector content where possible. Generated client-side via lazy-loaded `pptxgenjs` — first invocation downloads the vendor chunk (~123 KB gz), subsequent exports are instant.

The PDF route is the most polished for static layout. Use Print Preview for the layout knobs, the direct PDF export for unattended use, the PowerPoint deck when you need a slide each section of the analysis for a presentation.

Markup

- **Markdown** — outline form. Paste into docs, GitHub, Notion.
- **OPML** — outline form. OmniOutliner, Bike, Logseq.
- **DOT (Graphviz)** — for tooling pipelines.
- **Mermaid** — both export and import; round-trips with the Mermaid live editor.
- **VGL (declarative)** — Flying Logic-flavored declarative text.
- **Flying Logic XML** — round-trips with Flying Logic.

Reasoning + Workshop

- **Reasoning narrative** — Markdown, sentence-per-edge prose. See Chapter 15.
- **Reasoning outline** — Markdown, nested-list form.
- **CSV** — entities + edges + groups in one RFC-4180 file.
- **Annotations only (.md / .txt)** — just the description bodies, for content-review workflows.
- **EC Workshop Sheet (PDF)** — one-page PPT-style layout with guiding questions, EC-only.
- **Risk register (CSV)** — one row per UDE in the doc, with columns `risk_id / risk / trigger / consequence / mitigation / owner / status`. The exporter walks each UDE backwards through the causal graph to find any reachable injection or desired-effect; if one exists the row is `mitigated`, otherwise `open`. Owner comes from `entity.owner` (with a back-compat fallback to `entity.attributes.owner.value` for older docs). Imports cleanly into Jira / Linear / a spreadsheet. Only surfaces in the Export picker when the doc has at least one UDE — NBR diagrams and CRTs are the canonical sources; an EC has no UDEs by construction so it's hidden there.
- **Standalone HTML viewer** — single self-contained file.
- **JSON** — canonical schema-versioned export; the most lossless format.
- **Redacted JSON** — content-stripped JSON. Same structure, generic titles. Useful for sharing the *shape* of an analysis without the confidential content.

Importing back

Going the other direction — opening someone else's file — uses the single **Import…** picker (`Cmd+K → Import…`). The dialog fans out five sources as cards: **TP Studio JSON** (full round-trip), **Flying Logic file** (`.fll`), **Mermaid diagram** (`.mmd`), **Entities CSV** (append rather than replace), and **Paste from whiteboard** — the last is the escape hatch for Miro / Mural / FigJam / any text source. Copy stickies from the source board, paste into the textarea, one entity is minted per non-empty line. Bullet markers (`-` `*` `•` `1.` `1)`) are stripped, tab-separated content keeps only the first column. Connectors aren't inferred — Miro / Mural don't expose arrow structure in client-accessible exports, so this path gets the entities into the canvas; you wire causality after.

Save to file / Open from file

On Chromium browsers (Chrome / Edge), a trio of commands lets you work with a *real file on disk*. `Cmd+K → Open from file…` reads a `.tps.json` into a new tab. **Save to file** writes the current document back — and the first save (or an open) *remembers* the file, so every **Save to file** after that re-writes the same file **in one click, no dialog**. **Save to file as…** always opens the picker, for stashing a copy somewhere new. A small link-chip beside the document title shows which file you're bound to. It's all purely additive — the localStorage auto-save, the tabs, `Cmd/Ctrl+S`, and the Export/Import pickers behave exactly as before; this just adds a file on disk as a target for the same JSON.

> 💡 ***Put your trees on OneDrive.*** *Save into your synced* `OneDrive\…` *folder and the OneDrive client backs the file up and syncs it across your devices — no account linking, no setup.* ***Open from file…*** *the same file on another machine, edit, then* ***Save to file*** *to write straight back where you left off. (On Firefox / Safari, the commands are hidden — use* ***Export → JSON*** *and* ***Import…*** *for the same file, downloaded and uploaded.)*

Generating a diagram with an AI assistant

Sometimes the fastest first draft isn't drawing — it's *describing*. TP Studio ships a Claude **skill**, `tp-studio-import`, that turns a problem, goal, conflict or plan written in plain language into an importable TP Studio document. Tell Claude what you're looking at — "*a Current Reality Tree for why onboarding churns*", "*turn this dilemma into an Evaporating Cloud*" — and it emits a schema-valid `.json` for any of the nine diagram types, which you load through the same **Import… → TP Studio JSON** doorway above.

The skill lives in the repository at `.claude/skills/tp-studio-import/`, so it's available automatically to anyone using Claude Code inside the project; it can also be installed into Claude.ai or a personal Claude setup (its `SKILL.md` explains the format, `reference/format.md` the full schema). A guard test (`tests/skills/tpStudioImport.test.ts`) imports every bundled example through the real validator on each CI run — and fails if a new diagram or entity type is added without updating the skill — so the generated shape can't drift from what the app actually accepts.

Treat the output as a *first draft*. The assistant gets the entities and the causal skeleton onto the canvas in seconds; you still apply the CLR scrutiny of Chapter 13 that turns a plausible-looking tree into sound logic.

The U-Shape — linking trees into one journey

A folder of separate trees isn't a Thinking Process; the *journey* is. Cohen's spine is the **U-Shape**: pinpoint the **core problem** on a Current Reality Tree, surface the conflict underneath it as a **Core Cloud**, break the cloud with an **in-**

jection, and check that injection's consequences in a **Future Reality Tree**. TP Studio stitches those separate documents into that one reasoning flow — without changing how any single tree is drawn.

Three opt-in moves, all on the palette (`Cmd+K`):

1. **Mark as core problem.** Select the CRT entity you've concluded is the core problem and mark it (also a one-click toggle in the inspector). This is *your* call — distinct from the app's computed "core driver" suggestion; it's the hinge the rest of the journey pivots on.
2. **Create the Core Cloud from this entity.** Opens a fresh Evaporating Cloud in a new tab, pre-tagged as a *Core cloud* and titled after the problem, already **linked back** to the CRT entity. Fill the five boxes, find the breakable assumption, draft the injection.
3. **Carry this into a new FRT.** From the injection, open a Future Reality Tree in a new tab with the injection seeded in, again linked back. Now grow the desired effects and trim any negative branch.

At every hop you get a new tab plus a reciprocal **"Linked to"** chip in the inspector. Click a chip to jump straight to the partner entity in its tab — so you can walk **CRT problem → Core Cloud → FRT injection** (and back) one click at a time, the U-Shape made navigable. The links are pure navigation: they add no causal arrows and change nothing about any individual diagram.

Share links

`Cmd+K → Copy read-only share link`. Generates a URL that:

- Has the format `https://your-tp-studio-host/#!share=<base64-gzipped-json>`.
- Encodes the entire current document in the fragment.
- Never reaches a server — the fragment after `#` is client-side-only.
- On the receiver: TP Studio detects the `#!share=` fragment on boot, decodes, loads the doc, and **engages Browse Lock automatically** so the receiver can't accidentally commit edits to a foreign document.

Trade-offs:

- **Size**: a typical 20-entity CRT lands under 2 KB compressed; a 200-entity Goal Tree might push past 8 KB and trip length limits in some email clients. A toast warns when the link exceeds ~4 KB.
- **Visibility**: the fragment is in the receiver's browser history. Treat shared diagrams as "public enough to email" — same threat model as JSON export.
- **Hostility defense**: Session 98 added a 5 MB ceiling on the decompressed payload to defend against gzip bombs.

Print preview

`Cmd+K → Print / Save as PDF`. Opens the **Print Preview Dialog** with:

- **Mode picker:** Standard (full color), Workshop (bold high-contrast), Ink-saving (no fills, lighter strokes). Each renders the same diagram differently for context.
- **Annotation appendix toggle:** include an addendum listing every entity description as a numbered footnote.
- **Header / footer:** plain text + `{title}` / `{pageNumber}` / `{pageCount}` merge fields.
- **Selection-only toggle:** print only the currently-selected entities (renders with non-selected nodes set to `visibility: hidden` so the canvas geometry stays intact).

Browser print dialog opens once you click Print.

The standalone HTML viewer

The most underrated export. `Cmd+K → Export → Standalone HTML viewer` generates a single `.html` file that contains:

- The full TP Studio canvas runtime, bundled.
- The current document, base64-embedded in a `<script type="application/json">` tag.
- The current theme.

Double-clicking the file opens it in any browser. No network calls. No JavaScript dependencies. Read-only — the viewer has no edit gestures wired up.

Perfect for: airgapped audiences, security-conscious recipients, slides with embedded HTML, archival.

Review comments

Sharing a diagram for feedback used to mean the feedback came back *somewhere else* — a reply email, a Slack thread, a track-changes Word doc — detached from the diagram it was about. **Review comments** keep the feedback inside the file.

A comment is a short note pinned to one of three places: an **entity**, an **edge**, or the **whole diagram**. Open the panel with the speech-bubble button in the TopBar (or `Cmd+K → Comments`), select the thing you want to talk about, type, and post. With nothing selected the comment attaches to the diagram as a whole.

Because comments live in `doc.comments` — part of the document, not a side-channel — they travel with **every** lossless route out: JSON export, the read-only share link, and the standalone HTML viewer all carry the threads with them. The async-review loop becomes: share the link (or send the JSON) → the reviewer pins objections to the causality where it's wrong → they send it back → you open it and see each note attached to the exact entity or edge in question.

Each thread supports one level of **replies**, a **resolve / reopen** toggle, and inline **edit / delete**. The panel filters to **Open**, **Resolved**, or **All**, so a review pass is simply "work the Open list to zero." Click a thread's anchor chip to jump the canvas to whatever it's pinned to. Comments are signed with the name you set in the panel's *Signing as* field — a local label, not a login; TP Studio has no accounts.

Two boundaries worth knowing:

- **Comments are plain text.** No formatting, no `@`-mentions, no notifications — they're review notes, not a chat system.
- **Deleting the anchor deletes the comment.** Remove an entity or edge and any comments pinned to it go with it (whole-diagram comments stay). That keeps the thread list honest — every open comment points at something that still exists.

Sidebars

> 🛠 ***How TP Studio helps***
>
> - `Cmd+K → Export` — *unified Export Picker.*
> - `Cmd+K → Copy read-only share link` — *fragment-encoded URL share.*
> - `Cmd+K → Print / Save as PDF` — *Print Preview Dialog.*
> - ***Standalone HTML viewer*** — *self-contained share artifact.*
> - ***EC Workshop Sheet PDF*** — *one-page handout for EC workshops.*
> - ***Browse Lock auto-engages on share-link load*** — *receivers can't accidentally edit.*
> - ***Review comments*** — *notes pinned to an entity / edge / the whole diagram, carried inside every JSON / share-link / HTML export.*
> - `tp-studio-import` ***Claude skill*** — *describe a problem in words; get an importable JSON for any of the nine diagram types.*

> 💡 ***Practitioner tips***
>
> - ***Capture a snapshot before exporting for stakeholders.*** *"What I showed them on Tuesday" is a revision you'll want later.*
> - ***PDF for static audiences, share link for interactive ones.*** *Stakeholders click PDFs; analysts click links.*
> - ***Use redacted JSON when sharing the shape of an analysis.*** *"Here's our diagnostic shape, with anonymized content" is a real workflow.*
> - ***The EC Workshop Sheet is great handout material.*** *Print one per participant.*
> - ***Use comments for async review.*** *Share the link, let reviewers pin objections to the exact edge, then work the Open filter to zero.*

> ⚠ ***Common mistakes***
>
> - ***Sharing the wrong revision.*** *When you send a link, the link encodes the current doc state — not whichever snapshot you thought it would be. Capture + restore the right revision first.*
> - ***Forgetting Browse Lock when demoing locally.*** *Click the lock icon before screen-sharing. Otherwise an errant double-click during the demo creates a stray entity.*

🔁 **Chain to next:** sharing is the asynchronous mode. Workshops are the synchronous one. Next chapter covers the latter.

→ Continue to Chapter 17 — Workshops with TP Studio

Chapter 17 — Workshops with TP Studio

> *TOC workshops have a particular shape. Three to eight participants, one facilitator, four to six hours, one CRT or one EC built collaboratively. The patterns that make them work are mostly about discipline and rhythm; the tool plays a supporting role.*

Setting up the room (or the call)

Before the workshop opens, you'll want:

- **Browse Lock OFF.** You're going to edit live. Make sure the lock icon is unlocked. (Lock it during breaks if you're walking away from the laptop.)
- **Reading Instructions strip visible** on EC docs — `Cmd+K → Toggle EC reading guide` if it's currently hidden. New participants need the 1/2/3 reminder of EC reading direction.
- **System Scope filled.** Document Inspector → System Scope section. Answer the seven CRT-Step-1 questions before you start. "What system are we analyzing?" "Who are the stakeholders?" "What's the boundary?" The workshop will be 30% less productive if you skip this.
- **Method Checklist visible** in the Document Inspector. The canonical recipe is right there; refer back to it during the workshop to keep the group on the rails.

For remote workshops:

- **One person drives.** Screen-shared. Everyone else watches.
- **Use a video call with view-only screen share, not a tool that lets remote participants click on your canvas.** Multi-cursor editing of a CRT is chaos.
- **Capture snapshots at every transition** — between CRT and EC, before the negative-branch hunt, before re-reading. The snapshot history is the workshop's memory.

Facilitator gestures

Five gestures the facilitator uses repeatedly:

1. **Zoom-up annotation.** When a participant says "wait, that one — the third one from the left," hover over the node and let the zoom-up overlay surface its full title and description. Solves the "which entity?" problem instantly.
2. **Walkthrough overlay** (`Cmd+K → Start read-through`). When the group has been building for 20 minutes and is starting to lose the thread, switch to walkthrough. Reading the diagram aloud one edge at a time *re-centers* the group on what's been said.
3. **Side-by-side compare.** When a structural choice is being debated ("should we group these as AND or as two separate causes?"), branch first, try option A, snapshot, restore, try option B, snapshot. Now compare the two. The right answer is often obvious; the wrong answer was a function of having only seen one.
4. **CLR walkthrough.** Near the end of the workshop, run `Cmd+K → Start CLR walkthrough` to surface every open reservation. Address each as a group — even the dismissals are conversations.
5. **EC Workshop Sheet PDF.** For EC workshops, generate the workshop sheet PDF and print copies before the session. Each participant gets one; they write candidate assumptions on it during the cloud-construction phase, you transcribe the best onto the canvas.

6. **Park objections as comments.** When a participant raises a doubt the group isn't ready to resolve — "I'm not convinced that cause is *sufficient*" — drop a comment on the edge (`Cmd+K → Comments`, or the speech-bubble button) rather than stalling the build. The note pins to the exact edge and stays out of the way. At the closing CLR pass, open the Comments panel's **Open** filter and work through every parked note as a group — resolving each one is its own small conversation. Nothing real gets lost to the pace of the room.

A 4-hour CRT workshop — example agenda

This is one viable shape. Adapt to your context.

Time	Activity	TP Studio gesture
0:00–0:15	Intro + ground rules. Verbalisation discipline explained.	—
0:15–0:30	System Scope. Fill the seven questions live.	Document Inspector → System Scope
0:30–1:15	UDE brainstorm. Each participant offers 1-3 UDEs. Capture as `Undesirable Effect` entities.	Double-click + Type → UDE
1:15–1:30	Break + verbalise. Read aloud all UDEs in sequence.	—
1:30–2:30	First cause chain (highest-impact UDE). Ask why. Build downward. Capture snapshot when done.	`Cmd+K → Capture snapshot`
2:30–2:45	Break.	—
2:45–3:15	Second cause chain. Look for convergence with the first.	—
3:15–3:45	Find core driver. CLR walkthrough. Dismiss with notes.	`Cmd+K → Find core driver`, then `Start CLR walkthrough`
3:45–4:00	Walk through final CRT. Capture final snapshot. Export reasoning narrative for distribution.	`Start read-through` then `Export → Reasoning narrative`

The EC workshop is shorter — typically 2 hours — but follows the same shape: scope, build, verbalise, validate.

What goes wrong, and what to do

- **One participant dominates.** The verbalisation discipline helps. So does rotating who reads aloud. If one voice is over-claiming, hand them the read-aloud job for the next five minutes.
- **The group disagrees on a cause.** Branch the doc and explore both. Reconverge at the snapshot.
- **A cause-claim "feels wrong" but no one can articulate why.** That's a CLR check waiting to be raised. Open the Inspector's Warnings list on the contested edge; usually a `cause-effect existence` or `cause-effect reversal` warning is already firing. If none fires, drop a comment on the edge so the doubt is captured and revisited at the closing pass rather than lost.
- **The workshop runs long.** Snapshot, set a hard stop, schedule a follow-up. A half-finished CRT is more valuable than a forced-completion one.

- **The CRT looks like a project plan.** You're confusing CRT (diagnosis) with PRT/TT (planning). Re-set: this diagram is *why things are the way they are now*, not *what we'll do about it*.

Sidebars

> ***🛠 How TP Studio helps***
>
> - ***System Scope dialog*** *(Document Inspector) — "Step 0" for any analysis.*
> - ***Method Checklist*** *— the canonical recipe per diagram type, always visible.*
> - ***Zoom-up annotation*** *— readable titles at any zoom.*
> - ***Walkthrough overlay*** *— re-centers the group on the diagram's reading.*
> - ***Side-by-side compare*** *— for structural disagreements.*
> - ***CLR walkthrough*** *— final discipline pass.*
> - ***EC Workshop Sheet*** *— printed participant artifact.*
> - ***Browse Lock*** *— read-only mode for demos and screen recording.*
> - ***Capture snapshot*** *— workshop memory.*
> - ***Review comments*** *— park objections on the exact entity/edge during the build; resolve them in the closing pass.*

> ***💡 Practitioner tips***
>
> - ***Capture a snapshot every 20 minutes*** *in a workshop. Even unlabeled ones — you'll appreciate the rollback option.*
> - ***Read the diagram aloud at least three times*** *in a 4-hour workshop. Once after first cause chain, once after second, once at end.*
> - ***End with the exports.*** *Send the reasoning narrative + a PDF of the final CRT to participants within 24 hours.*
> - ***Collect async review with comments.*** *Send the JSON or share link to stakeholders who couldn't attend; their comments come back pinned inside the file, ready to work through.*

> ***⚠ Common mistakes***
>
> - ***Skipping System Scope*** *because "we all know what we're analysing." You don't all know. The first 15 minutes are not optional.*
> - ***Multiple cursors on the canvas.*** *Cooperative editing of a TOC diagram does not work. One driver, one canvas.*
> - ***No verbalisation during build.*** *Just typing into entity titles is not a workshop; it's a brainstorm with a fancy outline tool. The aloud-reading is the gesture.*

🔁 **End of Part 4.** The appendices are reference material — keyboard shortcuts, CLR rule details, settings, a worked end-to-end case study, glossary, further reading.

→ Continue to Appendix A — End-to-end case study

Appendix A — End-to-end case study

Customer-support firefighting

> *The worked example used in Chapters 4–6 is sketched here as one continuous narrative. Read it once in full to see how the CRT, EC, and FRT compose into a single analysis.*

The scenario

A B2B SaaS company. The product is mature; the install base is ~400 customers paying an average of $40K ARR. Customer Success and Support are organizationally separate; Support has 8 agents (2 L1, 4 L2, 2 L3) and one team lead (formerly L3, promoted 14 months ago).

The VP of CS calls a meeting. Three concerns:

- Renewal pull-through is down. Net retention has dropped from 112% to 94% over the last 4 quarters.
- The Support team is burnt out. Three resignations in the last 6 months, all citing "constant firefighting."
- Cost per ticket — measured in agent-hours per resolution — is up 40% YoY despite no change in product complexity.

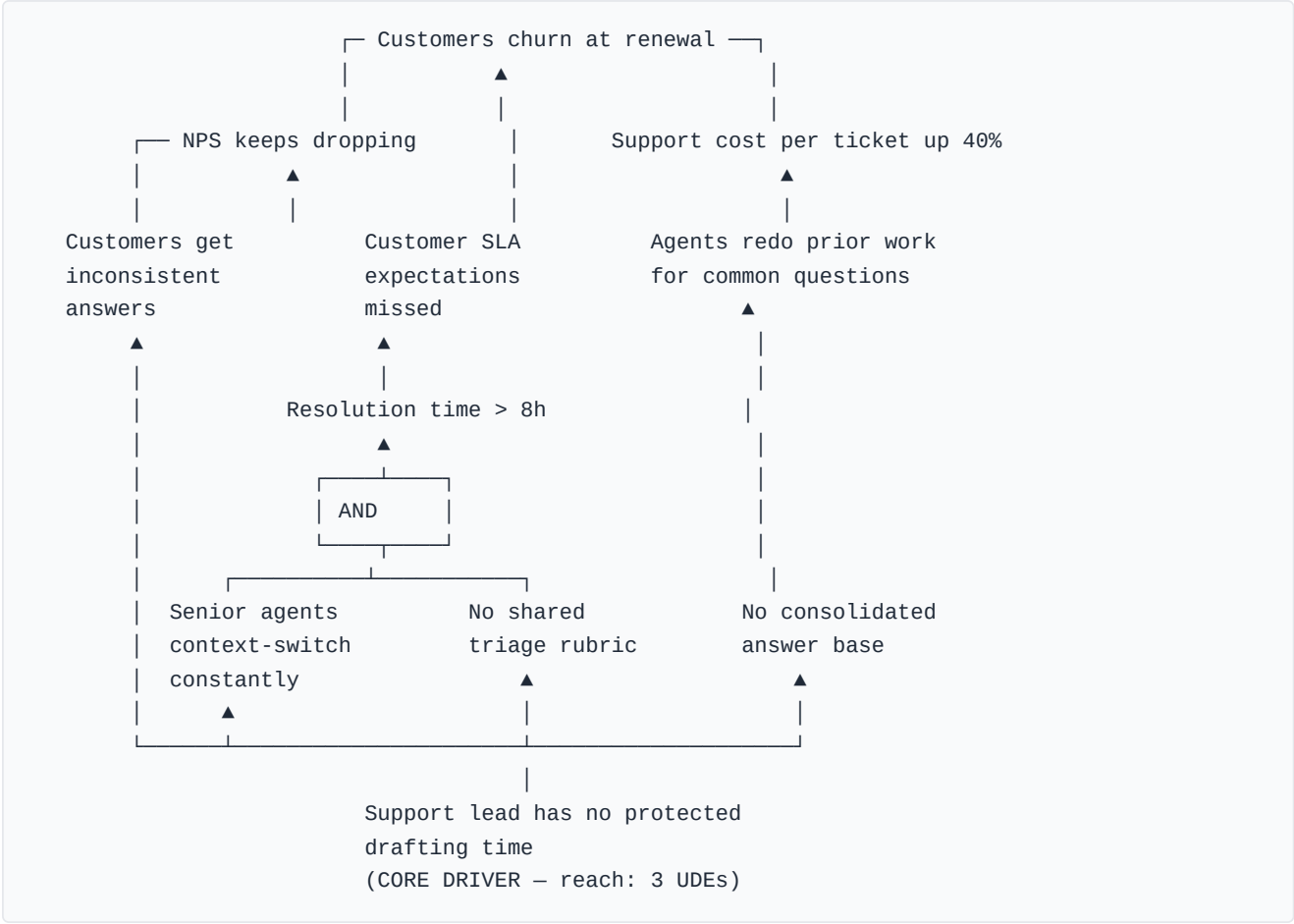
The conventional moves have already been tried. Hiring more agents didn't help. A new ticketing tool didn't help. A re-org didn't help. The VP wants a diagnosis.

CRT

Three UDEs:

- **Customers churn at renewal**
- **NPS keeps dropping**
- **Support cost per ticket up 40%**

Cause chains (built bottom-up):



The structure converges: three UDEs trace through five intermediate effects to two terminal causes ("Senior agents context-switch constantly" and "Support lead has no protected drafting time"), with the lead's protected-time problem feeding three of the four intermediate effects. The core driver is structural — not a hiring problem, not a tooling problem, not a re-org problem. The lead has not been protected from incoming work long enough to build the team's scaling apparatus.

EC

Why has this persisted? The EC.

- **A — Goal:** A sustainable support function.
- **B — Need:** Keep ticket queue responsive to retain at-risk customers.
- **C — Need:** Build durable structure (rubric + answer base) so the team scales.
- **D — Want (satisfies B):** Support lead stays on the queue.
- **D′ — Want (satisfies C):** Support lead takes 2 days/week off the queue to build structure.
- **Mutex:** D and D′ conflict — one person.

The cloud reads aloud:

> *In order to achieve a sustainable support function, we must keep ticket queue responsive to retain at-risk customers; therefore the support lead stays on the queue. In order to achieve a sustainable support function we must also build durable structure (rubric + answer base) so the team scales; therefore the support lead takes 2 days/week off the queue. And these two wants conflict.*

Assumptions on B → D:

- *Only the support lead can resolve the hard tickets.*
- *Tickets sufficient to keep the queue responsive must all be resolved by humans.*
- *We can't temporarily reduce inbound ticket volume.*
- *No structural improvement could pay off within the timeframe leadership cares about.*

Assumptions on C → D′:

- *Building the structure requires the lead specifically — no other agent can do it.*
- *The structure must be built in dedicated blocks, not incrementally.*

The breakable assumption is **"Only the support lead can resolve the hard tickets."** Trained L2 agents could handle the hardest 20% of ticket types, freeing the lead's time without sacrificing queue responsiveness.

The cloud evaporates. Injection: **Train 2 L2 agents on the hardest 20% of ticket types**.

FRT

Does the injection actually work?

Primary chain:

- Inject: Train 2 L2 agents → L2 agents handle hardest 20% → Lead's queue load drops 30% → Lead has 2 days/wk drafting time → Rubric + answer base exist → Resolution time drops, no answer-redoing → SLA met + cost per ticket drops → **CRT UDEs eliminated.**

Negative branches:

- **NB1:** L2 training takes time. Queue load on existing seniors spikes during training. → Inject: Temp contractor for 8 weeks during training.
- **NB2:** L2-resolved tickets might be lower quality than L3. → Inject: 2-week shadowing program before L2s go solo.

Three injections total, ready for the PRT.

PRT (sketch)

Below each injection, the obstacles + IOs. For the primary injection:

- *Obstacle:* No list of "hardest 20%". → *IO:* Audit complete; hardest 20% defined.
- *Obstacle:* L2 candidates have full queues. → *IO:* L2 capacity freed by re-routing.
- *Obstacle:* No curriculum. → *IO:* Curriculum drafted, lead-reviewed.
- *Obstacle:* No budget for contractor. → *IO:* Budget approved.

For the temp-contractor injection: → *IO:* Contractor identified, contracted, onboarded. (Standard recruiting / contracting workflow; mostly off-canvas.)

For the shadowing injection: → *IO:* Shadowing rubric drafted. *IO:* L3 calendars cleared 2 hours/day during shadowing weeks.

TT (sketch)

The most operationally explicit. One example chain from the PRT's "Audit complete" IO:

1. **Pull 6mo of escalated tickets via helpdesk API.** Precondition: API token. Outcome: CSV of ~2400 tickets.
2. **Bucket the CSV by type and compute escalation rate.** Precondition: CSV + categorization rubric. Outcome: Ranked list.
3. **Mark top 20%, circulate to L2 candidates.** Precondition: Ranked list + L2s identified. Outcome: Signed-off list.
4. **Publish list as workspace doc.** Precondition: Signed-off. Outcome: IO achieved.

Each leaf is sized for one person, one day at most.

The result

Six weeks after the workshop:

- L2 training in week 4, all 2 L2s certified on hardest-20% types.
- Lead has had 5 weeks of 2-days-protected time. Rubric drafted, sections 1-3 of answer base done.
- Resolution time has dropped from a median of 11 hours to 6.5.
- One renewal saved that the VP's gut said would have churned. Another is at risk but the underlying complaint shifted from "you're slow" to product-specific gaps — addressable in the next product cycle.

Twelve weeks later, both NB injections also paid out: the temp contractor extended for 4 weeks beyond plan because the lead wanted more drafting time; the L2 shadowing turned into a permanent pairing convention, with one L2 explicitly on a promotion track.

The CRT was a structural diagnosis. The EC named the false assumption. The FRT predicted the result. The PRT/TT planned the rollout. None of these was the answer alone; the chain was.

Appendix B — Keyboard reference

> *Mirrors the Help dialog (`?` icon in the TopBar, or `Cmd+K → Show keyboard shortcuts`). Reproduced here for offline reference.*

Canvas

Shortcut	Action
`Double-click` (empty canvas)	Create a new entity at click point
`Click` (entity / edge / group)	Select
`Shift+click`	Add to selection
`Cmd/Ctrl+click`	Toggle selection
`Alt+click` (on another entity, with one entity selected)	Create edge from selected → clicked
`Drag` (empty canvas)	Marquee-select
`Drag` (handle on entity edge)	Create edge to drop target
`Alt+drag` (entity onto edge)	Splice the dragged entity into the edge
`Middle-click drag` / two-finger scroll	Pan
`Wheel`	Zoom
`+` / `-` / `0`	Zoom in / out / fit-to-view
`Esc`	Cascade dismiss (close palette → close picker → exit search → clear selection)

Selection-driven

Shortcut	Action
`Tab` (entity selected)	Create child below the selection
`Shift+Tab` (entity selected)	Create parent above the selection
`Enter` (entity selected)	Enter inline title edit
`Alt+Enter` (inside inline editor)	Newline in title
`Esc` (inside inline editor)	Cancel without committing the in-progress edit
`A` (edge selected)	Add an assumption to the edge
`Delete` / `Backspace`	Delete selection (confirms when there are connected edges)
`Cmd/Ctrl+C` / `X` / `V`	Copy / cut / paste
`Cmd/Ctrl+Shift+→`	Select all successors
`Cmd/Ctrl+Shift+←`	Select all predecessors

Document-wide

Shortcut	Action
`Cmd/Ctrl+Z` / `Shift+Z`	Undo / redo
`Cmd/Ctrl+K`	Open command palette
`Cmd/Ctrl+F`	Open find panel
`Cmd/Ctrl+,`	Open settings
`Cmd/Ctrl+E`	Palette pre-filtered to Export commands
`Cmd/Ctrl+\`	Close inspector (clear selection)
`E` (no modifiers, not in text field)	Open Quick Capture
`?`	Open Help dialog
`Cmd/Ctrl+T`	New tab *(installed app only)*
`Cmd/Ctrl+W`	Close tab *(installed app only)*
`Cmd/Ctrl+1` – `9`	Switch to tab 1–9, 9 = last *(installed app only)*

> *The three tab keys above fire only in an installed PWA (`display-mode: standalone`). In a normal browser tab those keys belong to the browser, so use the palette tab commands below instead.*

Palette commands worth memorising

Command	Purpose
`New diagram…`	Diagram type picker
`Load example…`	Example picker
`New from template…`	Templates library
`Capture snapshot`	Save a revision
`Open history panel`	RevisionPanel toggle
`Comments`	Review-comments panel toggle
`Add comment on selection`	Comment on the selected entity / edge (or the whole diagram)
`Start CLR walkthrough`	Iterate open warnings
`Start read-through`	Verbalisation overlay
`Find core driver(s)`	Highest-reach root cause
`Spawn Evaporating Cloud from selected entity`	CRT → EC pivot
`Splice entity into selected edge`	Create-and-splice
`Group as AND` / `OR` / `XOR`	Junctor grouping
`Group selected as new group`	Generic group
`Move selection to Archive group`	Quick archive
`Toggle EC reading guide`	EC-only
`Reopen creation wizard`	If you dismissed and want it back
`New tab` / `Duplicate tab` / `Close tab` / `Next tab` / `Previous tab`	Tab management (works in any browser)
`Forget closed documents`	Reclaim storage from documents you've closed
`Copy read-only share link`	Fragment-encoded share URL
`Export`	Open the unified picker

The complete list is in the palette itself — open `Cmd+K` and scroll. Categories are visible at the right edge of each row.

Appendix C — The CLR rules in detail

> *One entry per implemented validator. Tier color and the diagram-types it fires on.*

Clarity tier

The most pedagogical layer. These warnings are about *legibility* — would a reader understand what you mean?

Rule	Fires on	Suggests
`empty-title`	Any entity / edge / group with no title	Add a meaningful title.
`clarity-word-limit`	Entities whose titles exceed ~25 words	Shorten — long titles indicate compound claims that should be split into multiple entities.
`goalTree-multiple-goals`	Goal Tree docs with more than one Goal entity	One-click action: `convert-extra-goals-to-csfs` downgrades all but the oldest to CSF.
`disconnected-graph-floor`	Diagrams with more than 3 connected components	Check whether the components are genuinely separate analyses (split into separate docs) or whether you forgot to connect them.
`goalTree-multiple-goals` *(soft)*	Goal Tree docs with > 1 Goal	Soft warning — dismissible. Reclassify or accept.
`clarity-similarity`	Two entities with > 85% similar titles	Likely duplicates. Merge or rename.

Sufficiency tier

About logical structure — is the causal claim complete?

Rule	Fires on	Suggests
`cause-sufficiency-missing-co-cause`	CRT/FRT edges where the analyst has indicated a sufficient-cause claim that probably needs an AND-group	Group with co-causes.
`complete-step` (TT-only)	Actions in a TT whose outgoing edge to an Outcome lacks a non-action sibling (precondition)	Add the missing precondition.
`st-tactic-assumptions` (S&T-only)	Tactics with fewer than three necessaryCondition feeders	Add supporting NCs.
`ec-missing-conflict` (EC-only)	EC docs where no want↔want edge is flagged `isMutualExclusion`	Tag the conflict edge as mutex.

Causality tier

About direction — is the arrow pointing the right way?

Rule	Fires on	Suggests
`cause-effect-reversal`	Edges where the typed natural reading suggests the arrow direction is wrong	Reverse or rewrite.
`external-root-cause` (CRT-only)	Root causes flagged `span: external`	Push one level deeper — the real root cause is usually within `control` or `influence`.

Predicted-effect tier

About implications — does the diagram make a prediction that should be visible elsewhere?

Rule	Fires on	Suggests
`predicted-effect-existence`	Sufficiency-arrow edges where the predicted effect of A → B suggests something else should also be observable	Look for the predicted second effect; if it exists, confirm the structure; if not, the original claim is suspect.

Diagram-type scoping

Most rules fire only on specific diagram types. The table below maps:

Rule	CRT	FRT	PRT	TT	EC	Goal	S&T	Freeform
`empty-title`	✓	✓	✓	✓	✓	✓	✓	✓
`clarity-word-limit`	✓	✓	✓	✓	✓	✓	✓	✓
`disconnected-graph-floor`	✓	✓	✓	✓	—	✓	✓	—
`clarity-similarity`	✓	✓	✓	✓	—	✓	✓	—
`cause-sufficiency-missing-co-cause`	✓	✓	—	—	—	—	—	—
`complete-step`	—	—	—	✓	—	—	—	—
`st-tactic-assumptions`	—	—	—	—	—	—	✓	—
`ec-missing-conflict`	—	—	—	—	✓	—	—	—
`cause-effect-reversal`	✓	✓	—	—	—	—	—	—
`external-root-cause`	✓	—	—	—	—	—	—	—
`predicted-effect-existence`	✓	✓	—	—	—	—	—	—
`goalTree-multiple-goals`	—	—	—	—	—	✓	—	—

Freeform receives almost no validators by design — see Chapter 11.

Appendix D — Settings reference

> *Everything in `Cmd+K → Open settings`, what it does, when to flip it.*

Appearance

Setting	Default	What it does	When to change
Theme	Light	Light, Dark (4 dark variants: Coal / Navy / Rust / Ayu), High-contrast	Dark for evening work; High-contrast for accessibility / projector use.
Animation speed	Normal	Off / Slow / Normal / Fast	Off for screen recordings; Fast for keyboard-driven work.
Show minimap	Off	Toggles the bottom-right minimap with viewport indicator	On for large diagrams; off for clean exports.
Edge color palette	Default	Default / Colorblind-safe (Wong palette) / Monochrome	Colorblind-safe in mixed-audience workshops. Monochrome for print.

Display

Setting	Default	What it does
Causality reading	Auto	none / because / therefore / auto (per-diagram-type) — fallback edge label when no per-edge label is set. See Chapter 3.
Show UDE-reach badge	Off	Toggles the amber `→N UDEs` pill on each entity.
Show reverse-reach badge	Off	Toggles the sky `←N root causes` pill.
Show annotation numbers	Off	Renders each entity's `annotationNumber` as a small badge.
Show entity IDs	Off	Renders the entity's stable id as a tiny corner badge. Mostly for debugging.

Behavior

Setting	Default	What it does
Default layout direction	Bottom → Top	BT / TB / LR / RL — for newly-created CRT/FRT/PRT/TT/Goal Tree/S&T docs. EC ignores this (manual layout).
Show creation wizard for Goal Trees	On	Auto-open the wizard on new Goal Tree docs.
Show creation wizard for ECs	On	Auto-open the wizard on new EC docs.
Show selection toolbar	On	The floating 3-5 verbs above selected entities.
Open documents in new tabs	On	Importing, or loading a pattern / template / example / shared link, opens a new tab instead of replacing the current document. Off restores the pre-tabs "replace what's open" behaviour.

Layout

Setting	Default	What it does
Bias	0.5	Dagre layout bias parameter — affects horizontal pull.
Compactness	Default	Slider — denser packing vs. more breathing room.

Behavior — advanced

Setting	Default	What it does
Browse Lock on share-link receive	On	Auto-engage Browse Lock when loading a `#!share=` URL.
EC chrome collapsed	On	Hide the EC reading-instructions + verbalisation strips by default. Toggle via `Cmd+K → Toggle EC reading guide` when you want them back for a workshop.
System scope nudge	Off	Show a soft toast on every CRT load reminding you to fill System Scope. (Most users find this intrusive; off by default.)

Appendix E — Glossary

> *Terms used throughout. The TOC tradition is acronym-heavy; this list disambiguates.*

Term	Definition
AND junctor	A combinatorial node in TP Studio rendering "all inbound causes jointly sufficient." Visual: violet circle labeled `AND`.
Assumption	A claim that a causal arrow depends on, and that someone could plausibly challenge. First-class entity since schema v7.
Back-edge	An edge flagged as a deliberate cycle-closer; the cycle CLR rule suppresses warnings on cycles whose closing edge is back-tagged.
Browse Lock	TP Studio's read-only mode. Toggle in TopBar. Auto-engages on share-link load.
CLR	Categories of Legitimate Reservation. The six discipline-checks for evaluating a causal claim.
Core driver	The root cause with the highest UDE-reach in a CRT — the candidate constraint.
CRT	Current Reality Tree. "Why is this happening?"
CSF	Critical Success Factor. Middle layer of a Goal Tree.
D / D′	The two "wants" in an Evaporating Cloud — the actions one side and the other side advocate.
DAG	Directed Acyclic Graph. Most TP Studio diagrams are DAGs (back-edges are explicit annotations of cycles, not structural cycles).
DE	Desired Effect. The FRT's top-of-tree entity (what the system would produce after the injection).
EC	Evaporating Cloud. The 5-box conflict diagram.
Effect	An entity in a CRT/FRT that's caused by something and causes something else — intermediate.
Evidence	A first-class list on every entity (since Session 134): structured rows backing the entity with a `description`, a 5-way `source` (`Observed / Stakeholder / Metric / Policy / Assumption`), a 3-way `strength` (`Weak / Moderate / Strong`), an optional URL, and a per-row validation stamp. Surfaces in the Inspector beneath the Owner block and feeds the `evidence` column of the Risk Register (CSV) export.
FRT	Future Reality Tree. "What would it look like solved?"
Goal	The top of a Goal Tree. Single (typically). Time-bounded (preferably).
Goal Tree	Top-down decomposition: Goal → CSFs → NCs. Strategic-planning shape.
Injection	A proposed change to the system. The hypothesis to test.
IO	Intermediate Objective. A state that, achieved, dissolves an obstacle. PRT-specific.
NA	Necessary Assumption. The first facet of an S&T card — "why this matters now."
NBR	Negative Branch Reservation. A forward-causal sub-tree from a candidate injection that maps its unintended consequences and the mitigation that breaks the chain. Available in TP Studio as both (a) a "Negative Branch" group preset inside an FRT for sub-branch capture, and (b) its own first-class **NBR** diagram type since Session 134 — `Cmd+K → New diagram… → NBR`.
NC	Necessary Condition. Lower layer of a Goal Tree.
NPS	Net Promoter Score. Used in the case study as a UDE signal; not a TOC term.
OR junctor	"Any one of the inbound causes is sufficient." Visual: indigo circle.

Term	Definition
PA	Parallel Assumption. The third facet of an S&T card — "why this specific approach."
Owner	Per-entity free-form text field naming whoever's accountable for the entity (decision owner, action assignee, validation owner). Feeds the `owner` column of the Risk Register (CSV) export.
Pattern library	Curated starter diagrams for common TOC scenarios. `Cmd+K → Pattern library…` lists every pattern with a filter chip row for diagram type. Distinct from "Load example…" which loads one canonical example per diagram type.
PRT	Prerequisite Tree. "What's in our way?"
Risk register	A tabular accounting of identified risks, one per row, with `risk / trigger / consequence / mitigation / evidence / owner / status` columns. TP Studio's **Risk register (CSV)** export (Chapter 16) generates one from any doc containing UDEs by walking each UDE backward through the causal graph to find reachable injections (the mitigations). The `evidence` cell renders the UDE's `evidence[]` entries as semicolon-joined `[strength/source] description (url)` rows. Status is `mitigated` if any mitigation reaches the UDE, `open` otherwise.
Root cause	A terminal cause at the bottom of a CRT — the leverage point.
S&T	Strategy & Tactics Tree. Operational-deployment decomposition with 5-facet cards.
SA	Sufficiency Assumption. The fifth facet of an S&T card — "why this tactic is enough."
Locus	Per-entity flag: `control` / `influence` / `external`. Previously labelled "Span of control" in TP Studio; the schema field name `spanOfControl` is retained for backward compatibility.
Strategy	The second facet of an S&T card — the outcome-shaped "what."
Sufficiency edge	An edge claiming "this cause, by itself, produces the effect." Default for CRT/FRT/TT edges.
Necessity edge	An edge claiming "the effect requires this cause." Default for PRT/EC edges.
Tactic	The fourth facet of an S&T card — the concrete actions.
TT	Transition Tree. Action / precondition / outcome triples. "How do we get there?"
TOC	Theory of Constraints. Goldratt's framework.
UDE	Undesirable Effect. The symptom layer at the top of a CRT — what stakeholders / customers / the market actually feel.
VerbalisationStrip	The above-canvas paragraph rendering of an EC, updates live.
XOR junctor	"Exactly one of the inbound causes occurs." Visual: rose circle.

Appendix F — Further reading

> *The TOC literature. Goldratt's novels first; then the more rigorous works.*

Foundational — Goldratt's novels

These are the canonical entry points. Read in order of publication, not in order of process depth.

- **Eliyahu M. Goldratt, *The Goal: A Process of Ongoing Improvement* (1984).** The original. Manufacturing-focused but the mental model transfers everywhere. The Five Focusing Steps emerge here.
- **Eliyahu M. Goldratt, *It's Not Luck* (1994).** Introduces the Thinking Processes in narrative form — CRT, EC, FRT, PRT, TT. The chapter where the protagonist draws his first Evaporating Cloud is the best onboarding to ECs in any TOC literature.
- **Eliyahu M. Goldratt, *Necessary But Not Sufficient* (2000).** Applies TOC to enterprise software (ERP). Useful for software people.
- **Eliyahu M. Goldratt, *Critical Chain* (1997).** TOC applied to project management. The pattern transfers; the application is dated. Read for the patterns, skim the project-management specifics.
- **Eliyahu M. Goldratt, *The Choice* (2008).** Late-career meta-reflection. Read after the others.

Rigorous / academic

- **James F. Cox III & John G. Schleier Jr. (eds.), *The Theory of Constraints Handbook* (2010).** McGraw-Hill, ~1200 pages. The canonical reference. Each chapter is a topic by a different practitioner. The Thinking Processes chapters by William Dettmer are the deepest single source available.
- **William H. Dettmer, *The Logical Thinking Process: A Systems Approach to Complex Problem Solving* (2007).** Dettmer's expansion of Goldratt's TPs with more rigor and notation precision. The CLR is covered in clinical detail.
- **William H. Dettmer, *Goldratt's Theory of Constraints: A Systems Approach to Continuous Improvement* (1997).** The earlier, leaner Dettmer book. Often a better starting point than the 2007 expansion.
- **H. William Dettmer, *Strategic Navigation: A Systems Approach to Business Strategy* (2003).** Goal Tree and Strategy & Tactics applied to corporate strategy.
- **William H. Dettmer, *Thinking with Flying Logic* (2011).** A practitioner companion specifically for the Flying Logic software. The closest structural parallel to the present book — same shape (a practitioner-facing companion for a TOC software canvas), different canvas. Worth a read if you also use Flying Logic, or if you want to see how the same method-first / tool-second framing reads from inside the other tool's vocabulary.

TOC software lineage

- **Vector Goldratt site.** https://www.goldrattresearchlabs.com/ AGI's research arm. Papers, white papers, training material.

Practitioner blogs and field literature

- **Allan Dabrowski's writing on TOC implementations** — search for his Lean Six Sigma + TOC integration material.

- **Lisa Scheinkopf, *Thinking for a Change* (1999).** A practical workbook for using the TPs. Less novelistic than Goldratt, more accessible than Dettmer.
- **Eli Schragenheim, *Management Dilemmas: The Theory of Constraints Approach to Problem Identification and Solutions* (1998).** Worked TOC analyses across multiple industries; pattern-recognition material.

On the verbalisation discipline

There's no single source. The discipline is implicit in Goldratt's writing (he wrote his TPs as conversations because that's how TPs *work*), explicit in Dettmer's CLR chapters, and varies in name across practitioners ("read-aloud," "scrutinise," "challenge by reading"). The point survives all the terms.

Beyond TOC

Not strictly TOC, but adjacent and useful:

- **Peter Senge, *The Fifth Discipline* (1990; rev. 2006).** Systems Thinking — different vocabulary, similar instincts. The reinforcing-loop / balancing-loop notation is what TP Studio's back-edge feature owes to.
- **Donella Meadows, *Thinking in Systems* (2008).** The plainest introduction to systems thinking. Reads in an evening.
- **Russell L. Ackoff, *Re-Creating the Corporation* (1999).** Pre-TOC systems thinking applied to organizational design. Same instincts, different generation.
- **Edward Tufte, *Visual Display of Quantitative Information* (1983).** Tangentially relevant — the discipline of *what makes a graphic legible* applies to TOC diagrams as much as to any other.

A short reading order

If you want a guided sequence:

1. *The Goal* (Goldratt).
2. *It's Not Luck* (Goldratt).
3. This book.
4. *The Logical Thinking Process* (Dettmer).
5. Selected chapters from *The Theory of Constraints Handbook*.

Three weeks of evening reading, give or take. By the end you have a working TOC mental model that holds up in workshops.